Charlie Vellacott
CENTRE NUMBER: 58 CANDIDATE NUMBER: 9717

Contents

Analysis 6
The Problem 6
The Client and Users 6
Communication With the Client 7
The Current System 8
The Users 9
User Needs 10
Proposed Solution 11
Overview 11
Technology 11
User Interface 12
Algorithms 13
Data Structures 13
Objectives 15
1. Overview Objective 15
2. General Dashboard 15
3. Teacher Dashboard 16
4 Assignment Pages 16
5. Lesson Library 16
6. The In-Lesson Screen 17
7. Cryptography Tools and Demonstrations 17
8. Help & Explanation Screen 18
9. Log-in & Sign-up 18
10. Progress Tracking & Audit Log 19
11. Performance & Error handling 19
Research 19
Alternative Systems 19
General Research 25
Computing Tools/Techniques 28
Alterations 30
Design 32
Overall System Design 32
Application Overview 32
Development tools needed 32
Input, Processes, Output and Storage Diagram 33

Linking Objectives to Design Choices 34
Technical Skills 35
Databases 35
Dynamic UI generation and Windows Forms 35
Algorithms 35
Parameterised API 35
Cross-table parameterised SQL 35
Files 36
Image Data in Lessons Table 36
Aggregate SQL functions 36
OOP – Inheritance 36
OOP – Polymorphism 37
OOP - Encapsulation 37
OOP – Interfaces 37
OOP – Namespaces 37
Event-driven programming 37
Partial Classes 37
Async methods 38
Speech Synthesis 38
Data Structures 38
Simple data types 38
Composite data types 38
Custom classes 38
UI objects 39
System Architectural Design 40
UI layer 40
Business-Logic Layer (BLL) 40
Data Access Layer (DAL) 40
Data Flow Diagram 41
Classes 42
Class Diagram 42
Class Documentation 43
Class Property and Method Design 44
Method Descriptions 46
Attribute Descriptions 52
Database Design 55

Database Table Designs 55
Client-Server System 56
Data Deletion 56
SQL Query Design 57
1. Selecting the assignments to display on the teacher dashboard 57
2. Creating an assignment 57
3. Retrieving assignments 57
4. Adding a new user to the Users table 57
5. Getting all the lesson titles for a given syllabus 57
6. Updating the audit log when someone logs out 57
7. Checking the user's details on log-in 57
8. Adding the user's log-in time to the AuditLog table 57
9. Getting the lesson to display 58
10. Getting the current user's total time spent on the application 58
11. Getting the average user's total time spent on the application 58
User Interface 59
UI Flow Diagram 59
Annotated User Interface 60
Algorithms Design 67
Creating the Ciphers 67
Deep Dive on the SHA-256 Hash 70
UI 70
Algorithm 71
Database 72
Data Security and Robust Programming 72
Parameterised Queries 72
Robustly Closing Connections and Scope Identities 72
UI Authorisation 73
Hashing Passwords 73
Storing Keys 73
Read-Only 73
Unsigned Integers 73
Validation 73
1 Pages 73
2 Demonstrations & Teaching Tools 74
Namespace Design 74

Use of AI in the Database 75
The Query Sent 75
The Response 76
The Query Used 82
Technical Solution 0
Cryptography Classes 0
CryptographicAlgorithm.cs 0
CaesarCipher.cs 4
VernamCipher.cs 6
RailFenceCipher.cs 8
SHAHash.cs 11
Main Classes 16
User.cs 16
Lesson.cs 17
Assignment.cs 18
Data 20
DataClasses.cs 20
UserRepo.cs 20
LessonRepo.cs 26
AssignmentRepo.cs 28
User Interface 35
BaseFormsPage.cs 35
GeneralDashboardForm.cs 37
Lessons 39
Log in & Sign up 59
LoginForm.cs 59
Help Page 63
ChatBot.cs 63
HelpForm.cs 64
Program.cs 66
Testing 0
Generic and Robustness Tests 0
Validation tests 30
Evaluation 40
Summary 40
Objectives & Completeness of Solution 40

User Feedback 0
Overall thoughts 0
Key positive aspects 0
Areas for further development 1
Improvements 2

Analysis

The Problem

Cryptography is a difficult topic to confidently grasp, especially given the lack of existing resources and their ability to accommodate all different learner types. The issue that I have acknowledged relates to this. There are simply not enough resources available to assist both teachers in explaining and students in grasping the complexities of encryption. Although it is possible to find demonstrations or walkthroughs of certain encryption processes, there is a lack of structure to these resources. They tend to be one-off videos or short guides rather than a general hub for all things encryption. Their focus is on the single concept rather than the GCSE or A-Level course.

Clearly, there is a definite need for a structured cryptography teaching tool that is tailored to specific GCSE and A-Level courses. With features that accommodate all learner types and abilities, including providing an interface for teachers to demonstrate and test these concepts with their students. In addition, I have seen no other tools utilise AI for providing instant and personalised guidance in an application. While providing all these features, the priority is engaging students both inside and outside the classroom, including an extension to the A-Level courses, so that students can gain insight into industry applications of cryptography.

Therefore, I aim to use technology to teach 6th-form *(college)* students about encryption. I could implement 3 separate areas inside the application, which will best solve my seen problem:

- GCSE
- A-level
- Post A-level / extra-curricular

The Client and Users

My client, and the individual for whom I will be developing this program, will be my computer science teacher. He has the task of teaching our class the challenging topic of cryptography, and with the current tools available, he mentions that he has a hard time with his explanations while also keeping the class eager to learn.

My general target client will be any college teacher looking to develop their students' understanding of cryptography. They will use the application in school or lectures, except for individuals who are self-learning with this tool. The client will teach encryption, decryption and the algorithms that tie them together. The application should help the teacher in their explanations of these concepts and thus should include clear breakdowns (step-by-step). In addition, they should be able to record and assess their students' knowledge; therefore, my project will need to contain a way of linking the students' work to the teacher. This might involve a space where the teacher can send the students documents to complete.

The students themselves will also be using the application. I will assume that their knowledge can be simply defined by the key stage that they are in, for example, GCSE, A-level or beyond. By doing this, I can ensure that the student has access to the specifics that they will need to know and not overwhelm them with too complicated or too easy tasks within the application. This student will likely try to mess around with the platform; therefore, I need to design the application to be robust. Firstly, I will ensure that it is only the teacher who can set tasks/assignments, as I believe the students would find amusement in setting their friends clusters of tasks. In addition to this, I believe

that the students could use the help feature (which will be discussed later) for other inappropriate uses; therefore, we will customise the queries it is able to respond to and limit the number of tokens it can have.

According to an article from Intel (https://www.intel.com/content/www/us/en//laptops-for-college.html), the average college laptop has around 8 GB of RAM and an i5 processor. Therefore, if I am making a lightweight desktop application, its efficiency is evidently not too much of an issue.

Communication With the Client

I set up a small interview with my client, where I asked him several questions based on both his current systems and how the application should function. The questions are as follows:

What are your main issues when it comes to teaching encryption?

To which he replied, "Explaining the differences between plaintext and ciphertext, and that the encryption part is actually an algorithm". Here, I can see that my application for all algorithms must clearly separate and label the plaintext from the ciphertext. In addition, I should ensure that the lessons show the encryption techniques as a sequence of logical steps that achieve a given result.

What do you currently use for teaching encryption?

"PowerPoint slides and PGOnline worksheets. A Vernam cipher program. Sometimes we use online cipher tools to explore the shift cipher"

What exam boards do you teach for GCSE and A-level? And what encryption techniques do you cover for each?

"GCSE: Pearson Excel – no techniques, just encryption in the context of cyber security

A Level: AQA – Shift, Vernam"

Here, I can see that there are a few techniques that I need to cover specifically for my client. However, I will look to create more for extended studies…

If a student wanted to do further studies into encryption, what should they initially look to learn?

"Maths – number theory, prime numbers, algebra

Computer Science – data structures, algorithms, programming, symmetric, asymmetric, hashing functions. In particular, Railfence."

Although Maths is the work behind encryption, I want to focus my tool specifically on the purely computer science areas of cryptography. Therefore, I will be looking to create lessons on symmetric/asymmetric encryption, hashing and more.

Where do you think the first place your students go is, when they get stuck understanding or answering a question?

He added that students go to "The textbook [this is the Bob Reeves textbook], Isaac Computing, AI Chatbot, code.org videos". From this, I take away the ease and accessibility of AI, specifically when looking to get help with work. This gives me more reasoning for providing students access to a chatbot.

Would you like your students to have their own access to this application?

My client's response: "Yes, an .exe would be fine" denotes that I do not need to focus on creating any more accessible form, instead on the function of the application.

Are most of your homeworks/assignments in the form of a text/word file?

"Mostly, but some are programming tasks, also use Smart Revise". This answer denotes that most homework tasks are pre-created or, like Smart Revise, automatically mark. Therefore, it is not essential that the program has a form of auto-marking.

The Current System

Currently, my client uses PowerPoint slides as his main method of communicating the content to his students. He has premade resources from PGOnline – a company focused on creating "editable teaching units, books and revision guides".

Looking at the slides on the Vernam cipher, they won't be very intuitive or easy to follow for someone who has never looked at this topic. The images are not clear enough to be able to follow through without guidance from a teacher or someone with an already solid understanding of this cipher (as shown in Figure 1).

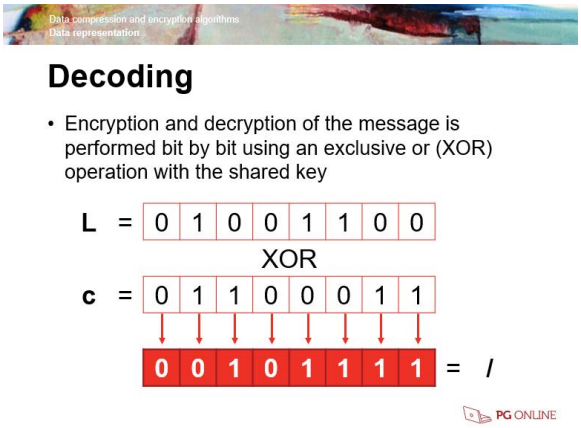


FIGURE 1 –THE PGONLINE SLIDE

However, there is a big upside to using presentation software, and that is that the teaching can become more flexible. Rather than following a rigid course where you learn only what is written, the PowerPoint allows students to freely ask about the subject. Which is great for sparking a genuine passion for what they are learning about, rather than just rote learning. This is heightened by the fact that the teacher could rewrite, add and alter the slides – allowing them to add things that their

In addition, PowerPoint allows the teacher to annotate on the slides while providing discussion (engaging the class) and talking through the complexities of the topic. This is useful to help many different learning types (eg drawings for visual, talking through for auditory, etc). Similarly, the students can easily get a hold of the PowerPoint and annotate it for themselves – allowing for a useful revision resource.

On the contrary, the slides cannot dynamically walk through each step in an efficient way, i.e. they cannot easily create step-by-step explanations without using many slides. These PGOnline slides do not follow this pattern, but some slides can become overly full of information/text.

Finally, they do lack engagement for practical learners; therefore, they must be paired with another resource – for example, worksheets. Which leads to the teacher having to manage a far greater number of resources.

In summary, the pros and cons of using this teaching method are:

Pros

- Based on trusted premade resources.
- Allows for flexible teaching.
- Encourages discussion around the subject.
- Supports multiple learner types.
- Easily shareable and editable for both students and teachers.

Cons

- The lack of ease of following the slides independently.
- Cannot dynamically walk through processes in a highly visual/animated method.
- Some slides can become overloaded with information.
- Offers little practical engagement of physical learners.

Alongside these presentations, worksheets can be used to assess and reinforce knowledge. They involve simpler processes and do not provide instant feedback, but remain a quick, cheap, and simple way to develop an understanding of cryptography.

A more interactive method of teaching the concepts would be videos *(typically on YouTube).* These are usually created by other teachers or experts in the industry and are free to watch. They can be a particularly great method for revising topics, without spending overly long looking over them. However, not all information on them is necessarily correct, and a single creator does not tend to cover all scopes of the topic.

These fundamental techniques are more loosely related to my idea; although they provide a simple method of learning, they aren't suitable for some individuals.

The Users

My users will mainly be the students; however, there are some instances where this would also be the teacher *(sending students assessments and monitoring their work)*. I could split the users into two groups: students (*primary users)* and teachers *(secondary users)*.

The average user will be aged 16-18 and studying A-level computer science. However, they may also wish to go beyond the specification – I will need to ensure it is clear what each different type of user should learn. For those going above the spec, the aim could be to show evidence of extra interest in the subject for the use of a CV or personal statement; my application must allow for this.

They should have a fair understanding of computer science fundamentals; some may already know the Caesar Cipher from the GCSE, but not all. Thus, in the GCSE section of the application, this must be covered. They will struggle with some of the more complex ideas inside cryptography, and as I will be accommodating a range of users, I will ensure that all learning types *(if applicable)* are considered in the method(s) of conveying the topics.

In addition to accommodating different learner types, I will strive to fit all different accessibilities. If possible, I may need to think about colour-blind users – this may be relevant if I use high degrees of colours that are often merged because of being colour-blind. I believe my users, especially as they are in a sixth form/college, will respond well to interactive and engaging resources, rather than a text-dense alternative. The system can be made easier to use through a clear UI:

- There should be an obvious route to getting help from every page of the application.
- Similarly, each user must be able to find their appropriate lesson quickly and intuitively.

This can be done by having a menu that they can select between A-level, GCSE and further study. Also, choosing whether they are a teacher or a student will help simplify this menu.

User Needs

Student Needs:

- Logically sequenced step-by-step explanations for each of the concepts.
- An informative and interactive resource that keeps students engaged.
    - Inspiration could be taken from a simple game *(with achievements and progression bars)*
- A section where users can extend their knowledge above the A-level specification
- The ability to complete tasks to develop their knowledge further.
- Support for all different learning styles.

Student Wants:

- Practical
- Clear UI
- Easy-to-understand explanations
- Ability to test – with quick feedback (LLM)
- Fast access to help

Teacher Needs:

- A tool that will aid the cohesion of their explanations to the students. They should be able to quickly and easily display concepts with several suitable methods.
- Teachers are always busy and swamped with work; therefore, this tool must be a time-saver in many ways. Time should be saved by:
    - Not having to create further resources for teaching the topic of encryption.
    - Reducing the time taken to explain the concepts to students.
- They should be able to upload their exercises (question papers) in a text file format.
- The application must provide a method of tracking the student's progress.
- As teachers will be demonstrating things to students, their general dashboard should be the same. They should be able to access the teacher's dashboard via a button which will open a separate window.
- All data must be stored securely including hashing passwords.

Teacher Wants:

- An off-the-shelf piece of software designed to save them time and improve their students' knowledge of cryptography.
- Having a large language model mark work for them – providing a mark scheme is given, or practice questions on the application, e.g. a quiz.
- Provide a range of tasks for students of different abilities.
- Improve the engagement of their lessons.

Proposed Solution

Overview

The program will provide an interface for teaching a range of users (from GCSE to beyond and outside A-level) with many different prior understandings about cryptography. The application aims to be interactive and like a game (for engagement), so it will have progress tracking in the form of a percentage. The tool will have 3 main functions: teaching and understanding the topic, applying the skills in practical exercises and testing/assessing the knowledge.

- To successfully teach many different learners, the tool must incorporate a method for as many learning styles as possible:
    - Visual, Auditory, Verbal, Physical, Logical, Social, Solitary, Naturalistic

The user should be able to test themselves by completing questions to improve their knowledge, perhaps performing an encryption. Finally, the application must be tailored to the users and clients. Therefore, there should be different sections for: teachers, GCSE, A-level and beyond A-level – where inside each of these areas, they can access everything they need.

The adjacent data flow diagram shows my initial concept. It displays how the different areas of my program could interact – later, I will look to expand this into a more detailed and precise diagram after research.

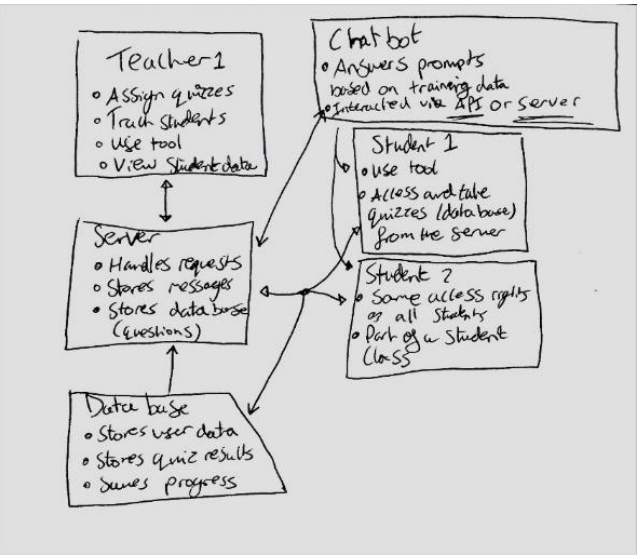


FIGURE 2 – AN INITIAL DATAFLOW DIAGRAM

Technology

I will use C# and Microsoft Forms to create the UI and program the application. The application is intended for a Windows desktop (but, if possible, I could create it for Mac as well). I will be using the OOP programming paradigm as this will help with the code organisation (classes for different algorithms and types of users), managing the UI and much more. If possible, I would be keen to implement a server model that will allow the teacher and students to interact with each other from remote desktops. Similarly, I will utilise OpenAI's LLM with an API to create a personalised chatbot that will help users if they get stuck. I want to have multiple users (e.g. many students per teacher), thus I will be storing these relationships via a database.

User Interface

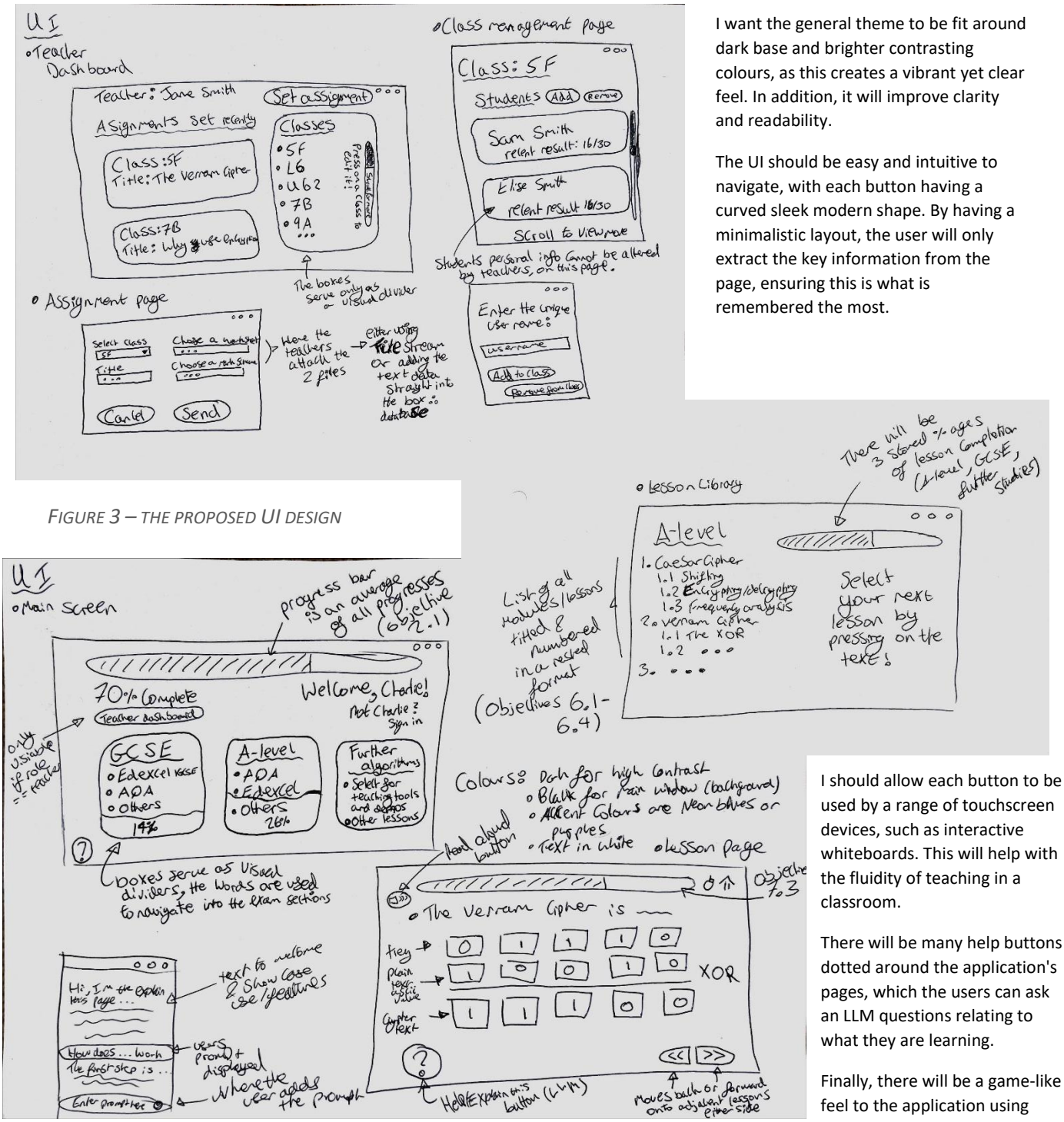


FIGURE 3 – THE PROPOSED UI DESIGN

I want the general theme to be fit around dark base and brighter contrasting colours, as this creates a vibrant yet clear feel. In addition, it will improve clarity and readability.

The UI should be easy and intuitive to navigate, with each button having a curved sleek modern shape. By having a minimalistic layout, the user will only extract the key information from the page, ensuring this is what is remembered the most.

I should allow each button to be used by a range of touchscreen devices, such as interactive whiteboards. This will help with the fluidity of teaching in a classroom.

There will be many help buttons dotted around the application's pages, which the users can ask an LLM questions relating to what they are learning.

Finally, there will be a game-like feel to the application using progress bars.

Algorithms

Cryptography algorithms

These are specification and user-dependent, but some examples may include:

- As per the AQA A-level Computer Science specification:
    - Caesar cipher, Vernam cipher
    - Symmetric vs Asymmetric encryption
        - Public and Private keys
    - *Certificates and signatures*
    - Hashing
- As per the Edexcel iGCSE, only the Caesar cipher is required.
- For the above A-level course:
    - Rail fence could be used to highlight my consideration for the extended course; however, if I were to further develop this application, I would add more cryptography elements here.
    - In terms of hashing I will have a demo for the SHA-256 algorithm
- The demo section will take the following input:
    - *Strings, text files, arrays and lists*
    - *Could implement a binary model, which shows users how the process is carried out on a binary level*

Client-server computations

- I have decided to use SSMS to store the database, so all tables need to be stored in a data base that is accessible on visual studio through SSMS.
- I will need to research how to store files on this database – as my teacher will need to upload, and my student download, assignments.
  Note: After research I have decided to store the binary data of the files in the database

Log-in and User management

- Storing data in an SQL database
    - This includes passwords; therefore, it would be a good idea to hash the data
    - This could be done using the System.Security.Cryptography namespace
- Managing different access rights (using OOP):
    - Teachers:
        - Add assignments
            - Stored as a list of “assignment” objects (which will contain data for marks, original file, completed file and mark scheme)
        - View students' marks
            - Inside the objects in the assignments list
    - Students:
        - Complete assignments and submit them for marking
            - Sending the data will be done with

Other

- Text-to-speech will be done by feeding text into the System.Speech.Synthesis namespace.

Data Structures

General Structures

- Basic structures like strings, integers and floats will be essential for the cryptography demonstrations since the user will need to provide input.
- After researching Railfence I have understood that I will need to create a matrix – I will do this using a 2-dimensional array as I think this is most appropriate, especially given that the matrix will not change size.

- In addition, if I create a hashing algorithm, I will be using 3-dimensional arrays. This will simplify the process of managing blocks, words and bits.
- As I will also be managing binary arrays and converting these into ints I will be using the uint data type. This stands for unsigned integer and is use when the MSB could be a one leading to negatives when not wanted.
- Similarly, I will be storing the files as a list of bytes (the byte[] type in C#) temporarily before uploading or downloading the assignments.
- As part of the LLM chatbot, I will have to use OpenAI's completions object to access past queries and responses.

OOP Structures

- I will need to create an object for the generic user (whether a teacher or not).
    - Then, using inheritance and polymorphism, I can create more specific teacher and student objects.
    - However, I will only do this if there is a great enough difference between the methods and variables being stored for each type of user. Currently I do not think that I need this is needed, as the only difference is being able to access the teacher dashboard or view assignments.
- There will be a few ways I can make the processing of displaying and navigating lessons easier using OOPs. Instead of creating individual forms for each exam board or each lesson I can have a generic lesson forms page that other pages inherit from. This generic lessons forms page will have a flow layout panel for displaying the text/lessons and both help and back buttons. I can use SQL commands to grab the appropriate data and place it into the layout panel.
    - When navigating to the demos I will need separate forms pages. I can use selection to open the appropriate demos page (if the lesson text being clicked on matches the demo).
- In addition, I will need an object for each of my encryption techniques (and possibly the lessons), which will have the appropriate methods associated with it.
    - I will have a generic encryption algorithm which my other will inherit and overwrite where appropriate.
- I could also have various data classes for accessing different tables in the databases. These could all inherit from an overall data class which contains the connection string.

Client-server model

I initially planned on having a single users database, that stored assignments as lists of the object type assignment. This was before studying the databases topic I didn't know about normalised data structures therefore this initial design included non-normalised (non-atomic) data.

After this topic I altered my design following the normalisation process, I finnished with this design:

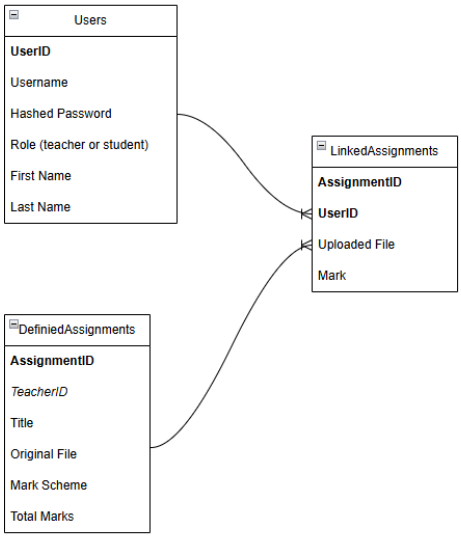


FIGURE 4 – A BASIC AND POTENTIAL IDEA DATABASE DESIGN. NOTE THE VOLUME OF DATA BEING STORED CHANGED MASSIVELY AS THE APPLICATION DEVELOPED.

Note: I had not decided to store the lessons dynamically until the later stages of my analysis. To view the finalised table, please refer to the design section.

Objectives

1. Overview Objective

1.1 My application will be a teaching tool that aims to teach the complex topic of cryptography to students, in a manner that focuses on all types of learners *(eg visual, auditory, etc)*. This can be seen in every section relating to the teaching of cryptography, where a feature for each type of learner must be incorporated *(see 6.1 for measurable outcomes).*

1.2 The application should split the learning process into sections, one for GCSE and another for A-level, and inside these, each exam board and the topics that should be taught. In addition to the GCSE and A-level sections, there should be an extension level.

1.3 The application should provide separate sections for students with varying previous knowledge of the subject to learn as they wish. The sections must cover all knowledge required for that stage, and then the user can choose to learn them as they wish.

1.4 It should be accessible by both teachers and students; therefore, it should also allow the teacher to use the lessons in a presentation manner.

2. General Dashboard

2.1 The user should be able to view their overall progress on the application, as a percentage. This will keep students motivated and well-informed on their achievements/tasks to do.

2.2 It should set aside each exam board, providing quick access to them *(see 3.1)*

2.3 There must be a button, easily visible, which will launch a window where the user can ask an LLM for an explanation or help on a topic relating to cryptography.

2.4 A suitable welcome message should be displayed on the dashboard, the currently logged in user's first and last name should be confirmed.

2.5 Teachers should be able to access a separate dashboard from the main dashboard by pressing a button labelled "Teacher Dashboard" *(see objective 5)*

2.6 In addition to being able to access the lesson library from the dashboard, the user should also be able to locate the encryption tools. These are simple demos of the encryption methods where the user can enter some text and watch the algorithm cypher it.

2.7 Throughout the application, the UI must contain large buttons that can be used on large touchscreen devices such as interactive whiteboards, which are seen in most classrooms today. Therefore, the teacher can navigate the UI using only the whiteboard.

3. Teacher Dashboard

3.1 From this menu, the teachers should be able to view the assignments set.

3.2 These features can be accessed using a button on the teacher's page, which is separate from the typical dashboard.

3.3 The teacher dashboard must only be accessible by teachers.

3.4 The dashboard must display recent assignments showing their date and title.

3.5 The dashboard must display recent results – this is per student and will show the teacher their result for the specific assignment.

4 Assignment Pages

4.1 There must be two assignment pages, depending on the user. A teacher should only be able to access the setting assignment page and not the viewing assignments page.

4.2 The teacher must be able to upload a document, which is the work to complete.

4.3 The teacher must be able to select students based on username in a simple manner.

4.4 If the username doesn’t exist (appropriate validation must occur), they should be told that.

4.5 Equally, measures should be taken to prevent the task being set twice for one individual.

4.6 Similarly, the teacher must be able to remove a user from the list if accidentally placed on it.

4.7 The teacher must also be able to set a maximum mark that can be achieved on the paper.

4.8 The student must be able to view all assignments that have been sent to them.

4.9 And be able to use buttons for downloading the file and uploading their marks.

4.10 The SQL database must be updated as the marks are uploaded.

5. Lesson Library

5.1 The lessons should be sorted in a nested manner, where you can access the correct content by selecting your level, then exam board. By abstracting the process of finding the correct content, we create a more intuitive user interface.

5.2 The lessons must be in an order so that, with no previous knowledge, the user can go from start to finish with each topic being introduced appropriately. This is given that they have completed the previous stage (e.g. for A-level this would be the GCSE stage).

5.3 The lessons should be given an appropriate numbering system, which will help to divide topics further.

5.4 In addition to being given a number, each lesson must also have an appropriate title.

5.5 The users will select their exam board each time they go to choose their content.

5.6 The content must not be duplicated but instead filtered each time *(e.g. the Caesar Cipher lesson in GCSE must be the same as the A-level).*

5.7 All lessons must be generated dynamically, to save time with development and reduce the storage required to install this application *(this is important as at schools' storage and resources are limited by cost)*.

6. The In-Lesson Screen

6.1 Each lesson must be factually correct and contain no more than 250 words.

6.2 Each page of a lesson must contain all the following features to accommodate many different learners:

> 6.2.1 An Image (for visual). 6.2.2 A button to read through the text (for auditory). 6.2.3 A help button (for the practical individuals). 6.2.4 A prompt for discussion (for social learners).

6.3 A help button must be accessible on every lesson page, along with the menu, text-to-speech and other navigational buttons.

6.4 The text (and content) size must be appropriate all throughout the application, but most importantly on the in-lesson pages for easier readability.

7. Cryptography Tools and Demonstrations

7.1 This page must be accessible from the general dashboard via a button, which, when pressed, will give the user a list of encryption techniques to choose from…

7.2 There must be a Caesar Cipher:

> 7.2.1 The cipher should be able to manage any type of text data, even non-alphabetic, and produce a cipher for both.
>
> 7.2.2 If the user enters a non-alphabetic character, it must not have a shift applied to it; the punctuation can stay in place.
>
> 7.2.3 The result must be correct and displayed as a string with all characters shown.
>
> 7.2.4 Negative keys must be accounted for.

7.3 There must be a Vernam cipher:

> 7.3.1 The cipher should accept text data for plain text and produce a correct result no matter the key and text pair. Note, because this is a basic implementation (for ease of teaching), the cipher should not accept a string with any characters that do not have an 8-bit representation (an encoding value about 127).
>
> 7.3.2 If left blank by the user the key will be randomly created each time but must only contain 1s and 0s. The user should be made aware of the key.
>
> 7.3.3 If not, the user must be able to input a key using a text box. Appropriate validation must occur to ensure it is of the correct length.
>
> 7.3.4 The result of this cipher must be displayed in hexadecimal.

7.3.5 The user should not be able to enter a non-hexadecimal result into the cipher-text box and press decrypt. They must be told of the incorrect chars and the upper case must be automatically replaced with lowercase.

7.4 There must be a Rail Fence cipher:

7.4.1 The user must be able to enter a numeric key; it should not be possible for them to enter a non-numeric or non-integer value.

7.4.2 A string with any type of characters can be entered for this cipher, and must produce a correct result.

7.5 There must be a hashing algorithm (SHA-256 hash):

7.5.1 No matter the length of the input the hash must produce a 64-digit hexadecimal result.

7.5.2 All hash results must be returned in hexadecimal.

7.6 Each tool must allow the user to perform both an encryption and a decryption process for each cipher.

7.7 The help and navigational buttons must be visible here

8. Help & Explanation Screen

8.1 The screen should pop up in a separate window, thus not affecting the lesson or task currently being completed.

8.2 It should be closable at any time and feature a modern/sleek layout that ties in with the rest of the program.

8.2.1 The screen must use rounded buttons, contrasting colours and a dark (eye-friendly) colour palette.

8.3 The LLM should provide further resources, like YouTube videos. In addition, the user must be made aware of this through a message when the window opens.

8.4 No matter where the help button has been pressed, it must open the same window *(without providing page context to the query)*; this is because the user will likely be giving context anyway when asking the question.

8.5 The LLM should be given the workings of the application, so that it can help guide users around the UI.

9. Log-in & Sign-up

9.1 The following data must be collected to create a user on the application:

9.1.1 First name 9.1.2 Surname 9.1.3 Username 9.1.4 Password 9.1.5 Role *(teacher/student)*

9.2 The user's password must not be stored in plaintext, instead a hash should be applied and then this stored in the database.

9.3 All details should be stored appropriately in a database – if any information is incorrectly entered in the sign-up page, then the user must be told what the issue is. Incorrect details include:

9.3.1 A non-unique username 9.3.2 Any information left null (empty)

9.4 Typical password requirements must apply:

9.4.1 Longer than 8 chars 9.4.2 Contains at least one uppercase letter 9.4.3 Contains at least one lowercase letter 9.4.4 Contains at least one number 9.4.5 Contains at least one special symbol

9.5 A link for the sign-up window must be logically placed on the log-in page.

9.6 When a user enters a password on either the log-in or sign-up page the password must not be visible – for each character the '*' symbol must appear.

9.7 Appropriate measure must be taken to prevent SQL injection on both pages.

10. Progress Tracking & Audit Log

10.1 Keeping users motivated is important; therefore, there must be a progress bar on the general dashboard based on time spent.

10.2 The users' log-in and log-out times must be stored in an audit log.

10.3 The progress bar must show the user compared to the average with textual data to confirm this. The max value must be at least 2x the mean; any outliers should be plotted at the 100% mark.

10.4 The average total for each user must be computed efficiently *(perhaps using aggregate SQL)*, as this process mustn't slow the application down.

11. Performance & Error handling

11.1 If the SQL database is not available, then a suitable message must be displayed without the application crashing.

11.2 All SQL queries must be embedded inside try catch blocks, in the case of any server issues.

11.3 Media must be loaded efficiently to prevent unnecessarily long loading times.

11.4 Most errors must contain guidance onto how to solve the issues, in non-technical language.

Research

Alternative Systems

CrypTool

There are a very small number of online resources that are dedicated to teaching cryptography, especially in schools. Most resources, such as CrypTool, are focused on interactive self-teaching.

CrypTool is an online platform (https://www.cryptool.org/en/) which has a large library of different puzzles and encryption techniques. It provides some levels of demonstration and explanation for the new users; however, I get the feeling that it is for those more accustomed to cryptography already. On opening the site, you are given a dashboard that has no clear and easy access to the teaching aspects of the program.

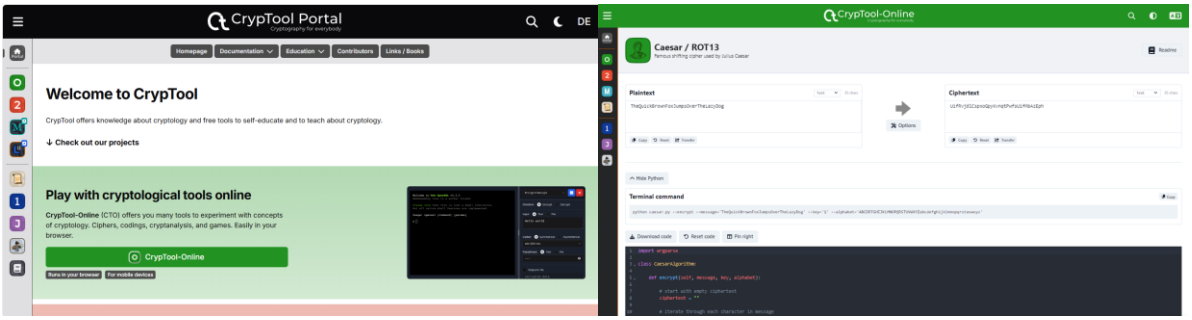
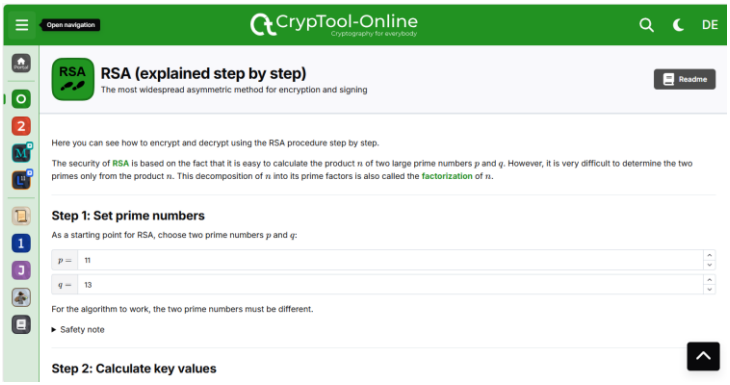


FIGURE 5 – THE CRYPTOOL DASHBOARD AND UI

The site is mainly composed of challenges where the user can place already learnt skills into practice, which is highly helpful for those looking to go above and beyond the A-level specification. This is a positive aspect of this tool as it does, without doubt, encourage students to go further and develop skills that will translate directly to real-world applications of encryption.

The site also focuses on demonstration tools, which I would certainly be keen to implement in my program. The tool can be seen as interactive in the sense that the user can enter plain text to see its ciphertext, and they can see the code that is needed for the encryption (in Python). However, I would argue that (as seen on the right) its method of teaching how each algorithm works consists of lengthy, plain and boring text. This is not how my user will learn.

Some sections do improve on this – like the step-by-step walk-through of RSA, which does appear to be more in-depth and easier to read.



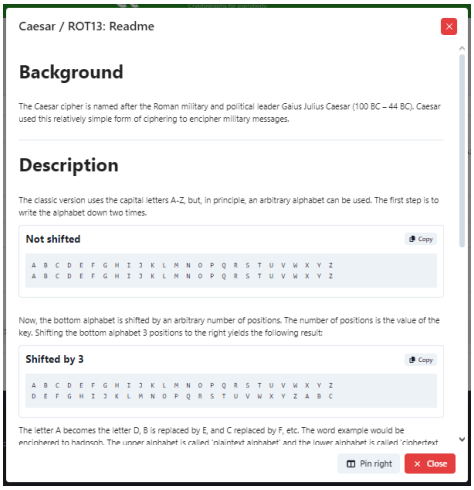


FIGURE 6 – THE TEACHING UI AND METHODS

The learning sections are good however, I would argue that they are a weak point of the application – I found no method of easily following the GCSE or A-level specifications.

In summary, the pros and cons of using this teaching method are:

Pros

- Provides a great extension to the A-level specification, allowing users to gain practical skills.
- Makes the learning process due to the competitive nature of the puzzles.
- The demonstration tools, which feature an IDE and the ability to look through the process of encryption, are highly effective at communicating the processes in a nonverbal manner.
- The focus on using cryptography as not only something to learn but also demonstrating how the skills can be applied helps provide an insight into what it is like working in the field.
- It can't be overlooked that the tool does have a great deal of in-depth explanations, even if they aren't as engaging to follow through as other tools.

Cons

- The tool is hard to navigate and has a non-beginner friendly feel to it.
- Lacking a GCSE or A-level section makes the program less inviting to teachers. Similarly, it doesn't focus on providing teaching tools, which is what I want to base my application on.
- The teaching methodology is only applicable to those who are truly motivated to work, and don't need extra engagement.

CryptoHack

This tool is very similar to CrypTool in the sense that they both have the same purpose: to self-teach and test individuals in the field of cryptography. However, it does differ in its teaching methodology.

CryptoHack (https://cryptohack.org/) is designed to be an intuitive and engaging platform for users to practically apply and develop their skills in cryptography.

The first page which I saw under their menu was the courses section, which, when pressed, opened a list of "lessons" informing the user how to navigate the system and the basics of cryptography. This is a great example of one of the many ways CryptoHack focuses on learning.

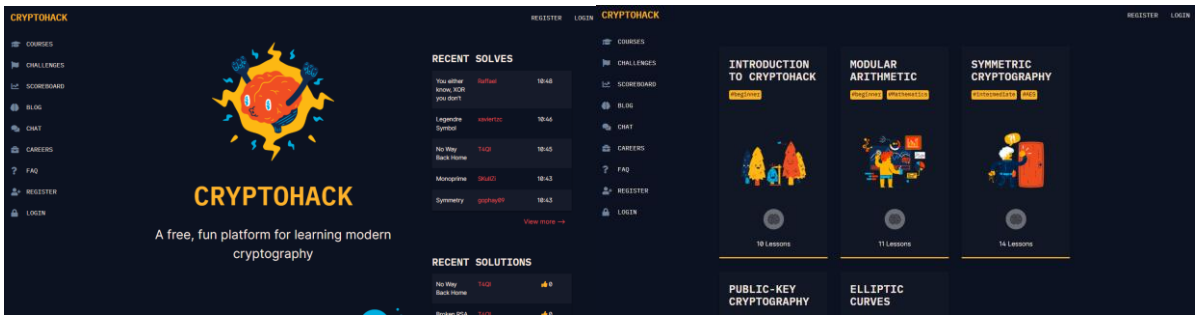


FIGURE 7 – THE WELCOME PAGE AND COURSES PAGE

It places a great deal of focus on ensuring the user has all the foundations needed to understand and successfully apply each of the encryption techniques (as shown below).

This is unlike CrypTool, which jumps around and has no logical formatting for how the user explores cryptography.

In addition, CryptoHack has a Chat and Careers page. I believe these areas are a real boost to the application because it allows users to gain help and explanations for topics they are struggling with, in a social aspect.

CATEGORY	CHALLENGE	POINTS
Introduction - Challenges	Finding Flags	★ 2
Introduction - Challenges	Great Snakes	★ 3
General - Encoding	ASCII	★ 5
General - Encoding	Hex	★ 5
General - Encoding	Base64	★ 10
General - Encoding	Bytes and Big Integers	★ 10
General - XOR	XOR Starter	★ 10
General - XOR	XOR Properties	★ 15
General - XOR	Favourite byte	★ 20
General - XOR	You either know, XOR you don't	★ 30

FIGURE 8 – THE BEGINNER COURSE LESSONS GUIDE

CryptoHack – like CrypTool – has no specific reference to GCSE or A-level courses and no student-teacher interaction. However, I believe it would be easier to learn A-level topics on this site due to the interface and more engaging feel. I believe the gamified aspects of this application really add to the engaging feel. The topics covered in the easier sections of the course are still above GCSE standard, but most of the challenges are great for all levels.

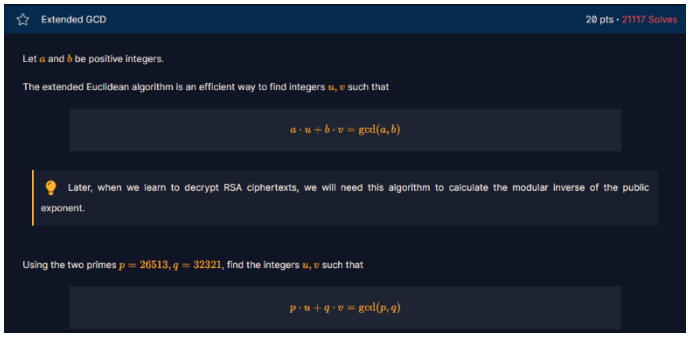


FIGURE 9 – THE IN-LESSON SCREEN

In summary, the pros and cons of using this teaching method are:

Pros

- CryptoHack has a strong focus on logically structuring their learning, teaching the user the fundamentals, and sequentially placing lessons.
- CryptoHack, similarly, focuses on the application of skills in a plethora of areas. Which will help users in the more practical job market.
- The ability to chat to others, asking questions about certain topics, is helpful, especially to those without a teacher to go to.
- The careers resources are excellent, again preparing the user for a job in this sector.
- The gamification of the application is excellent for keeping users motivated and engaged.

Cons

- The focus of this application is mainly on self-teaching resources for after A-level. There is no mention of any GCSE or A-level standard courses. This disregards those in school and especially those completing GCSE computer science.
- There are no features that involve interactions between a teacher and their student (for example, a teacher being able to set tasks for a student).
- The topics covered in the course are above GCSE level, even though the competitions aren't, which prevents the application from being regarded as a teaching tool for GCSE students.

CS Field Guide

Although not an extensive method of teaching, I thought I would include The Computer Science Field Guide (https://www.csfieldguide.org.nz/). It is a tool that is designed for two of the main courses for computer science, in New Zealand (AP-CSP and NCEA).

It provides a collection of simulations for many areas, including a few for cryptography…

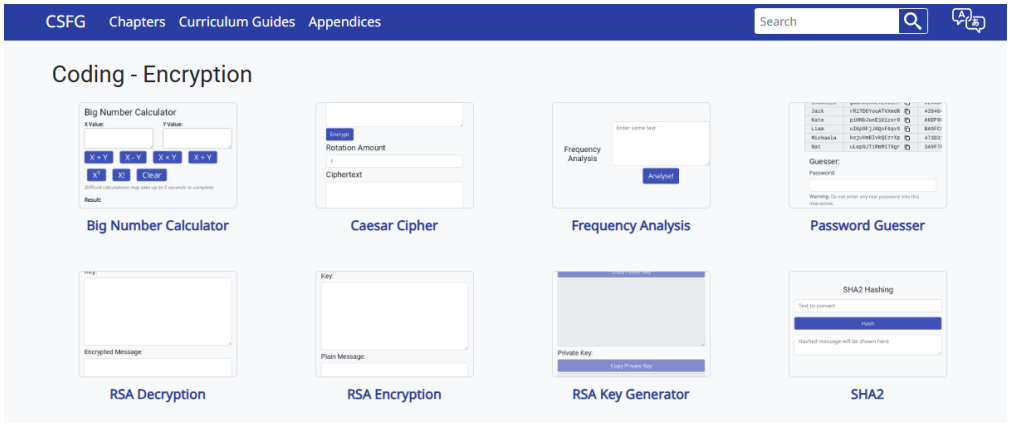


FIGURE 10 – THE ALGORITHMS COVERED

Inside each of these tools is an environment where you can enter plaintext and see the result of the encryption. I found this to be helpful and would have a section of my application like this. However, this website is by no means a standalone method of learning about encryption.

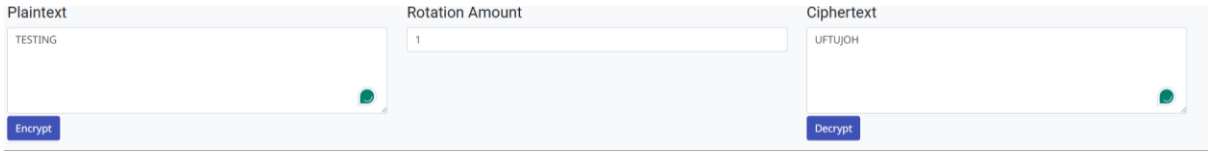


FIGURE 10 – THE TOOL

In summary, the pros and cons of using this teaching method are:

Pros

- This tool is interactive and can be paired with any resources for a great in-lesson demonstration.
- It will be great for those who are visual learners.
- It is excellent for the basic skills and foundations of cryptography, like those taught at the GCSE level.

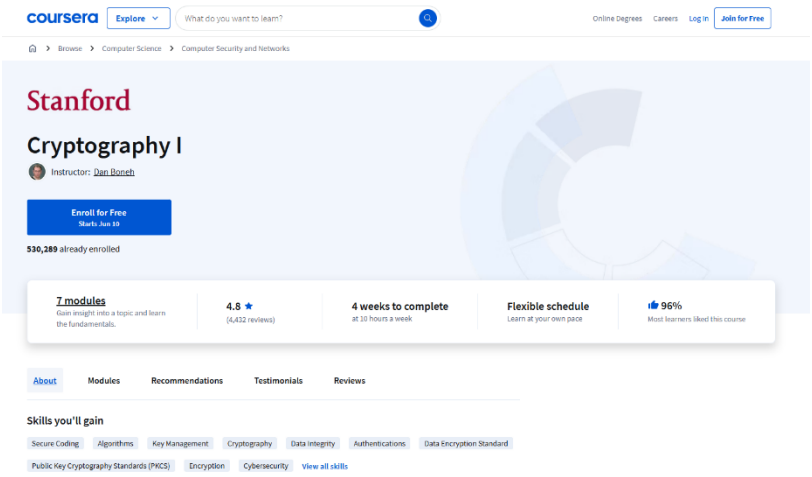
Cons

- This is not a stand-alone learning tool.
- It has some sections for deeper learning but not as extensive as other tools that I have researched.
- There is no ability to track progress or asses their knowledge, this would have to be done separately from the tool.
- The two curriculums that the tools are based on are non-UK, therefore are like GCSE and A-level but have no direct links. Students, if they used this tool on their own, could accidently learn the wrong topics.

Coursera and others

In addition to the web-based simulations and challenges, one could also learn through a traditional course (or MOOC). These are typically free and created by universities to encourage individuals interested in certain subjects to enrol. Usually, they will touch on A-level topics but quickly move well above.

An example of a course I found was Stanford's Cryptography I (https://www.coursera.org/). Others include "An Introduction to Cryptography" by the University of Leeds or "Introduction to Applied Cryptography" by the University of London.



Courses like these are split into modules with individual lessons, typically in the form of a YouTube video. Following the end of a module comes a test where users confirm their knowledge before moving on.

These are a great way of completing further studies, in an independent way, and gaining a qualification (or certificate) as a result.

FIGURE 11 – STANFORD'S "CRYPTOGRAPHY I"

However, with no A-level or GCSE standard, the courses become useless to my client. This is a great example of the lack of notice towards teaching students the basics of encryption.

To conclude, the pros and cons of this application are as follows:

Pros

- The courses are well structured and have a logical sequence of lessons and modules.
- They are created by trustworthy universities, and the information is fact-checked to a high degree.
- Allows for students, again, to learn above specification (however in a less practical method).
- The learning is independent, and where needed, individuals can spend longer looking through things.
- You gain a certificate or qualification at the end of completion.
- Lessons are taught using videos that have been edited and, as a result, are highly engaging.

Cons

- Using courses can lead to quickly going above and beyond the required knowledge for A-level.
- Following on, most of the courses available have no correlation to the A-level or GCSE.
- While entering the course is free, to gain a certificate you often must pay.

Conclusion

My conclusion of this research is that there are very few tools out there that focus on helping teachers convey and students understand GCSE/A-level encryption. Many interactive tools go beyond those that are at the correct level, yet aren't suitable for most/every user to learn from. This is the topic that I am very keen to focus my application on. As demonstrated, the only appropriate tools at this standard are PowerPoints (or textbook pages), YouTube videos and worksheets. Which

shows that my program will hopefully be filling a gap and benefitting many students and teachers in the future.

When looking at these tools, I found many features I want to include in my application. These are the following:

- The gamification aspects, such as progress bars and rewards.
- Logical structure to learning, especially looking at Coursera.
- Ability to communicate with industry experts and ask for help.
- Focus on practical application.
- Interactive simulations, mostly seen in the CS Field Guide.

General Research

The Railfence algorithm

As part of my application, I will be including a teaching tool for railfence – it will be part of the demos section. However, I do not currently know how the algorithm works; I must gain a good understanding before I begin programming it. I will use the page on GeeksforGeeks for research (https://www.geeksforgeeks.org/dsa/rail-fence-cipher-encryption-decryption).

The algorithm is a transposition cypher; this is where the order of the alphabet is rearranged to obtain the ciphertext.

In railfence specifically, the text is written diagonally up and down on adjacent columns of a grid – as shown by the figure below…

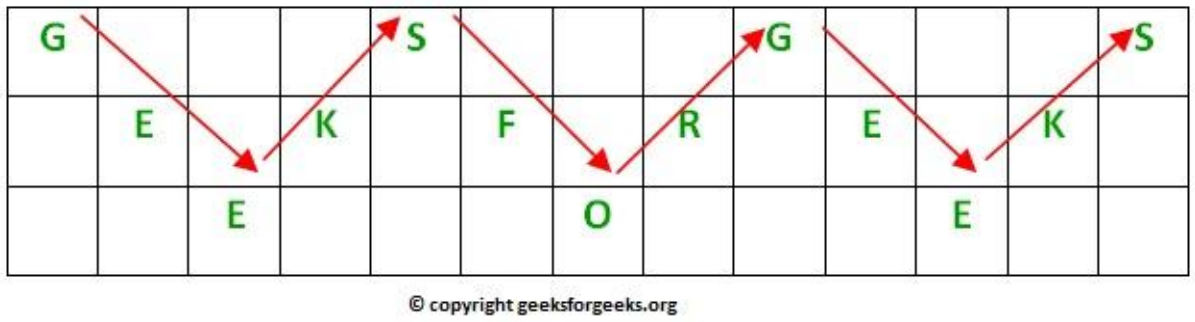


FIGURE 12 – A VISUALISATION OF THE ENCRYPTION PROCESS

The encryption process

Firstly, the matrix (usually a 2d array) is created where the number of columns is the number of characters in the plaintext, and the number of rows is equal to the key given.

The plain text is iterated through character by character, and each character is placed one row above or below the previous, based on whether the upper or lower rail was hit last.

The characters are then placed into a string, as the table is read from left to right, top to bottom. This is the encrypted string.

The decryption process

Firstly, we create the matrix in the same step as above and place the same character in each cell where the “zigzag” would pass through.

Then the matrix is iterated through from left to right, top to bottom. Every time the cell contains the given character, it is replaced with the corresponding numbered character from the ciphertext.

Finally, the matrix is read in the same manner as in the initial encryption stage, and each character is appended to the plaintext string. This should produce the same plain text every time.

How to use SQL Server Management Studio

My application could be used by any individual on any device. Therefore, I will need to store user information, including their assignments, in a database stored in the cloud. To simulate this, I will use SSMS – which is Microsoft's plug-in for Visual Studio that allows me to create, manage and import databases.

Firstly, I will have to create my server and then the database. According to Microsoft's documentation (https://learn.microsoft.com/en-us/ssms/sql-server-management-studio-ssms), I create the connection in the object explorer by clicking the connect button and selecting appropriate connection properties. It appears that for my application, using some of the default properties will be the most appropriate. After this, I can create the database by right-clicking on the server instance in Object Explorer. I can then write my query to create the database by running it.

Then, I can manage my databases by right-clicking on the database in the object explorer and writing an appropriate query. This allows me to create and alter columns/tables in my database.

Connecting the database to my Visual Studio project is simple. After installing the SQL Server plugin, I can then open the Server Explorer panel and click "Connect to Database", before selecting my database.

Finally, I need to be able to write the code needed to make a query. I learnt how to do this by getting an example from my teacher, which displayed the general format for creating a query as shown below…

```
static void ListUsers()
{
    using (var connection = new SqlConnection(connectionString))
    {
        connection.Open();
        string sql = "SELECT * FROM USERS";
        using (var command = new SqlCommand(sql, connection))
        {
            using (var reader = command.ExecuteReader())
            {
                while (reader.Read())
                {
                    Console.WriteLine($"UserID: {reader["UserID"]}, Username: {reader["Username"]}, FirstName:
                                      {reader["FirstName"]}, Surname: {reader["Surname"]}");
                }
            }
        }
    }
    Console.WriteLine();
}
```

Storing files in databases

As my client will want to set assignments for their students, I must have a method of storing the files they wish to send. I want to do this through the database; therefore, I will start by looking at my options.

Firstly, I could save files directly in the database in binary format (as what are known as BLOBs). This is done by creating a VARBINARY(MAX) column. This appears to be the simplest and most effective model as it can easily be incorporated with my already designed database, as seen above. Although my database will need to be further changed to ensure that it is normalised (in first normal form) and does not contain repeated copies of the same file. Which is especially important because they would take up lots of storage and slow down the database.

Secondly, I could use a cloud storage system, such as Google Cloud Storage, or a Web API. However, my very immediate concern is its ease of setup, integration with my current system, and its likely high cost. I believe this is too complex a method for a solution that is not too difficult.

Finally, I could consider making the user input the file data as a string, rather than them uploading the entire file. This would be the simplest option, yet it would still work effectively. This idea could be formed around reading the data from a file (using StreamReader), storing it in the database, and then rewriting it to another file on the user's end (using StreamWriter). The only negative to doing it this way is that only ".txt" files can be read from, as text documents are binary and contain many unreadable values.

Final concept

After researching and evaluating many different methods of storing the files in the database, I have decided to adapt the final method so that it can suit files of many types. This is also an easier-to-implement method that I still believe is highly effective. Using a Binary data type of SQL Server will allow me to store files of up to around 2 GB, which is plenty of space.

Hashing Algorithm

I need to implement a hashing algorithm for both the teaching tool and so that I can store the users' passwords securely. I have decided to learn how to implement a SHA-256 hash, as this is a complex cipher that will really stretch students who want to go above and beyond in the field of cryptography.

How does the algorithm work?

I researched this algorithm using RedBlockBlue's walkthrough (https://www.youtube.com/watch?v=orIgy2MjqrA). The algorithm takes an input of any given length and produces a single 256-bit value. There are three main sections to this algorithm:

1. The padding stage, the input is prepared in such a way that its total length is a multiple of 256 bits.
2. The initial hashing stage, which begins with eight 32-bit hash values H_0 to H_7, where each value is the fractional part of the square roots of the first eight prime numbers. This ensures randomness.
3. The compression stage, the input is taken and broken down into blocks of 256 so that the algorithm can later produce a single 256-bit block as the result. This is done by repetitively performing computations on the blocks and hash values (the output hash from one block becomes the input of the next block).

Computing Tools/Techniques

Visual Studio and C#

I am writing my program in C# as I think this is best suited for an application (as it is for a desktop computer). As a result, I will be using Microsoft's Visual Studio. I am also using C# as this is the language that my school encourages us to use for our Computer Science A-level course; therefore, this coursework will help me become more familiar with the language.

Using OpenAI's Large Language Model API

OpenAI – the founders of ChatGPT – released an API that can be used to provide access to their large language model (LLM). This means that I can use this to place my users' queries into their platform, directly from my application, and obtain a suitable text response. OpenAI call this API a "text in, text out interface", which is perfect for allowing my users to gain fast and detailed responses to questions.

Note: all research was conducted from OpenAI's own documentation (https://openai.com/api).

Using the API in C#

To use the API, I will first need a key. This is a code used to represent a user or product and serves as authentication for the user. I can create this key on the OpenAI Dashboard (once signed in). Then I must securely store this key to ensure that unauthorised access is not possible. I will use a "*.txt*" file for simplicity, which can be created with the key in plaintext, then read & saved using StreamReader.

After the key has been saved, the OpenAI SDK (software development kit) must be installed. This can be done with the .NET CLI from NuGet.

ChatCompletions is the specific API that will be used to generate a model response from a list of messages. A simple request, as given by OpenAI's website, is shown below:

```
using OpenAI.Chat;

ChatClient client = new(
  model: "gpt-4.1",
  apiKey: Environment.GetEnvironmentVariable("OPENAI_API_KEY")
);

ChatCompletion completion = client.CompleteChat("Say 'this is a test.'");

Console.WriteLine($"[ASSISTANT]: {completion.Content[0].Text}");
```

FIGURE 13 – A BASIC CHATCOMPLETIONS REQUEST

The completion.Content[0] is an array of chat messages. By default, it is only one element long, unless specified.

I will be placing this code, above, in a subroutine with an input – a string containing the users prompt – and then this will produce my desired result.

Storing Assignments for each user

Originally, I planned on storing each assignment in the associated user's row in my database. I was going to do this by creating a list of assignment objects, where each object contains variables for completion status, their mark, etc.

However, after going through the relational database and normalisation topic as part of the A-level theory, I realised that this wasn't an effective way of storing this information. This is because the data was not atomic, my newest design is in 3rd normal form and can be seen in the following…

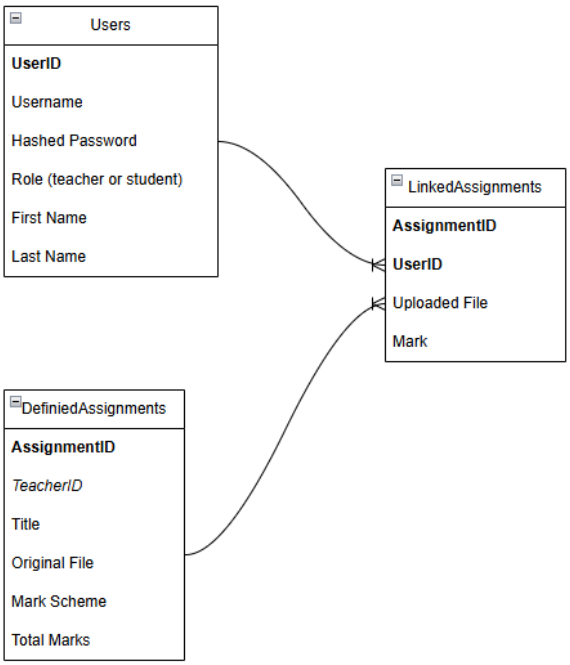


FIGURE 14 – THE CURRENT DATABASE DESIGN

I will now have 3 databases: a user, a defined assignment and a linked database. The user database will also contain details such as the first and last names of each user, along with their hashed sign-in details. The defined assignment database will have each assignment that has been created by any teacher, which is given a unique assignmentID (as a primary key). The linked assignments database will hold the details specific to the assignment specific to the student's assignment, like their mark and completed file.

I edited the layout of my database using the work I did in class on designing normalised databases.

> The reason I did this was to increase data integrity (which means that if information stored on an individual changes, the update only needs to be made once, and there is no chance that the previous value will be associated with the user anymore).

Later, though lead me to realise that if I were to save the binary file to the user's downloads folder, I would have to store the file name and file type in the database. So, I would amend the Defined Assignments database to include the filename, which will include the filename followed by the extension.

Alterations

I spoke to my client again at this point and realised that the scope of my application had increased. I was now creating an application that doesn't exactly fit their needs, adding features which I thought were relevant, but my client noted he wouldn't use these extensively (such as having an LLM mark homework tasks).

Therefore, I changed my objectives and focused the application onto providing a teaching tool rather than an environment for creating/managing tasks and classes. Here are the following objectives that I removed:

> 3.4 The teacher should be able to view all their classes on this page, and if a class is pressed, it should open the class management window.
>
> 4.2 The teacher must upload 2 files: a mark scheme and a test file. They must both be text files.
>
> 4.3 They should specify the class that the assignment is for and give it a title.
>
> 4.4 The results of assignments for each student can be viewed on the class management page.
>
> 4.3 They must then enter all the usernames of their students for whom the assignment is to be set for.
>
> **5. The class management page…**
>
> 5.1 This page can be accessed by clicking on a named class on the teacher dashboard, which will open a window to invite and manage students in the class.
>
> 5.2 Students can be invited by their unique username and do not have to accept the invitation; instead, they are added automatically.
>
> 5.3 The page should open in a separate window and display all the student names in the selected class.
>
> 5.4 The student's latest assignment result should be displayed next to their name.

In summary, my tool will still contain a section for teaching tools and demos of each algorithm, a login page, a lessons page that provides a summary of each tool, and the ability for a teacher to set assignments. However, I have removed the ability for teachers to create classes, as my teacher never mentioned this, and I have changed the techniques that will be featured in the application. Similarly, the LLM will still be featured in my application, but will no longer be used to mark the assignments, only to provide explanations.

GCSE:

- Caesar cipher

Further studies:

- Rail-fence – *as suggested by my client*

Demos:

- Caesar cipher
- Vernam cipher

A-level:

- Caesar cipher
- Vernam cipher
- Asymmetric vs symmetric encryption
- Hashing

- Hashing
- Rail-fence

Design

Overall System Design

Application Overview

The application will be built upon Visual Studio's Windows Forms interface, thus creating a desktop application suitable for the typical school Windows computer. There will be four key areas to the application, with the foundational one being the demonstrations or teaching tools. There will be a section that the user can navigate to, where they can practically 'play' with the encryption techniques covered in the application. These tools will be designed to be used by both teachers and students; therefore, they will be available to all. The next key area of the application is the lessons, which are designed to be used more by the student as a method of independently recapping or learning difficult-to-understand sections of cryptography. They will follow closely to the exam boards, which must be selected by the user as they open the relevant section. In addition, the levels of GCSE, A-level, further learning and teaching tools will be broken down on the dashboard, so they can easily and quickly be accessed/filtered. Each lesson will contain an element for each type of learner, which includes an image, a text-to-speech (speech synthesis) button and a button to access the help window (which is an LLM designed to answer encryption-related questions). Another key section of the application will be the assignment system. Teachers will be able to set groups of students' assignments with files attached. The files and assignment details will then be added to a database and thus be accessible to the appropriate students. Along with viewing the assignment and downloading the file for completion, students will also be able to upload a mark they achieved on the homework, for the teacher to view on their dashboard. The final key area is the audit log and progress tracking. Users will log in to the application with credentials they (or their teacher) have made. The application will then motivate students by tracking their progress on the application, comparing it to that of other users (or the average). This will be displayed in a game-like manner on the user's dashboard, comparing their total time to the average user's total time spent on the application.

Development tools needed

For my application, I will need access to the following:

- Visual Studio *(with C#)* as my IDE for building, programming and compiling the application.
- Windows Forms framework (.NET), which will serve as the foundation to my UI layer, displaying/collecting information to/from the user.
- SQL Server Management Studio (and SQL Server Engine) as the foundations of my database. This will be locally stored as part of this initial prototyping section, but I will later design a system for it to be stored remotely.
    - I will also need ADO.NET for communicating with the database.
- OpenAI's ChatGPT API for the application ChatBot.

Input, Processes, Output and Storage Diagram

Input	Processes
• **Account details**: username, password, role (teacher or student), firstname and lastname. • **Creating assignments:** a **l**ist of students' usernames, the assignment's title, a file (the directory and name need to be saved for the program to upload and the maximum marks that can be achieved. • **Viewing assignments:** the students' marks can be uploaded. • **Cipher demos:** plain text and a key. • **Help page:** any question relating to encryption is input. • **Hashing Passwords:** the user's password when logging in and signing up will be placed into the hashing algorithm	• **Account details:** validate the login, if signing up, create a new entry to the database. • **Creating assignments:** create the entries to the appropriate databases to set the assignments. Access the file from the attached path and save the binary data to the database. • **Viewing assignments:** amend the linked assignment table to save the user's marks. Note: the student will be uploading their own marks for self-assessed homework (no longer using the LLM for marking). • **Cipher demos:** create instances of the appropriate encryption classes to run the encrypt and decrypt methods, and ensure appropriate validation of the user's inputs is in place. • **Help page:** place the users' queries into the API and display the result on the help page. • **Lessons:** the appropriate lesson details must be obtained from the lesson table. • **Hashing passwords:** the user's hashed password will be calculated using the SHA-256 class and then stored in the database.
Storage	**Outputs**
• **Account details:** store a user ID, username, password, role, firstname and lastname for each user entry in the Users table. • **Hashing passwords:** the hashed password will be calculated and stored in the database. • **Creating assignments:** a **l**ist of students' usernames, the assignments title, a file (the directory and name need to be saved for the program to upload and the maxiumum marks that can be achieved. • **Login audit:** a new record must be created for each time a new users logs in. The login time is recorded and the log out time is then added to this record after the user logs out.	• **Account details:** the individuals name must be confirmed on the dashboard, this will help show who is signed in. • **Creating assignments:** each assignment the teacher has set must be visiable on the teacher dashboard, with appropriate information. • **Viewing assignments:** only the student should be able to get to the viewing assignments page and upload there marks. • **Lessons:** the lesson details must be displayed after getting them from the database. • **Cipher Demos:** the results of any cipher must be displayed, where in binary the result must be displayed as hexadecimal.

Note: The above diagram was created as an initial idea or scope for the project – during further design and development various limiting factors did come into play and other more essential features emerged. Thus, the plan for the function of the algorithm should be taken lightly.

Linking Objectives to Design Choices

Objective	Design Choice
9.2 – The password must not be stored in plain text	Will create a SHA256Hashing algorithm which inherits from CryptographicAlgorithm and overrides the IHashingAlgorithm interface for the Hash method. Hashed passwords will then be stored in the tables.
9.7 – Preventions must be in place to stop SQL injection	Will use AddParameterWithValue() method to add values into a query string rather than string interpolation.
3.3 – The teacher dashboard must only be accessible for teachers	The User object will contain a Role property, and each user's role will also be stored in the database. Thus, if a user does not have the Role set to 'Teacher', they should not be able to see the TeacherDashboard button.
4.1 – The view assignment page should only be accessible for students; teachers should instead be able to view assignments they have set on their own dashboard	Similarly to the above, the user's role will be stored and used to show or hide the ViewAssignment button.
4.2 – The teacher must be able to upload a file to an assignment	Implemented a BinFile column into the DefinedAssignments table of type VARBINARY(MAX). Along with this, the file's name and extension must be stored as a VARCHAR(50) called FileName.
4.3 to 4.6 – Adding students to assignments must be validated, as mentioned	Added method in UsersRepo called UserExists() which returns true if the username is valid. Will add validation checks to ensure that the users are not already on the list and can't be added twice.
4.10 – The database must be able to store a student'**s** marks on a specific assignment	Created a normalised assignment database section. LinkedAssignments will contain a MaxMarks column, which can be updated by the UpdateMark() method in the AssignmentRepository when also given an associated AssignmentID and UserID (these could be encapsulated in an instance of the Users and Assignment class, which is an example of information hiding).
5.6 – Lessons must not be duplicated in the database	A Lesson table stores a specific lesson's details, such as the text, images and number. This Lesson can then be linked to various exam boards via the LinkedSyllabus table.
5.7 – Information, such as lessons, must be loaded dynamically	Lessons will be stored in the database and queried for during runtime by the LessonRepository class. Different methods will be used to get lessons for different circumstances. The UI layer should then display the results dynamically.
7.1 to 7.6 – Various different cryptographic functions or techniques must exist (those stated in these objectives)	I will define an abstract base class (CryptographicAlgorithm) which contains useful cryptographic methods (both on a binary-level or higher) and attributes (constants). From this class, I will have the SHA256Algorithm, CaesarCipher,

	VernamCipher and RailFenceCipher inheriting from it. I will use polymorphism for the Encrypt() and Decrypt() methods and then an interface for the Hash() method.
7.6 – Each encryption tool must allow for both encryption and decryption to occur	The Encrypt() and Decrypt() methods will be a stock part of each encryption class as they will all inherit from CryptographicAlgorithm, which will define these methods.
10.2 – Log-in and log-out times must be stored	Will create an AuditLog table, and each record will have its own LoginID, its login and logout time.
11.1 to 11.2 – Accessing information from the database must be implemented in a robust manner	Try catch blocks will be placed around all queries to the database. Specific errors will be caught, and a defined message will be displayed, where appropriate, written in a non-technical manner.

Technical Skills

Databases

I will have **six databases**, which can be **split into two systems**: lessons and users. The tables will be linked with a variety of types of keys and will feature typical data types, along with **binary** and **time** data. I will be using **identities** to automatically generate unique keys for each record in a table (sorted by a single primary key). One main use of the tables is that they will be used to load data into my application **dynamically**, from lessons to assignments. This improves code maintainability and ease of addition for other things.

Dynamic UI generation and Windows Forms

For many pages, including lessons and assignments, it would be very time inefficient if I created a new forms page for each lesson. Thus, I will be storing these types of data in a database and **loading it into a flow layout panel**. In addition, to display the assignments set to or by a user, I will need to load the correct details into a custom form element (this will be my **custom bubble class**). To create the visual elements of the application, I will be using WinForms. However, as I will be creating **custom form classes**, I will have to learn and be able to use the forms architecture directly (with code rather than using the designer window).

Algorithms

A key component of my project is the **complex encryption algorithms** I will utilise. According to my objectives, I will need to implement a Railfence Vernam and Caesar cipher, as well as a hashing algorithm. I will be required to process **binary data, 3-dimensional arrays** plus arrays of chars.

Parameterised API

The solution to my chatbot (help menu) will be OpenAI's LLM API. Through using this, I will be required to **securely store my key**, use OpenAI's **SDK** and **completion object**, and **customise the model using parameters** (such as max_tokens, temperature or messages).

Cross-table parameterised SQL

I will have to use **cross-table SQL** in many ways – the most obvious being using it to select and insert assignments into my database, because the assignments are not stored in a single table. Therefore, an example is getting the assignments (which were set by the teacher) that have been completed by students from the database. To do this, I will need to access the following details:

- The AssignmentID, title, date set and total marks from the DefinedAssignments table, WHERE the TeacherID is equal to the UserID stored in the current user object (thus equal to a parameter given by the program).
- The AssignmentID, UserID and mark uploaded to the LinkedAssignments table, WHERE the AssignmentID here matches an AssignmentID collected from the above.
- The first and last name of a student from the Users table, WHERE the UserID here matches a UserID from the LinkedAssignments table.

Files

The ability for teachers to share worksheets and set tasks through my application is the key interaction between teachers and students (my two types of users). Due to this, I will be using **Visual Studio's File Dialogue** Window to allow the teacher to enter the path of a file, then read the file's binary data before uploading it to the database. Similarly, on the student's end, I will be doing the reverse: **reading from the database and writing the file into the user's downloads folder**. I ensure I **robustly** do this because the path of a user's downloads folder may vary.

Image Data in Lessons Table

To ensure the application's suitability for visual learners, I will incorporate images into the lessons. To work with my **dynamically loaded** lesson pages, I would have to store each image along with the lesson as binary data. To upload the images to the database, I would have to use a **Data Manipulation Language** query and insert the data using **OPENROWSET**. Here is what the query should look like:

```
UPDATE DefinedLessons
SET [Image] = (       // using [] around image as it is a column yet
    SELECT BulkColumn // SQL recognises it as a keyword
    FROM OPENROWSET(
        BULK 'C:\Users\User\Downloads\ceaserCipher.png',
        SINGLE_BLOB
    ) AS img
)
WHERE LessonID = 1;
```

Aggregate SQL functions

When calculating the **mean time** spent by all users and the **total time** spent by the current user, I will use aggregate SQL functions. An example is below:

```
SELECT COALESCE(SUM(DATEDIFF(MINUTE, LoginTime, LogoutTime)), 0),
COALESCE(COUNT(DISTINCT UserID), 0)
FROM AuditLog
WHERE LogoutTime IS NOT NULL;
```

OOP – Inheritance

To **dynamically** generate certain buttons or layout panels on pages, I will create my own **custom forms class**. Then, the appropriate pages will **inherit** from this page, which will mean that on **every form page** that I want to show the **back or help buttons**, I can just make that page inherit from my class. I will have multiple custom form classes for all types of pages across my application. By doing this, I can have custom form pages that also inherit from other custom form pages, which will allow me to dynamically load and pick the features I need for each page. In addition to this, all repository classes inherit from my DataClass, which ensures that if I need to **alter the connection string** to the database, I only need to **change it in one place**.

OOP – Polymorphism

Not only will I use polymorphism in my **custom form's classes**, but also in my encryption algorithms. I will define each of my algorithms as a separate class. These will all inherit from a generic **CryptographicAlgorithm class**, with a constructor plus **generic encryption methods** and **encrypt/decrypt/hash interfaces**. For most algorithms (e.g. Caesar), the decryption process is the same as running the encryption process again with a negative key. Therefore, in my CryptographicAlgorithm class, the decrypt method will call and return the encrypt method. For any encryption algorithm where this rule is not the case, I will use **polymorphism** to alter the method. In addition, there will be many other methods in the CryptographicAlgorithm class; they will be generic and frequently used processes (for example, XOR operations or binary shifts). I will be using **overloading** to keep all methods generic and usable for a range of datatypes. For example, the AND operation has an overload for int[] and an overload for just a single int – the same occurs for many other binary operations and functions in the SHA class.

OOP - Encapsulation

I will use encapsulation throughout my program to **protect data**. Examples of this include the following:

- I will need to store the SQL connection string inside my forms classes (including the custom forms classes); therefore, I will be setting the string to protected or private (given the scenario) to hide sensitive credentials.
- Similarly, most page-specific methods inside my form objects will be set to private or protected; however, some methods will need to be public, like the update marks, as each assignment button is generated dynamically.

OOP – Interfaces

As part of my parent cryptography class, I will be using **interfaces** to define the hashing method – this is because only the hashing algorithm should have it. Therefore, the hashing algorithm will be defined to inherit from the **CryptographicAlgorithm class** and implement the **IHashingAlgorithm interface**. Interfaces will also **encourage consistency** and **promote reusability** throughout other classes, which is essential if I were to consider developing this application further (adding more features onto the **pre-existing framework**).

OOP – Namespaces

To create **cohesive, loosely coupled modules** and easily **manageable scopes**, I have designed an **effective namespace structure**. The structure separates the code into the **key functions**: **generic** classes, user **interface**, cryptography **algorithms** and more.

Event-driven programming

Many elements on my pages will be loaded dynamically; therefore, their **actions** must also be **assigned dynamically** (depending on their function). This means that I will be **linking events with methods during runtime**. This can be a challenge because if not implemented correctly, it can lead to logical or runtime errors.

Partial Classes

I will define a single class for my forms pages across many files, using the **partial** keyword. This will help to improve code corruption as it prevents the logic of core elements from being altered accidentally. As I am having custom forms classes, this is essential because it will ensure robustness for these fragile elements.

Async methods

I will be using this type of method when making requests to my API. This is because the call may take a few seconds to complete; therefore, **I do not want to freeze the thread**. As part of this, I will need to use the task object and await a message. Another use of this is with speech synthesis, which I will cover next.

Speech Synthesis

Many students find benefit in listening to a lesson rather than reading it. Therefore, I will be using **speech synthesis** to take my **dynamically loaded lesson data** and read it aloud to the user. I will be combining this with **async** methods to ensure a **smooth performance**, so that users can still navigate the page during the reading.

Data Structures

Simple data types

- Integers for loop counts, IDs and keys for various ciphers.
- Unsigned longs will be used for the SHA-256 hash and any bitwise operations.
    - The use of this prevents sign errors when converting between binary and integers.
- Boolean for validation and selection.
- Characters for the Caesar cipher and other cryptographic functions (including for binary to hexadecimal conversions).

Composite data types

- Strings for plaintext and cipher text, hexadecimal outputs and lesson text.
- Character arrays (1D and 2D) for more advanced textual processing, such as that needed in the Caesar and Railfence ciphers.
    - This is needed over strings for manipulating given indexes.
- String arrays for storing usernames when creating assignments or storing the current conversation in the help page.
- Integer arrays for representing a series of 1s and 0s, like those needed in Vernam cipher keys.
- Unsigned long arrays for representing a longer series of 1s and 0s, such as words in the SHA-256 cipher.
- Byte arrays will be used for storing image or file data before converting it into a more useful form or storing on a database.
- Three-dimensional arrays for keeping track of blocks, words and messages in a SHA256 cipher.
- Lists of objects will be used for storing undefined lengths of data. Examples include:
    - Lists of Assignment objects
    - Lists of Lesson objects
- Dictionaries will be used for storing lesson-level mappings. They can be used to convert the string ‘GCSE’ into a numeric value assigned to represent it (eg 1).

Custom classes

- User class for storing the current user's information, which may be needed by current forms pages.
- Lesson class for storing all the details of a lesson to display – this may be just the title or the text, work prompt and images.

- Assignment class for storing generic and appropriate assignment-related details when displaying to the user.

UI objects

- FlowLayoutPanels for dynamically displaying information on a page within a certain, predefined area.
- TableLayoutPanel for creating logical page layouts.
- ListBoxes will be used for storing lists of student usernames when a teacher is creating assignments.
    - This will allow them to easily see who is on the assignment and add or remove users where appropriate.

System Architectural Design

I will be breaking my overall dataflow and thus system into three key levels to aid my understanding of how data will move throughout the application. I have chosen this three-level design to ensure that each layer acts independantly to each other – improving adaptability and keeping all elements separate.

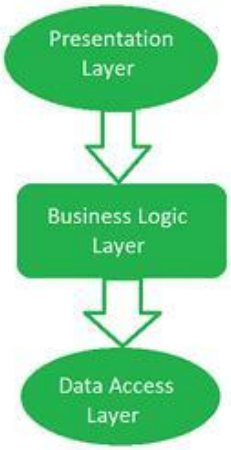


FIGURE 15 – A DIAGRAM FROM GEEKSFORGEEKS SHOWING A BASE FOR MY HIGH-LEVEL DATA FLOW DESIGN

UI layer

This layer includes any forms classes (pages) created and then all the way down to the smallest elements, such as input collection. This layer is only concerned with taking the user's input and displaying various outputs. This layer will not be concerned with performing encryption processes and executing SQL queries.

Business-Logic Layer (BLL)

This layer will serve as an intermediary stage between the UI layer and the Data Access Layer. This layer is associated with validating input, performing calculations (cryptographic processes) and communicating with the Data Access Layer. When looking specifically at my application, this layer will be concerned with validating and processing user input (for example, in the cryptographic algorithms), moving data between repositories and checking for access rights (with the teacher dashboard). Access rights will predominantly be checked at this level; however, the result of these rights may lead to UI elements being altered (hidden) or access to the database being denied.

Note: I decided to learnt about this layer as part of my NEA after a suggestion made by another student. I researched it through a GeeksForGeeks article…
(https://www.geeksforgeeks.org/dbms/business-logic-layer/)

Data Access Layer (DAL)

This layer is concerned with getting access to data stored in a database and returning it to the Business-Logic Layer. For my application, I will create repository classes for each of the following: Lessons, Users and Assignments. Then each repository will have various subroutines for getting different data from the database, and each will return it in its associated object or another form that furthers code reuse.

Data Flow Diagram

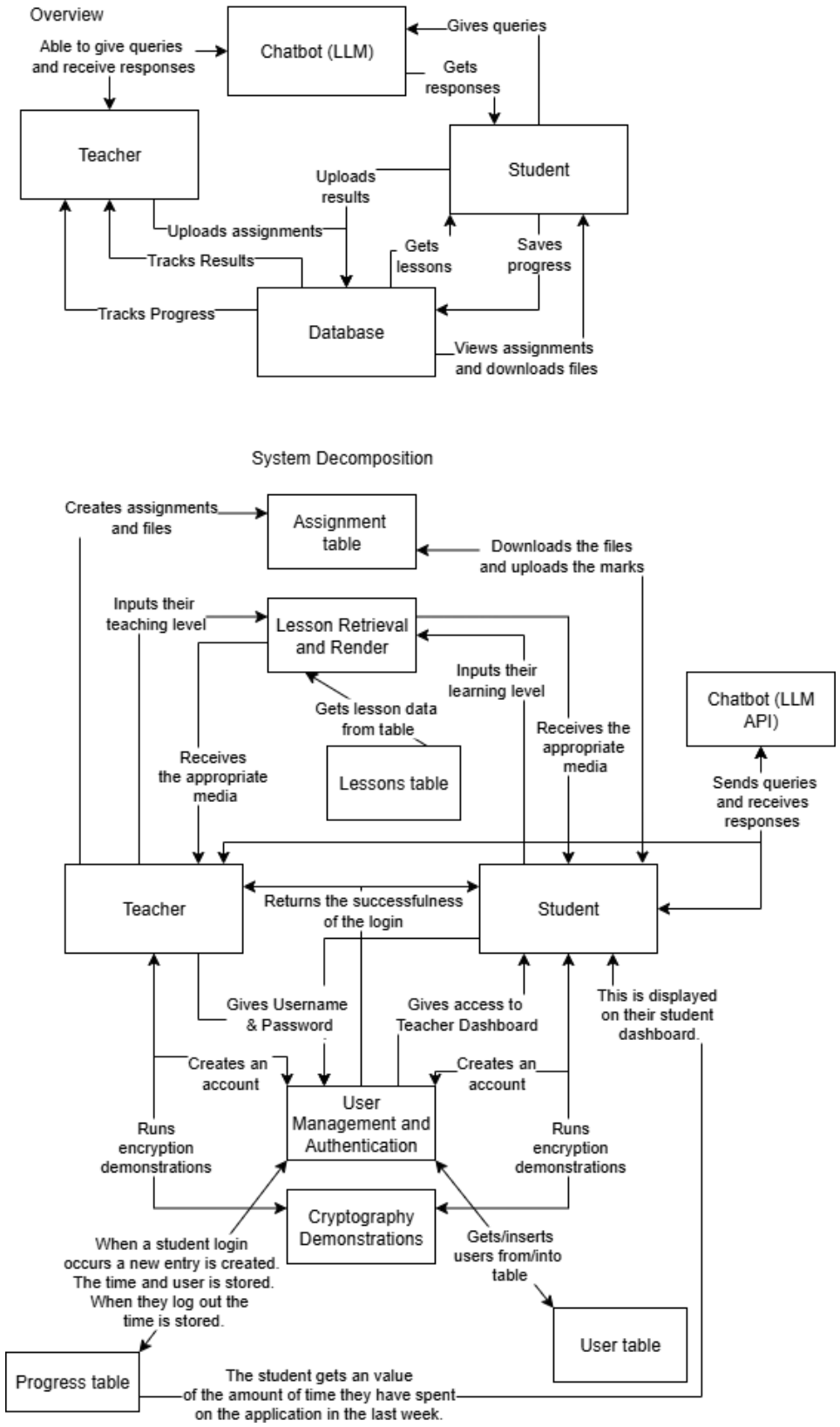


FIGURE 16 – THE DATA FLOW DIAGRAM OF MY APPLICATION. THIS SHOWS MY CURRENT IDEA FOR HOW EACH INPUT (DATA STORE OR USER INTERACTION) IS USED (PROCESSED) AND THEN THE RESULT OF THAT PROCESS (HOW THEIR INPUT IS DISPLAYED).

Classes

Class Diagram

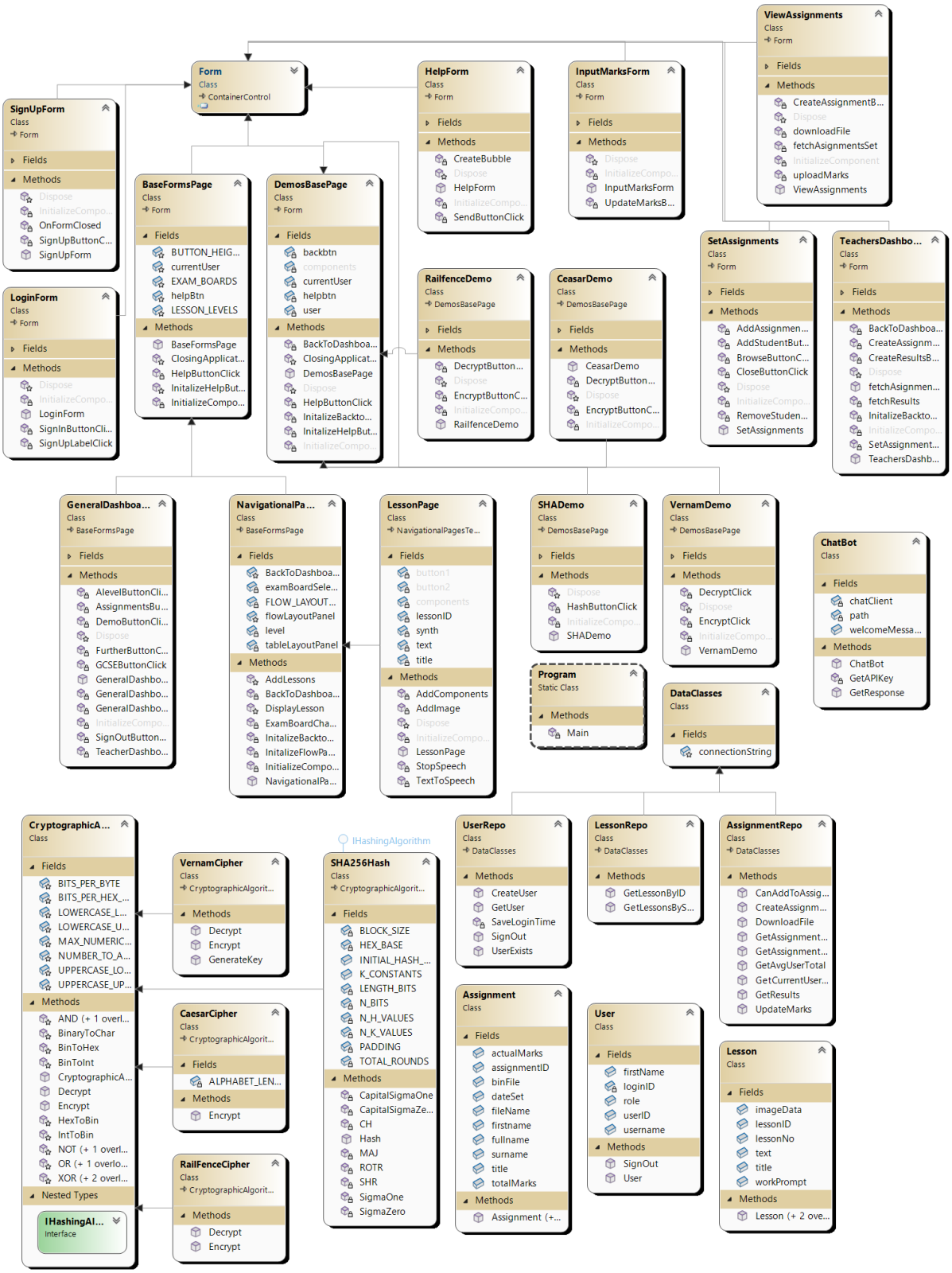


FIGURE 17 – THE CLASS DIAGRAM OF MY APPLICATION. THIS SHOWS HOW EACH CLASS INTERACTS WITH THE OTHERS, LOOKING CLOSELY AT THE INHERITANCE BETWEEN CLASSES, PLUS THE METHODS AND PROPERTIES THAT EACH CLASS HAS.

Class Documentation

User Class

This generic class will hold all essential information needed by the algorithm. When fetching data from the database, things like the user's ID are needed, which is why they will be stored in the class.

Lesson Class

This class will be used to hold details of currently retrieved lessons – this is especially useful as a list of lesson objects can be returned from various methods in the Lesson Repo class.

Assignment Class

This class will be used similarly to the above Lesson class – it will be used to hold details of assignments retrieved from the Assignment Repo class. This will also prevent numerous visits to the database.

Data Class

This class will be used to hold the connection string for all classes that need to access the database. As I am designing my application so that only classes that inherit from this will make the queries to the database, which will keep the database part of my application separate from other areas (like the user interface). The repository classes will inherit from this class, which will ensure that if the connection string ever needs to be changed, it can be done through this class in one place, thus improving data integrity.

User Repo Class

Instead of running queries on forms pages, I will modularise the process of querying the users table (on my database) through this class. This will result in better encapsulation, code reuse and maintainability.

Lesson Repo Class

Similarly to the User Repo, this class will be used to perform necessary select or insert into queries, but this time relating to the Lesson Syllabus and Lesson Details tables.

Assignment Repo Class

This repository class will have the greatest number of methods, it will be used to modularise and abstract the process of creating assignments, uploading and downloading files, uploading marks and getting the average time spent by users on the application.

BaseFormsPage Class

The help button will be visible in almost all pages; therefore, instead of recreating the code for each page where it is present, I will create a class that inherits from 'Form' and initializes the help button. Then, for the forms that will have the help button, I can make that form inherit from this class.

NavigationalPagesTemplate Class

This class inherits from the BaseFormsPage class and is in place to add a few further elements to the forms page. A back-to-dashboard button, a flow-layout and a table layout panel are added to the screen in the initializer. If this class is told so in the constructor, it will also add a combo box, which serves as a drop-down menu for selecting the exam board.

LessonPage Class

This class inherits from the GeneralForms class. There will be many pages that will need a back button and a flow layout panel in the same space, for example, a lesson on the Caesar Cipher. Similarly to the GeneralForms class, this is a parent to many different lesson screens (this could include viewing the syllabus or a lesson).

CryptographicAlgorithm Class

This is a parent class to all algorithms that I will have a demo of. The class will contain methods for all generic and frequently used computations, for example XORs on binary arrays or converting denary to hexadecimal. Therefore, all algorithms will inherit from it and be able to access or override certain methods to improve code reusability and maintainability.

CaesarCipher Class

I will be using polymorphism to override the CryptographicAlgorithm class's encrypt and decrypt methods with this shift cipher. The class will be used in the demos for encrypting and decrypting strings when given a key.

Vernam Class

This class inherits from the CryptographicAlgorithm class and will use methods from it relating to XORing strings, converting denary into binary, or denary to hexadecimal. All binary results will be displayed as hexadecimal.

Railfence Class

The CryptographicAlgorithm will be a parent to this class. Railfence is a transposition cipher that will need its own methods; therefore, extra methods will be added to this class.

SHA-256 Class

This algorithm is highly compicated and therefore I will be decomposing the overall algorithm into many smaller tasks. I will use methods from the CryptographicAlgorithm class (its parent) to improve the ease of writing and debugging this algorithm.

ChatBot Class

This class will be used for making calls to OpenAI's LLM API. When creating an instance of the object, a connection will be established using a securely stored API key, then the getResponse method may be called with the query as an input. This class will be defined as an async class, which will mean that the thread will not freeze during the API call.

Class Property and Method Design

User Class

Properties: userID, username, firstName, role, loginID	Methods: signOut

Lesson Class

Properties: lessonID, lesson, synth	Methods: various constructor overloads

Assignment Class

Properties: assignmentID, totalMarks actualMarks, fileName, firstName, secondName, fullName, title, binFile, dateSet	Methods: various constructor overloads

Data Classes class

Properties: connectionString

User Repo Class

Properties: None	Methods: CreateUser, GetUser, SaveLoginTime, UserExists, SignOut

Lesson Repo Class

Properties: None	Methods: GetLessonByID, GetLessonBySyllabus

Assignment Repo Class

Properties: None	Methods: CanAddToAssignment, CreateAssignment, DowloadFile, GetAssignmentsForStudent, GetAssignmentForTeacher, GetAvgUserTotal, GetCurrentUserTotal, GetCurrentUserAssignements, GetResults, UpdateMarks

BaseFormsPage

Properties: helpBtn, currentUser BUTTON_HEIGHTS, LESSON_LEVELS EXAM_BOARDS	Methods: helpBtnClick, InitializeComponent closingApplication, InitalizeHelpButton

NavigationalPagesTemplate

Properties: level, examBoardSelector tableLayoutPanel, FLOW_LAYOUT_PADDING	Methods: InitalizeBackToDashboardButton, InitalizeFlowPanel, AddLessons, ViewLessons InitalizeComponent, BackToDashboardButtonClick ExamBoardChanged

LessonPage

Properties: lessonID, text, title, synth (speech synthesizer object)	Methods: addComponents, AddImage, TextToSpeech, StopSpeech

CryptographicAlgorithm

Properties: BITS_PER_HEX_DIGIT, BITS_PER_BYTE, MAX_NUMERIC_VAL NUMBER_TO_ALPHA_ASCII_SHIFT, LOWERCASE_LOWER_BOUND, LOWERCASE_UPPER_BOUND, UPPERCASE_LOWER_BOUND, UPPERCASE_UPPER_BOUND	Methods: Encrypt, Decrypt, AND, OR, XOR, NOT, IntToBin, BinToInt, HexToBin, BinToHex, BinaryToChar, IHashingAlgorithm *(an interface for all hashing algorithms)*

Note: for each method relating to a Boolean operation, there are overloads for both array inputs and int inputs.
The children who inherit from this class will add their own methods and or overwrite the existing ones.

CaesarCipher

Properties: ALPHABET_LENGTH	Methods: overwrites Encrypt and uses original Decrypt

VernamCipher

Properties: *None*	Methods: overwrites Encrypt *(there is also an overloaded encrypt if the user does not give a key)*, overwrites Decrypt.

Railfence

Properties: *None*	Methods: GenerateKey, overwrites both Encrypt and Decrypt

SHA256

Properties: nBits, K_CONSTANTS, INITIAL_HASH_VALUES, N_H_VALUES N_K_VALUES, BLOCK_SIZE, PADDING LENGTH_BITS, HEX_BASE, TOTAL_ROUNDS	Methods: ROTR, SHR, MAJ, CH, SigmaZero, SigmaOne, CapitalSigmaZero, CapitalSigmaOne, Hash and all binary/Boolean methods from CryptographyAlgorithm

ChatBot

Properties: ChatClient, path, welcomeMessage

Methods: getResponse, getAPIKey

Method Descriptions

All numbers are represented using the type ulong to avoid conversion errors during bitwise operations. This is memory inefficient, however, completely necessary to ensure the correct result is achieved, which is essential for educational purposes.

User Class

- User(int uID, string Uname, string fName, string role, int login): This is the only constructor for the user class.
- SignOut(): This method is used to provide a **centralised** place for uploading a Log-out time to the AuditLog database.

Lesson Class

- Lesson(int lessonID, string text, string title, string workPrompt): This constructor is used when adding a lesson to the lesson viewer when the lesson does not have an image.
- Lesson(int lessonID, string text, string title, string workPrompt, byte[] imageData): This constructor is used when adding a lesson to the lesson viewer when the lesson does have an image.
- Lesson(int lessonID, int lessonNo, string title): This constructor is used to create a lesson instance for each lesson to be displayed in the lesson selector page.

Assignment Class

- Assignment(string title, DateTime dateSet, int totalMarks): This constructor is used to create an Assignment instance that stores the details of each assignment set by the current teacher for displaying on the teacher dashboard.
- Assignment(string title, DateTime dateSet, int totalMarks, int actualMarks, string fullname): This constructor is used to create an Assignment instance that stores the details of their students recent results for displaying on the teacher dashboard.
- Assignment(string title, DateTime dateSet, int totalMarks, byte[] binFile, string fileName): This constructor is used to create an Assignment instance that stores the details of an assignment that is to be set by a teacher.
- Assignment(string title, DateTime dateSet, int totalMarks, string firstname, string surname, int assignmentID): This constructor is used to create an Assignment instance that stores the details of an assignment set by a teacher for viewing on a student's assignment viewer.

User Repo Class

- CreateUser(string username, string hashedPassword, string firstname, string secondname, string role): This method is used to create an entry to the Users table with the required information passed in as arguments. It returns True if the result was a success.
- SignOut(int userID, int loginID): This method is called whenever a forms page that inherits from the BaseFormsPage closes. It adds a log-out time to the row associated with the user's current log-in.
- GetUser(string username, string hashedInput): This static method is used to return get a user's details from the database and then return an instance of the User object with their details.
- SaveLoginTime(int userID): This method is used to create an inital entrance to the LoginAudit table, which saves the time a specific user logs in. The SignOut method in the User object is used to add the log-out time to this entry.

- UserExists(string username): This static method is used to check for a username in the Users table, if it exists it returns True and otherwise False

Lesson Repo Class

- GetLessonByID(int lessonID): This static method is used to obtain, create and return the information needed for an individual lesson inside an instance of the Lesson object. The information is then used to display the lesson to the user in the LessonViewer page.
- GetLessonsBySyllabus(string section, string examBoard, Dictionary<string, int> lessonLevels): This static method is used to obtain, create and return a list of instances of the Lesson object which contain the lessons that are to be displayed to the user, given their selected exam board.

Assignment Repo Class

- CanAddToAssignment(string username, ListBox StudentsList): This static method is used to validate each username added to the list of people to whom an assignment is to be set. It validates them to ensure that their username exists, is a student rather than a teacher and has not already been added.
- CreateAssignment(User currentUser, Assignment assignment, string[] users): This static method is used to create an entry to the Assignments table, thus creating an assignment for the current teacher. In addition to storing the appropriate data, it also assigns the assignment to all necessary students.
- DownloadFile(int assignmentID): This static method is used to save the file uploaded by a teacher to an assignment.
- GetAssignmentsForStudent(User currentUser): This static method is used to obtain, create and return a list of assignments that have been set for the current student. The assignments will then be displayed on the student's assignment viewer page.
- GetAssignmentsForTeacher(User currentUser): This static method is used to obtain, create and return a list of assignments that the current teacher has set. The assignments will then be displayed on the teacher's dashboard.
- GetAvgUserTotal(): This static method is used to return the average total time spent by each user on the application. It works this out using the audit table and the number of users on the application.
- GetCurrentUserTotal(User currentUser): This static method returns the total time spent by a single user on the application. It does this through the audit log, with the user's ID being used to filter the table down.
- GetResults(User currentUser): This static method is used to obtain, create and return a list of all assignments set by the current teacher and their results.
- UpdateMarks(int marks, int uID): This static method is used to update the Marks field of the LinkedAssignments table, thus allowing a student to share their marks with their teacher.

BaseFormsPage

- BaseFormsPage(User user, bool isHelpBtnVis): This is the base (and only) constructor which adds the help button to the page and assigns the ClosingApplication method to the action of FormClosed.
- ClosingApplication(object sender, FormClosedEventArgs e): This method is used to call the SignOut method of the user (thus adding the signout time to an appropriate entry of the AuditLog table).
- InitalizeHelpButton(): This method is used to create and add a help button to the bottom-left corner of the pages.

- HelpButtonClick(object? sender, EventArgs e): This method creates and shows the help form page, it does not close the current page.
- InitializeComponent(): This method creates a page of size 1000, 600 in the centre of the screen

NavigationalPagesTemplate

- NavigationalPagesTemplate(User user, string Level): base(user, true) : This constructor is used to create a page suitable for viewing the lessons in any syllabus
- NavigationalPagesTemplate(User user, string Level, bool addComponents) : base(user, true): This constructor is called from the LessonPage constructor to set up the page for displaying a chosen lesson. This means displaying its text, workprompt, various navigational buttons and an image if necessary.
- NavigationalPagesTemplate(User user, bool helpButtonVis) : base(user, helpButtonVis): This is a default constructor for this class that calls the BaseFormsTemplate constructor passing if the user intends to make the help button visible. This constructor is not in current use - however is still a default, so necessary to be kept for future development.
- NavigationalPagesTemplate(User user) : base(user, true): This is a default constructor for this class that calls the BaseFormsTemplate constructor passing that the user does want the help button visible (as not specified). This constructor is not in current use - however is still a default, so necessary to be kept for future development.
- InitalizeBacktoDashboardButton(): This method is used to create a back to dashboard button in the bottom right corner of the user's display.
- InitalizeFlowPanel(): This method is called by all the current constructors to create a flowlayout panel which fits in the centre of the screen with a padding of 12 from the edges. The **BUTTON_HEIGHTS constant** is **stored in the BaseFormsPage**, which creates both the help and back to dashboard buttons from it, thus if these need to change size, so will the padding and layout panel.
- AddLessons(string section, string examBoard): This method is used to get lessons for a specific syllabus and examboard from the database and appropriately display them to the user. In addition, **methods are dynamically assigned** to actions such as clicking on a specific lesson's link, thus opening the LessonPage and displaying the correct information.
- DisplayLesson(string text, string prompt): This method is used for adding the lesson details to the screen. The lesson text and work prompt are added as part of this method.
- InitializeComponent(): This method is used to add the exam board selector and flow laybout panel to the window.
- BackToDashboardButtonClick(object? sender, EventArgs e): This button is used to close the current window and open the general dashboard window, thus returning the user to the dashboard from wherever the button was pressed.
- ExamBoardChanged(object sender, EventArgs e): This button triggers the AddLessons method, which means that the lessons shown are changed each time the syllabus is changed.

LessonPage

- LessonPage(User user, int ID, string level): base(user, level, false): This is the only constructor for this page; it is used to add lesson information based on the lessonID given.
- AddImage(byte[] imageData): This method is used to load and add an image to the lesson page when given the image in a byte array. It does this in a manner that ensures the thread does not freeze, using a **MemoryStream**.

- AddComponents(): This method is used to initalize a flow layout panel before initalizing the rest of the window.
- TextToSpeech(object sender, EventArgs e): This method is called when the text-to-speech button is pressed and reads the text on the page **asynchronously**.
- StopSpeech(object sender, EventArgs e): This method is called when the stop button is pressed, and resultingly stops the voice speaking.

CryptographicAlgorithm

- IHashingAlgorithm *(interface):* This **interface** creates a Hash method for hashing classes that inherit from this, like the SHA-256 class.
- Encrypt(int key, string plainText): This method will be overridden by child classes to run their own version, ultimately returning the text encrypted.
- Decrypt(int key, string cipherText): This method calls the Encrypt method with a negative key, because most decryption processes are simply the same as encryption but with the key being used in a reverse manner. This method can be overridden.
- AND(ulong[] a, ulong[] b): This is a stock cryptography process and thus the method performs an and operation between two ulong arrays. Both arrays will only ever contain 1s and 0s in each index but is of type ulong to prevent conversion errors later in, for example, the hashing algorithm.

Note: See the explanation below the Method Descriptions heading for the use of ulongs.

- AND(ulong a, ulong b): This overload of AND takes two ulongs as opposed to a list of them. It simply performs an AND operation between the two.
- OR(int[] a, int[] b): This is a stock cryptography process and thus the method performs an OR operation between two ulong arrays. Both arrays will only ever contain 1s and 0s in each index but is of type ulong to prevent conversion errors later in, for example, the hashing algorithm.
- OR(int a, int b): This overload of OR takes two ulongs as opposed to a list of them. It simply performs an OR operation between the two.
- XOR(ulong[] a, ulong[] b): This is a stock cryptography process and thus the method performs an XOR operation between two ulong arrays. Both arrays will only ever contain 1s and 0s in each index, but is of type ulong to prevent conversion errors later in, for example, the hashing algorithm.
- XOR(ulong a, ulong b): This overload of XOR takes two ulongs as opposed to a list of them. It simply performs an XOR operation between the two.
- XOR(string ascii, string key): This overload of XOR takes two strings where each character is only a 1 or 0. This method has been created specifically for cryptography processes such as the Vernam Cipher, where ascii values are XORed with a key.
- NOT(ulong[] a): This is a stock cryptography process and thus the method performs a NOT operation on every bit in a ulong array. The array will only ever contain 1s and 0s in each index, but is of type ulong to prevent conversion errors later in, for example, the hashing algorithm.
- intToBin(ulong x, int length): This method is used to convert a positive integer to a binary value. It takes a value to convert and then an int which denotes number of bits to express it with. The method does not use a sign convention - only positive values are represented.
- binToInt(ulong[] bin): This method converts a binary array of any given length to a single ulong (positive) value. The value can only ever be positive.

- HexToBin(string hex): This method is used to convert a string of any number of hexadecimal digits to an unsigned binary array.
- binToHex(ulong[] bin): This method is used to convert an unsigned binary array of any given length into a string of hexadecimal digits which represent the value of the binary array.
- BinaryToChar(string binary): This method is used to convert an unsigned binary array of 8 chars into a single character, based on its ASCII code.

Note: *for each method relating to a Boolean operation, there are overloads for both array inputs and int inputs.*

The children who inherit from this class will add their own methods and or overwrite the existing others

CaesarCipher

- Encrypt(int key, string plainText): This static method overrides the original Encrypt method in CryptographicAlgorithm. It is used to process and string of text and key by performing a Caesar cipher on the string, shifting each char the given number of places in the alphabet. The result is returned as a string. Note: any symbols (such as punctuation) are not shifted but instead left in their place. This is because the symbols do not have a native alphabet and would still be convenient to be represented by this cipher.
- The Decrypt method from the CryptographicAlgorithm class is left alone.

VernamCipher

- GenerateKey(string plainText): This static method is used to generate a random ASCII key of the correct length. It is used when the user does not enter a key.
- Encrypt(int[] key, string plainText): This static method overrides the original Encrypt method in CryptographicAlgorithm. It is used to perform a Vernam cipher on a string. It returns the result in hexadecimal.
- Decrypt(int[] key, string cipherText): This static method overrides the original Decrypt method in CryptographicAlgorithm. A negative key cannot be created or used to decrypt the cipher text. Instead, a decryption can be done by performing the exact encryption process with the cipher text and the same key.

Railfence

- Encrypt(int key, string plainText): This static method overrides the original Encrypt method in CryptographicAlgorithm. It is used to perform a railfence cipher on a string using a given key. The result is returned as a single string.
- Decrypt(int key, string cipherText): This static method overrides the original Decrypt method in CryptographicAlgorithm. It then decrypts a cipher text given the key and cipher text as a string. The encryption process is different to the Encrypt process which is why a negative key cannot be used. The result is returned as a single string.

SHA256

- ROTR(ulong[] a, ulong n): This method is used to rotate a string of bits to the right by a given number of places. It shifts all bits to the right by n filling in the new spaces with the bits removed.
- SHR(ulong[] a, ulong n): This method is used to shift all of the bits in a binary array to the right by a given number of places the new space is filled with zeros.
- MAJ(ulong x, ulong y, ulong z): This method is used to perform a majority operation on three ulongs. The operation can be broken down to: **(X AND Y) XOR (X AND Z) XOR (Y AND Z).**
- CH(ulong x, ulong y, ulong z): This method is used to perfrom a choose operation on three ulongs. The operation can be broken down to: **(X AND Y) XOR (NOT X AND Z).**

- SigmaZero(ulong x): This method performs the lowercase sigma zero function unique to the SHA-256 hash. The operation can be broken down to: **ROTR(X, 7) XOR ROTR(X, 18) XOR SHR(X, 3).**
- SigmaOne(ulong x):This method performs the lowercase sigma one function unique to the SHA-256 hash. The operation can be broken down to: **ROTR(X, 17) XOR ROTR(X, 19) XOR SHR(X, 10).**
- CapitalSigmaZero(ulong x): This method performs the uppercase sigma one function unique to the SHA-256 hash. The operation can be broken down to: **ROTR(X, 2) XOR ROTR(X, 13) XOR SHR(X, 22).**
- CapitalSigmaOne(ulong x): This method performs the uppercase sigma one function unique to the SHA-256 hash. The operation can be broken down to: **ROTR(X, 6) XOR ROTR(X, 11) XOR SHR(X, 25).**
- Hash(string input): This static method overrides the interface and Hash method of the CryptographicAlgorithm class. It is used to perform the overall hashing process for SHA-256. The result is returned as a hexadecimal value.
- Many classes from the CryptographicAlgorithm will be used in the methods associated with this class.

ChatBot

- ChatBot(): This constructor is used to initialize the chatbot and create a ChatClient object, as per OpenAI's API.
- GetResponse(string input): This method creates an API call to obtain the LLM's response, given a message as input. It has been defined as **async** because it prevents the thread from being frozen during API calls. This is crucial as API calls to this service can be slow and computationally intensive.
- GetAPIKey(): This method is used to securely get the API key, which is securely stored in a given path. The key is not stored as plain text in a variable because this could allow others to get hold of and abuse the key.

Attribute Descriptions

User Class

- The attributes below correspond to each column in the Users table of the database and are part of the Users object, so they can be sent around the application together (in an encapsulated manner).
    - `public int userID;`
    - `public string username;`
    - `public string firstName;`
    - `public string role;`
    - `private int loginID;`

Lesson Class

- The attributes below correspond to each column in the DefinedLessons table of the database and are part of this object, so they can be sent around the application together (in an encapsulated manner).
    - `public int lessonID;`
    - `public string text;`
    - `public string title;`
    - `public string workPrompt;`
    - `public byte[] imageData;`
    - `public int lessonNo;`

Assignment Class

- The attributes below are collected from each column in the DefinedLessons, SyllabusLessons and Users table of the database and are part of this object, so they can be sent around the application together (in an encapsulated manner). This is particularly useful when assignment details or results need to be displayed.
    - `public string title;`
    - `public int assignmentID;`
    - `public DateTime dateSet;`
    - `public int totalMarks;`
    - `public int actualMarks;`
    - `public string fullname;`
    - `public string firstname;`
    - `public string surname;`
    - `public byte[] binFile; // this stores the binary data of the file sent by the teacher`
    - `public string fileName;`

Data Classes Class

- This string is used for creating a link to the database. It shows Microsoft.Data.SqlClient, where the database is stored, in this case locally.
    - `static protected string CONNECTION_STRING = "Server=DESKTOP-TIHP928;Database=NEA;Trusted_Connection=True;Encrypt=False;";`

BaseFormsPage

- This attribute holds all the current user's details so they can be accessed by the UI and then reacted to on this level:
    - `protected User currentUser;`
- This attribute stores the help button so that it can be added to the screen dynamically:
    - `protected Button helpBtn;`
- The following dictionary stores the mappings/encodings used for each level in the code below. It provides a fast way of converting between the string Level and then a integer representing the level. This dictionary will allow for lesson levels to easily be added and interpreted from the database correctly.

```
protected private Dictionary<string, int> LESSON_LEVELS = new
Dictionary<string, int>
{
    { "FURTHER", 3 },
    { "ALEVEL", 2 },
    { "GCSE", 1 },
    { "DEMOS", 0 }
};
```

- The following dictionary holds all of the Exam Boards that are individually recognised by the application
  - `protected string[] EXAM_BOARDS = { "AQA", "Edexcel", "OCR" };`
- This attribute provides a single constant for the height of buttons, which is then used when creating the button or padding for the display:
  - `protected int BUTTON_HEIGHTS = 40;`

NavigationalPagesTemplate

- The following constant stores the padding of the layout panel from windows edges. Thus, how much space there is between the key elements and the edge of the window.
  - `private int FLOW_LAYOUT_PADDING = 12;`

LessonPage

- This lesson object stores the details to be displayed on the page:
  - `private Lesson lesson;`
- The synth attribute is used to store and object of type SpeechSynthesizer which is later used to read the text out loud:
  - `private SpeechSynthesizer synth = new SpeechSynthesizer();`

CryptographicAlgorithm

- The following constants are values used in general cryptographic algorithms. They are stored as constants to provide a placeholder or identifier for the value and a central place to store the value.
  - `protected int BITS_PER_HEX_DIGIT = 4;`
  - `protected int BITS_PER_BYTE = 8;`
  - `protected ulong MAX_NUMERIC_VAL = 10; // this is the threshold for hexadecimal where numbers are no longer used (eg 10 = A)`
  - `protected ulong NUMBER_TO_ALPHA_ASCII_SHIFT = 87; // eg 10 + 87 = the letter A (as above)`
  - `protected int LOWERCASE_LOWER_BOUND = 97; // These demonstrate the bounds for the ASCII`
  - `protected int LOWERCASE_UPPER_BOUND = 122; // values of uppercase and lowercase symbols`
  - `protected int UPPERCASE_LOWER_BOUND = 65;`
  - `protected int UPPERCASE_UPPER_BOUND = 90;`

VernamCipher

- GenerateKey(string plainText): This static method is used to generate a random ASCII key of the correct length. It is used when the user does not enter a key.
- Encrypt(int[] key, string plainText): This static method overrides the original Encrypt method in CryptographicAlgorithm. It is used to perform a Vernam cipher on a string. It returns the result in hexadecimal.
- Decrypt(int[] key, string cipherText): This static method overrides the original Decrypt method in CryptographicAlgorithm. A negative key cannot be created or used to decrypt the cipher text. Instead, a decryption can be done by performing the exact encryption process with the cipher text and the same key.

SHA256

- The N_BITS constant defines what could be thought of as the word length of the algorithm (the number of bits that each process occurs on):
    - `private int N_BITS = 32;`
- The following array of strings contains the initial hash values, which are the fractional parts of the square roots of the first prime numbers. Each value is stored in hexadecimal.
    - ```
      private static readonly string[] INITIAL_HASH_VALUES =
      // these are H0-7

      {
          "0x6a09e667", "0xbb67ae85", "0x3c6ef372", "0xa54ff53a",
          "0x510e527f", "0x9b05688c", "0x1f83d9ab", "0x5be0cd19"
      };
      ```
- The following array of strings contains the round constants used, which are the fractional parts of the cube roots of the first 64 prime numbers. Each value is stored in hexadecimal.
    - ```
      private static readonly string[] K_CONSTANTS =
      {
          // H0-H7
          "0x428a2f98", "0x71374491", "0xb5c0fbcf", "0xe9b5dba5",
          "0x3956c25b", "0x59f111f1", "0x923f82a4", "0xab1c5ed5",

          // Later vals
          "0xd807aa98", … , "0xc67178f2"
      };
      ```
- The following constants define various helper constants. They are stored as constants to provide a placeholder or identifier for the value and a central place to store the value.
    - `private int N_H_VALUES = 8;`
    - `private int N_K_VALUES = 64;`
    - `private int BLOCK_SIZE = 512;`
    - `private int PADDING = 448;`
    - `private int LENGTH_BITS = 64;`
    - `private int HEX_BASE = 16;`
    - `private int TOTAL_ROUNDS = 16;`

ChatBot

- This attribute stores OpenAI's ChatClient object (which serves as a link between their LLM and my code):
    - `private ChatClient chatClient;`
- The WELCOME_MESSAGE constant defines what is to be displayed on the user's screen as they first open the application:
    - ```
      private string WELCOME_MESSAGE = "Hello, Welcome to the help page. I
      am an expert in all things cryptography related. Please ask any
      questions you have here and I will answer them. I can also suggest
      articles or YouTube videos to watch - just ask!";
      ```
- This PATH directs the program to the location of the txt file with the API key inside. It serves as a method of securely storing the key.
    - ```
      private string PATH = Path.Combine(
          AppDomain.CurrentDomain.BaseDirectory,
          "Help Page",
          "APIKey.txt"
      );
      ```

Database Design

Database Table Designs

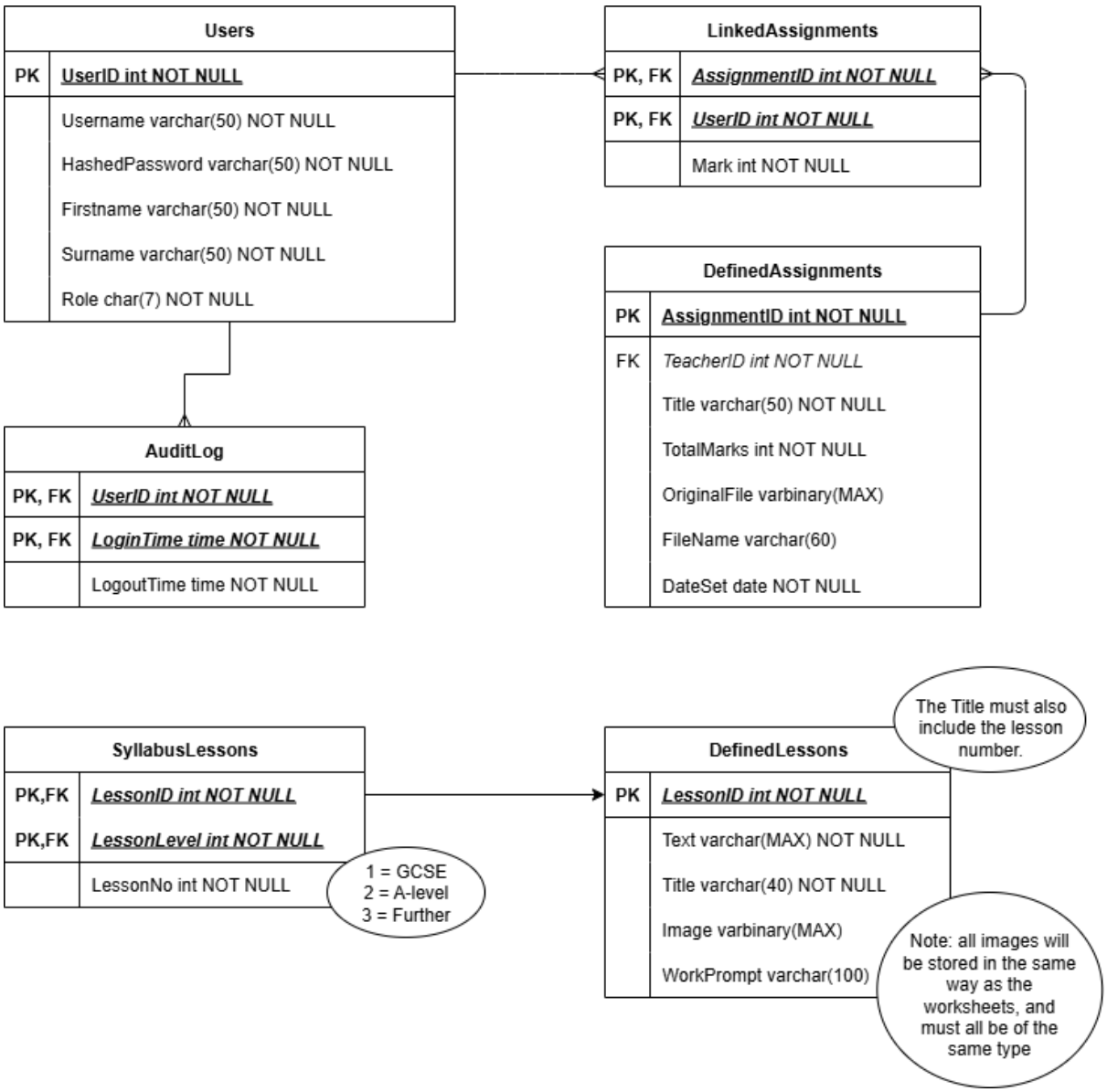


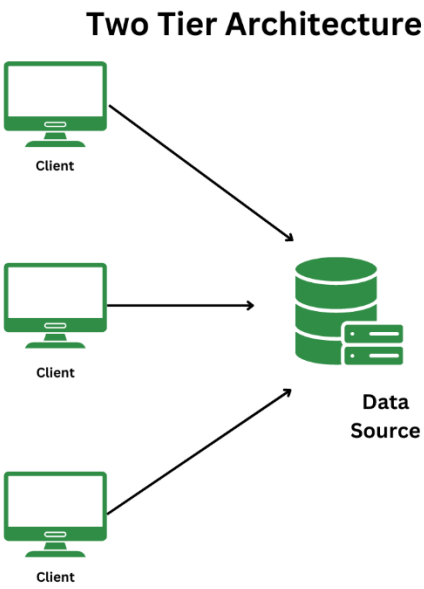
FIGURE 18 – THE DATABASE TABLE DESIGN FOR MY APPLICATION.

Note: *After making this table design, I have decided to store the hashed password as hexadecimal; therefore, the column will now be of type CHAR(64). Other changes are as follows:*

- *Primary key of the AuditLog table is now LoginID*
- *WorkPrompt is now VARCHAR(600)*
- *ExamBoard field added to SyllabusLessons of type VARCHAR(10)*

Client-Server System

Due to various limitations, I will not be able to deploy a remote (non-local) database system for the design and initial production of this application; this would eventually be the goal, as the application would not function correctly. A local DB means that for a teacher to set a task, they would have to be using the same computer as the student (ignoring the possibility of the database being on a school server), which is simply not viable. Therefore, I would look to adopt the following system…



I would look to use a two-tier (client-server) model as opposed to a three-tier architecture. This is because I do not have the intention of launching a web version of my application; therefore, a middle-tier application server is not necessary (because the application will be downloaded and run using the user's desktop resources).

FIGURE 19 – GEEKS FOR GEEKS IMAGE ON SERVER ARCHITECTURE TYPES.

(https://www.geeksforgeeks.org/dbms/difference-between-two-tier-and-three-tier-database-architecture/)

Data Deletion

It is important to note that data must – at some point – be deleted from the database. If this were not the case, redundant data would be kept, slowing down the retrieval times and increasing the amount of memory required by the server. Therefore, I have designed a system for deleting the data.

- Every 6 months, all assignment-related data will be deleted, which includes all the assignments and files set to students, and their results. This will provide the largest reduction as the files (binary data) could be large. In addition, it will help to keep the teacher's dashboard cleaner.
- If a user has marked their account as inactive via a button on the dashboard, a field in the Users table called Active will be set to false. If this status has been set to false for longer than 2 years, the account will be deleted – this includes all audit data, assignments and user data.

This process would occur on the server side rather than through my application – therefore, it will not be implemented in the program code of the application – instead, it would be incorporated if I were to develop the application further (especially looking at whether a cloud database is used).

For my prototype, I have not included the Status field in my Users table or added the account activity button onto the dashboard, as these features are less essential than others and only useful on a larger-scale remote database.

SQL Query Design

I will now draft some of the key and most complex queries in my application. Note that not all queries appear here due to some being left out.

Each query has been given a number for reference, which has appeared in the above stages of the report to display the links between classes, UI and the database (thus these queries).

1. Selecting the assignments to display on the teacher dashboard

```
SELECT Mark, DefinedAssignments.Title, DefinedAssignments.DateSet,
Users.Firstname, Users.Surname, DefinedAssignments.TotalMarks
FROM LinkedAssignments
INNER JOIN Users ON Users.UserID = LinkedAssignments.UserID
INNER JOIN DefinedAssignments ON DefinedAssignments.AssignmentID =
LinkedAssignments.AssignmentID
WHERE DefinedAssignments.TeacherID = @teacherid AND Mark IS NOT NULL
ORDER BY DefinedAssignments.DateSet;
```

2. Creating an assignment

```
INSERT INTO DefinedAssignments (TeacherID, Title, DateSet,
OriginalFile, TotalMarks, FileName)
VALUES (@userID, @title, @today, @binFile, @maxMarks, @filename);
SELECT CAST(SCOPE_IDENTITY() as int);
```

3. Retrieving assignments

```
SELECT Title, DateSet, TotalMarks, AssignmentID, Users.Firstname,
Users.Surname
FROM DefinedAssignments
INNER JOIN Users ON DefinedAssignments.TeacherID = Users.UserID
ORDER BY DateSet;
```

4. Adding a new user to the Users table

```
INSERT INTO Users (Username, HashedPass, Firstname, Surname, Role)
VALUES (@uname, @pword, @fname, @sname, @role);
```

5. Getting all the lesson titles for a given syllabus

```
SELECT SyllabusLessons.LessonID, SyllabusLessons.LessonNo,
DefinedLessons.Title
FROM SyllabusLessons, DefinedLessons
WHERE SyllabusLessons.LessonID = DefinedLessons.LessonID AND
SyllabusLessons.LessonLevel = @level
ORDER BY lessonNO";
```

6. Updating the audit log when someone logs out

```
UPDATE AuditLog SET LogoutTime = @logoutTime WHERE LogoutTime is
NULL and LoginID = @loginID;
```

7. Checking the user's details on log-in

```
SELECT HashedPass, UserID, Username, Firstname, Role FROM Users
WHERE Username = @username;
```

8. Adding the user's log-in time to the AuditLog table

```
INSERT INTO AuditLog(UserID, LoginTime) Values(@uID, @time);
SELECT CAST(SCOPE_IDENTITY() as int);
```

9. Getting the lesson to display

```
SELECT Text, Title, WorkPrompt, Image FROM DefinedLessons WHERE
lessonID = @lessonID;
```

10. Getting the current user's total time spent on the application

```
SELECT COALESCE(SUM(DATEDIFF(MINUTE, LoginTime, LogoutTime)), 0)
FROM AuditLog WHERE UserID = @uID AND LogoutTime IS NOT NULL; //
COALESCE means if there are no entries 0 is used
```

11. Getting the average user's total time spent on the application

```
SELECT COALESCE(SUM(DATEDIFF(MINUTE, LoginTime, LogoutTime)), 0),
COALESCE(COUNT(DISTINCT UserID), 0) FROM AuditLog WHERE LogoutTime
IS NOT NULL; // distinct ensures we don't count only
```

User Interface

UI Flow Diagram

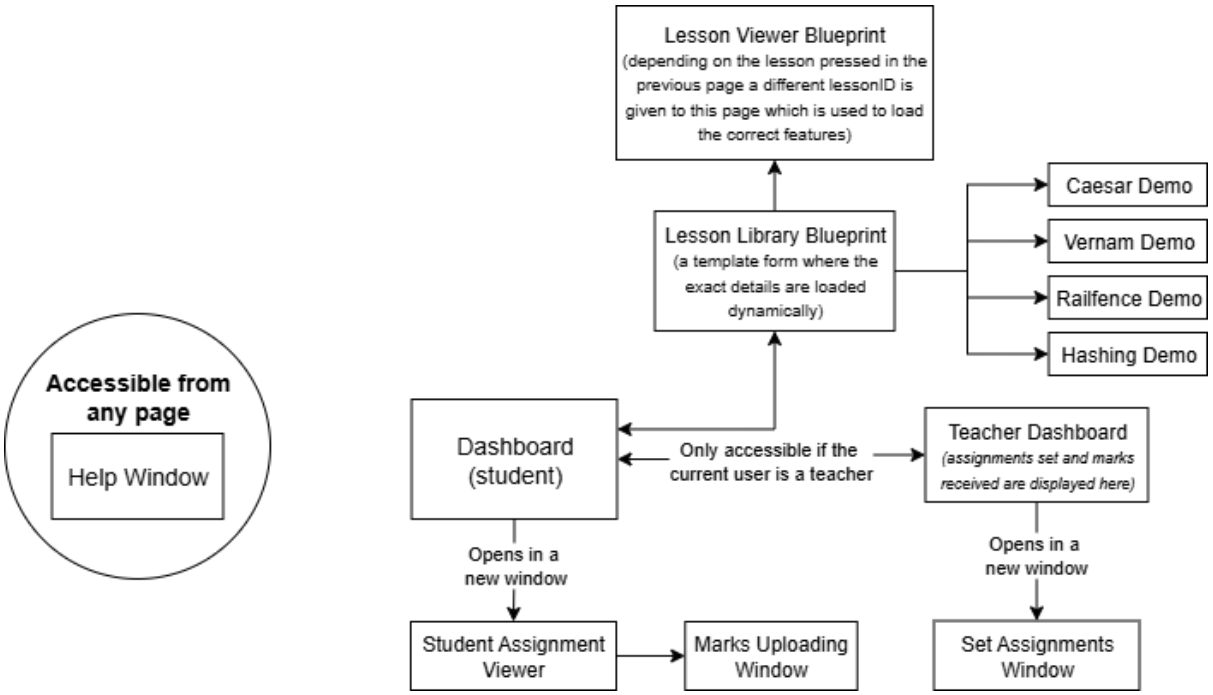


FIGURE 20 – THE UI FLOW DIAGRAM OF MY APPLICATION. THIS FOLLOWING DIAGRAM SHOWS HOW THE PAGES LINK TO EACH OTHER AND AS A RESULT HOW THE USER CAN BROWSE THE APPLICATION.

Annotated User Interface

Links to the database have been shown in blue pen – the numbers correspond to the later 'SQL Query Design' section, where the key queries have been drafted and numbered.

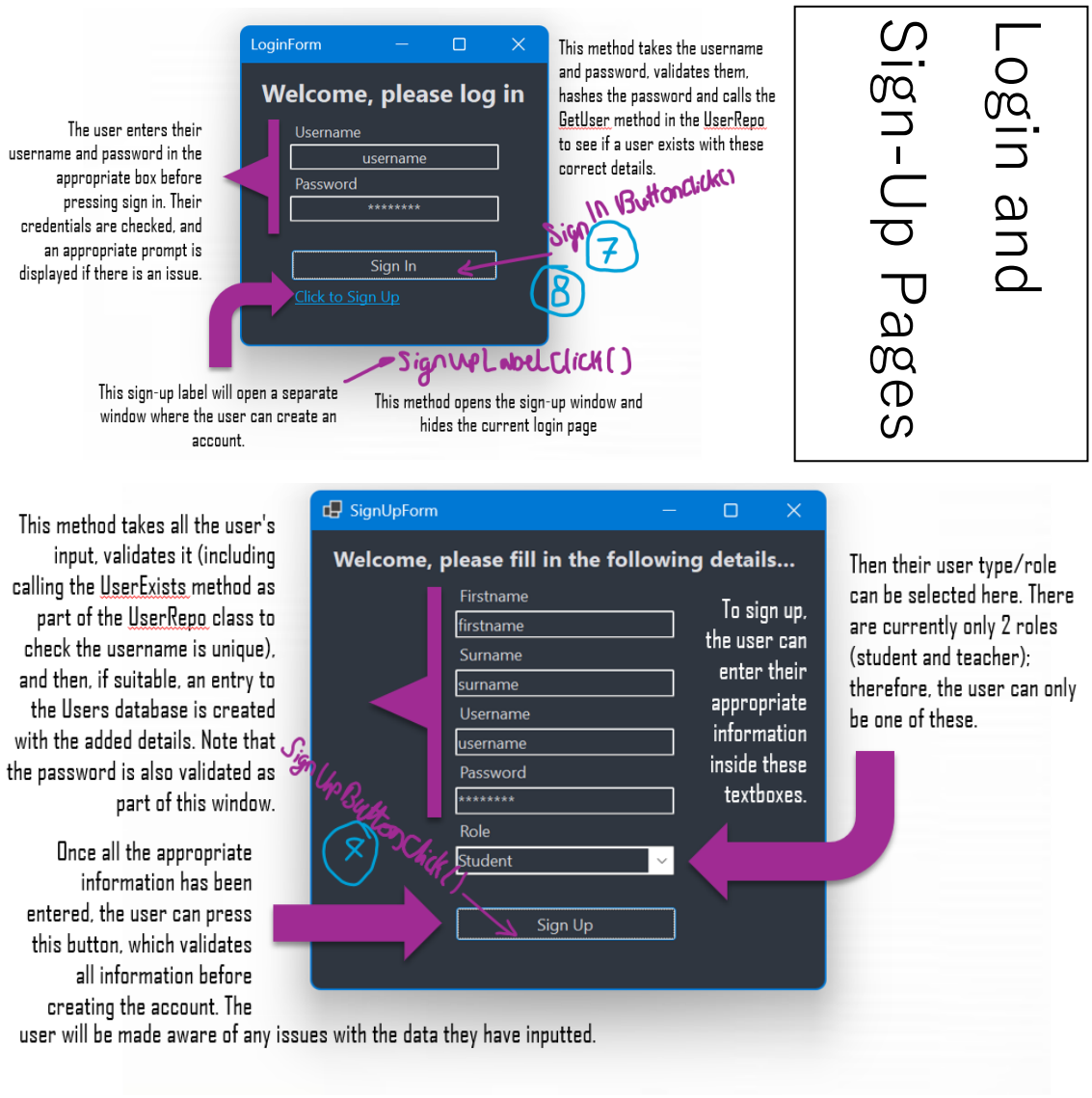


FIGURE 21 – AN INITIAL DESIGN OF MY LOGIN AND SIGNUP FORMS

The General Dashboard and Help Page

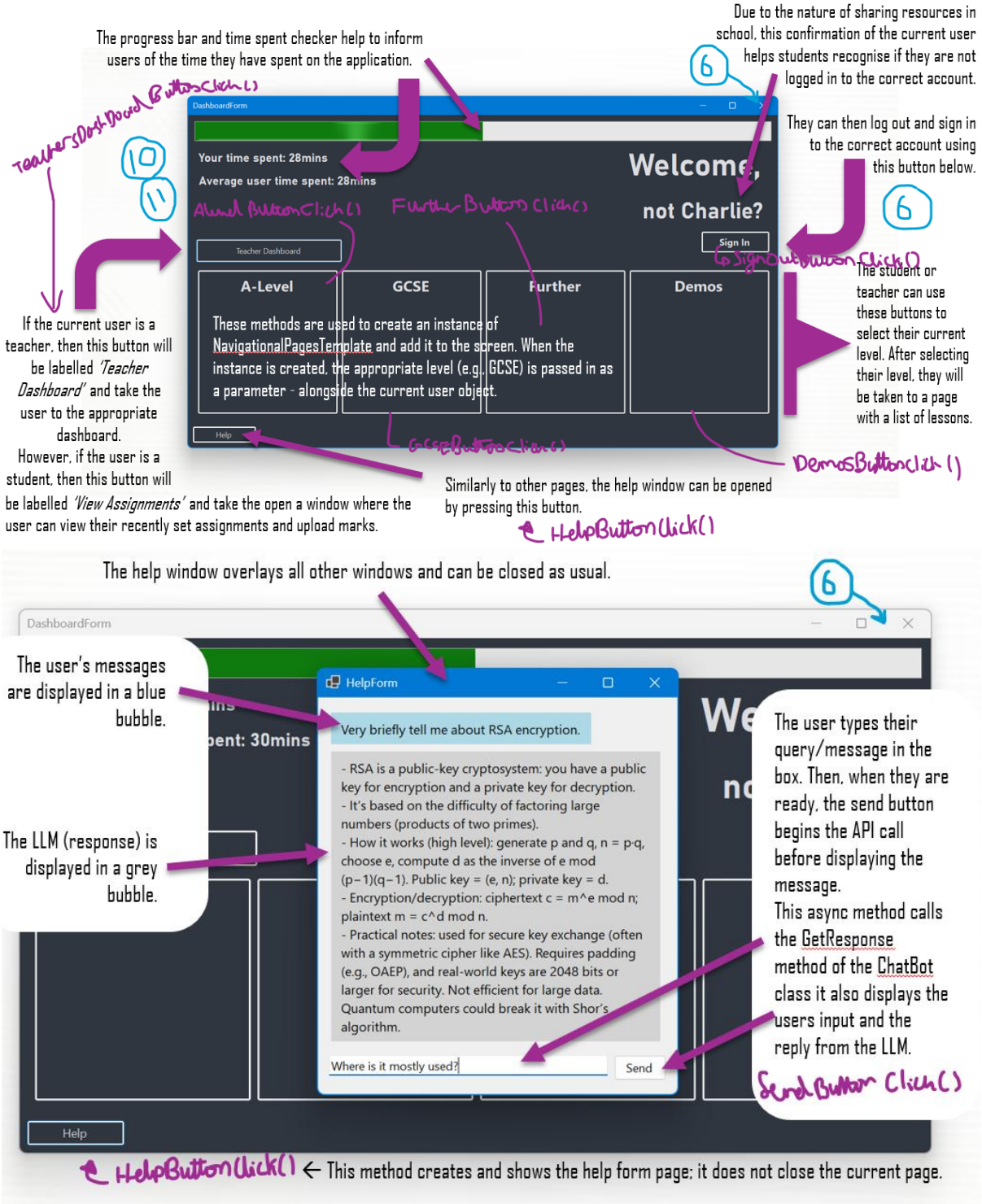


FIGURE 22 – AN INITIAL DESIGN OF MY GENERAL DASHBOARD AND HELP PAGE

Lesson Selector and Viewer

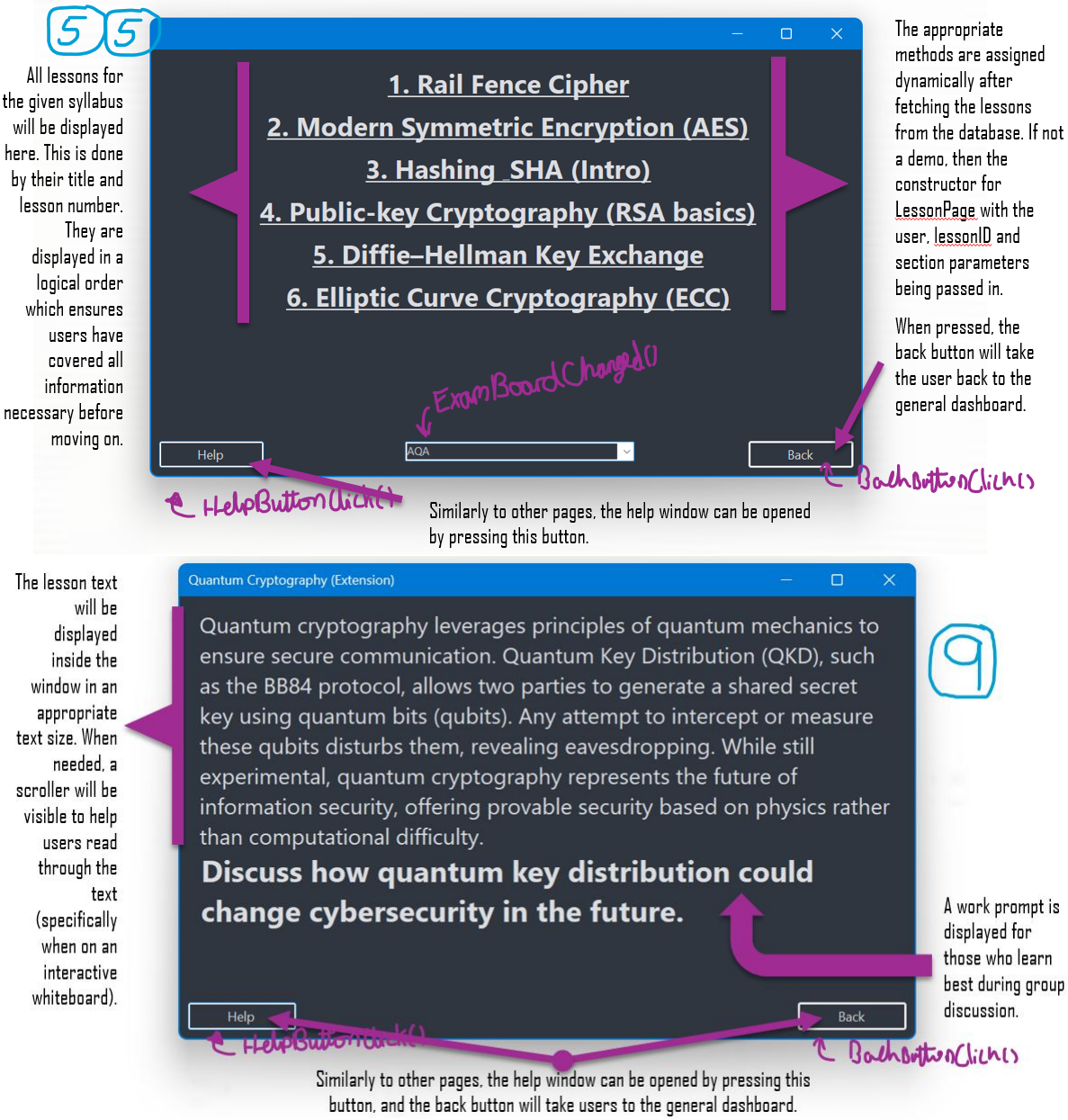


FIGURE 23 – AN INITIAL DESIGN OF MY LESSON PAGES (INCLUDING THE SELECTOR)

Algorithm Demonstration Pages

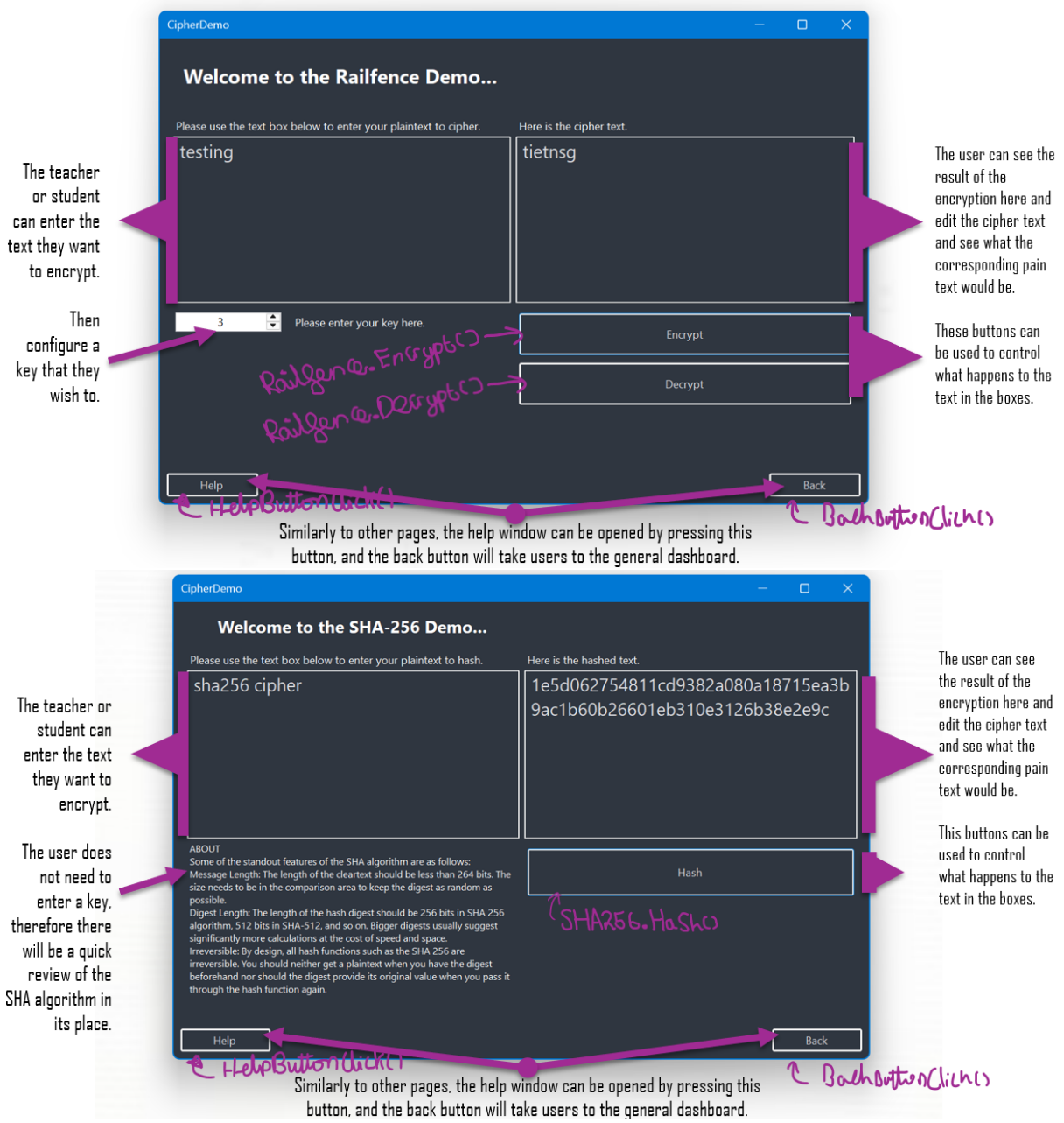


FIGURE 24 – AN INITIAL DESIGN OF MY RAILFENCE AND SHA-256 DEMONSTRATION PAGES

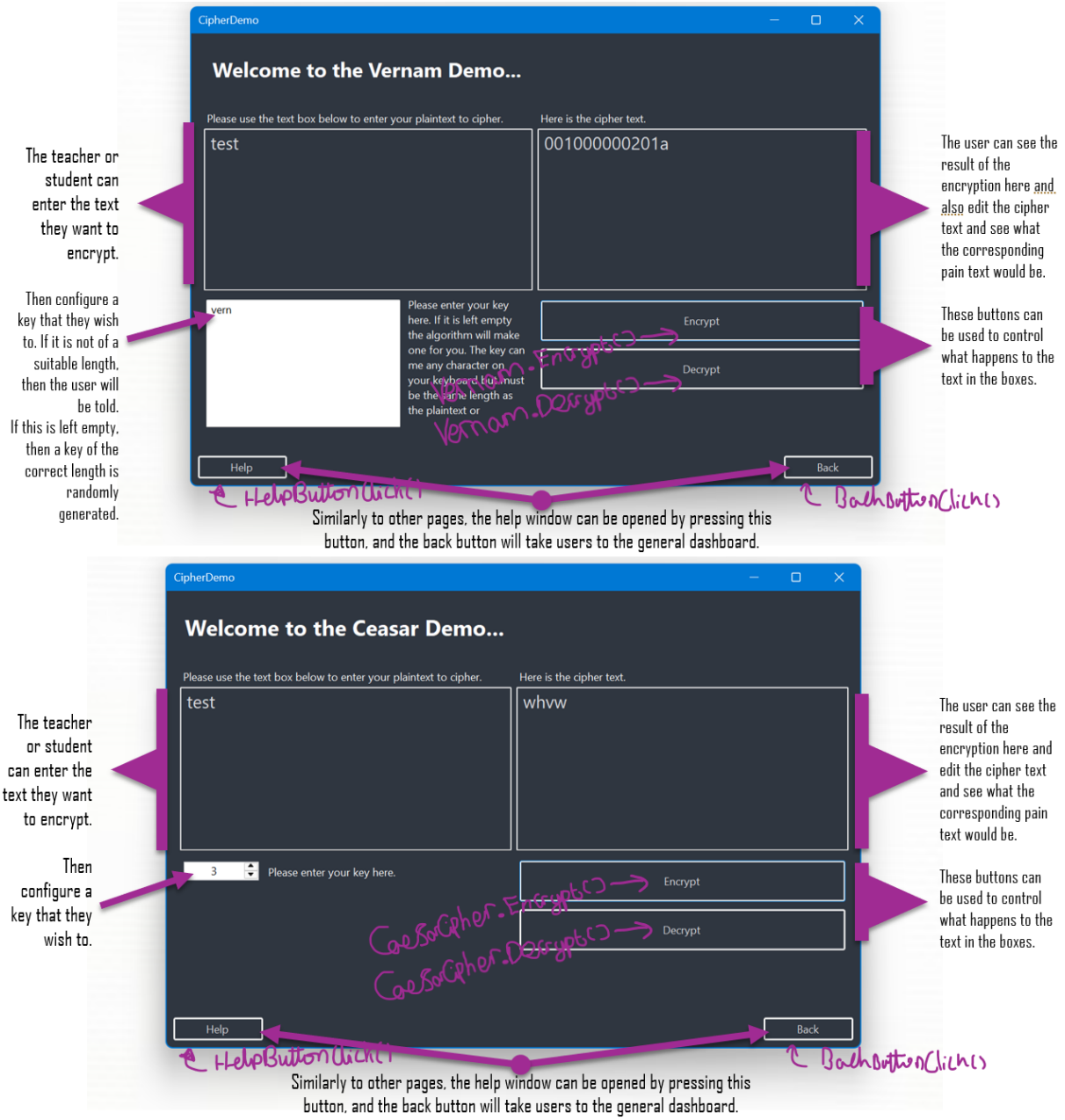


FIGURE 25 – AN INITIAL DESIGN OF MY VERNAM AND CAESAR PAGES

Teacher Dashboard and Assignment Setter

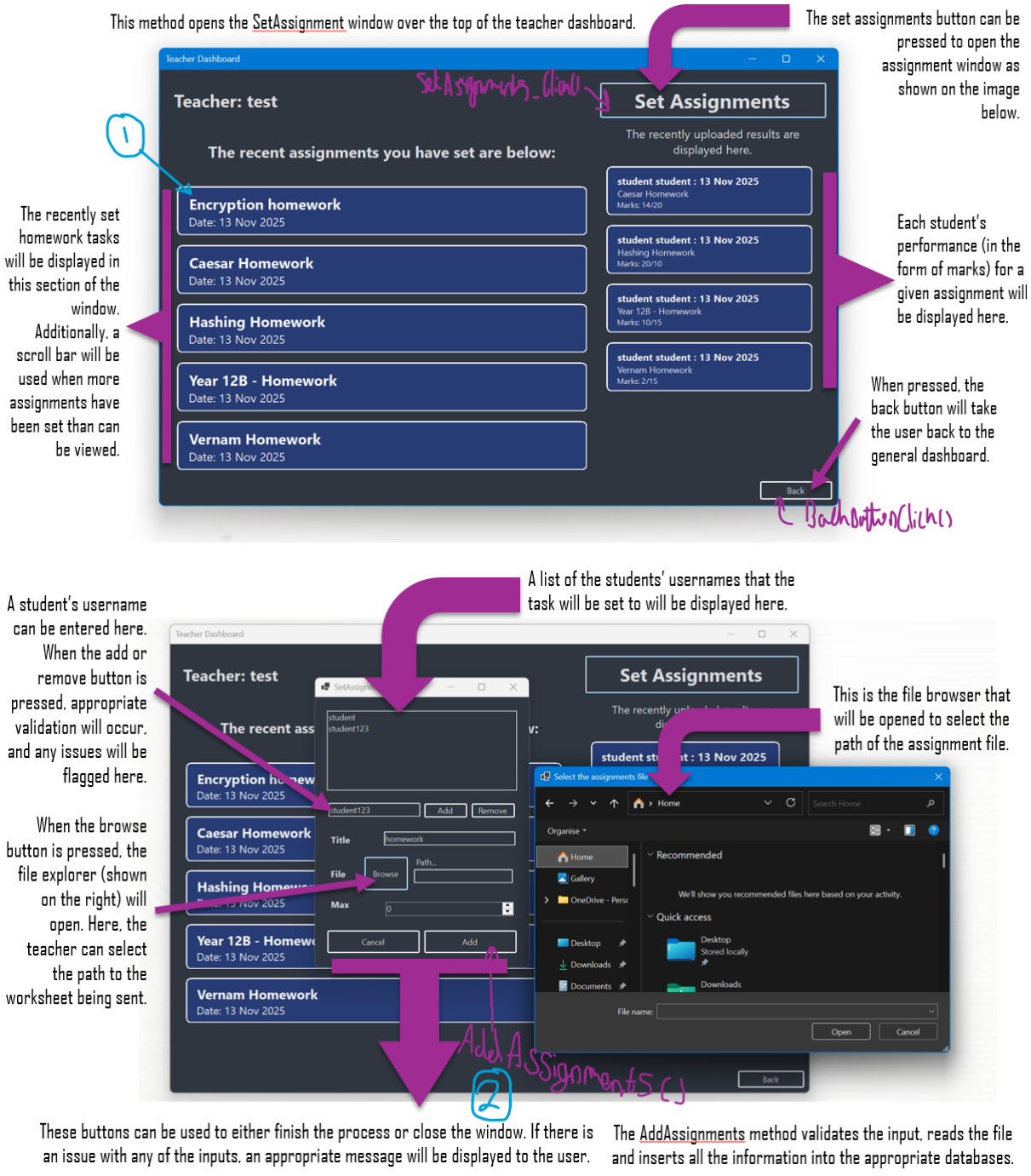


FIGURE 26 – AN INITIAL DESIGN OF MY TEACHER DASHBOARD AND ASSIGNMENT SETTER PAGES

Assignment Viewer

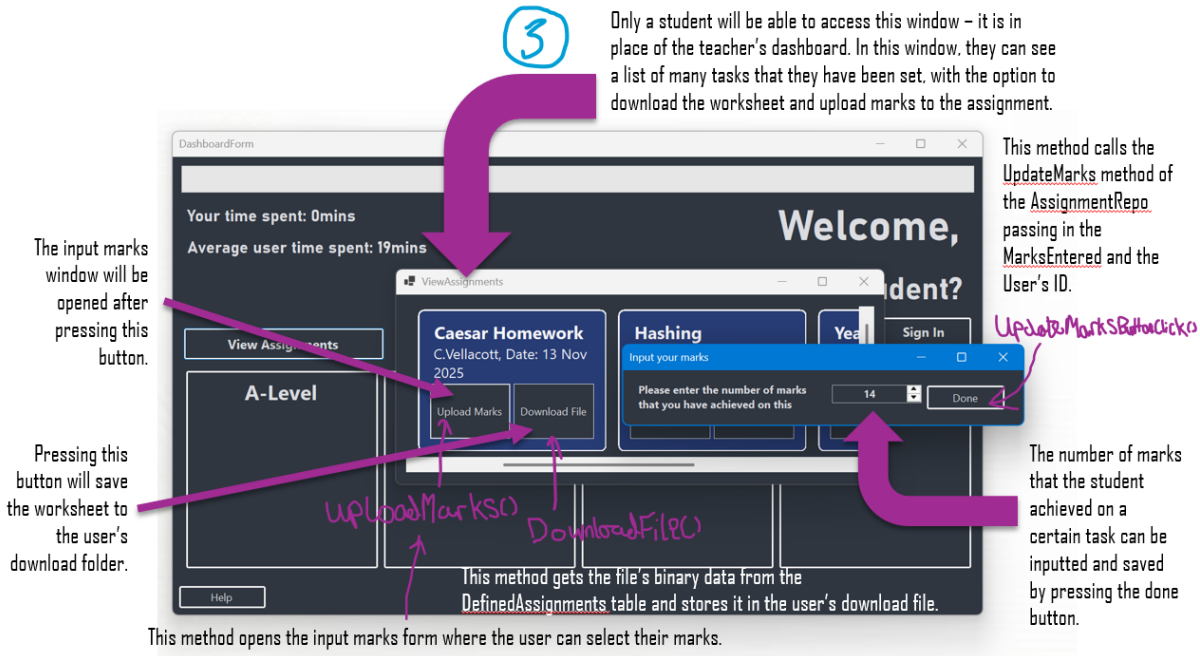


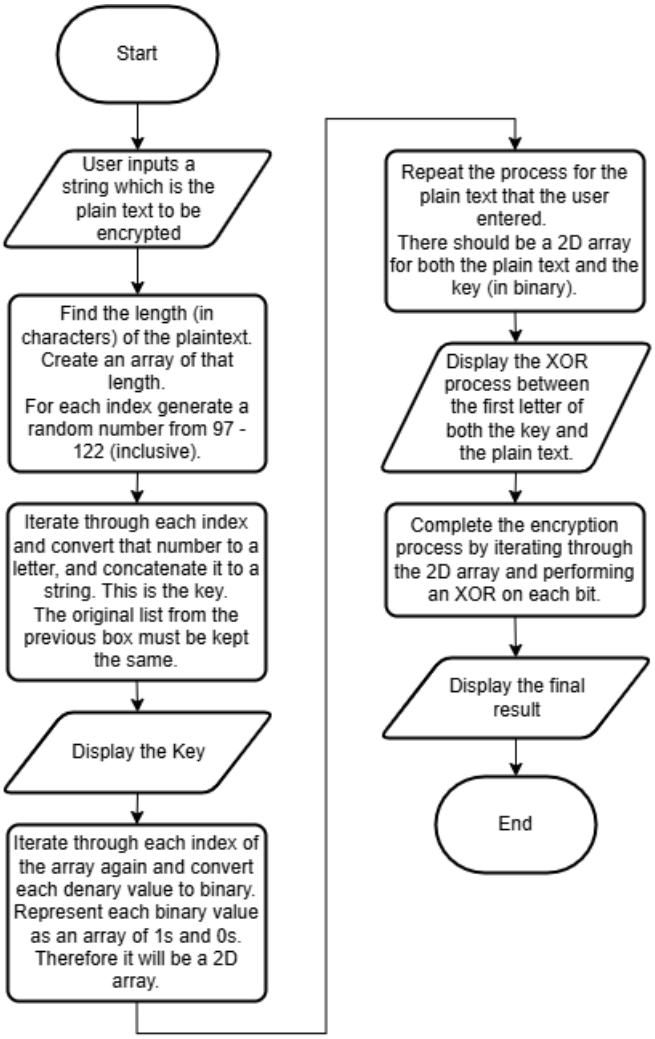
FIGURE 27 – AN INITIAL DESIGN OF MY ASSIGNMENT VIEWER AND MARK UPLOADING WINDOW

Algorithms Design

Creating the Ciphers

Vernam

The Vernam cipher can be split up into the following steps:



I created this flowchart with objectives for my encryption techniques explicitly in mind. These are things like viewing the processes that compose the cipher.

Note that appropriate validation is not mentioned above, but before beginning the flowchart, the following must be noted:

- The input must not be null, and the number of chars inputted must be ≥ 1.
- The plain text can include non-alpha characters, including numbers, punctuation or spaces.
- The output must also be binary (only 1s and 0s) or hexadecimal.

FIGURE 28 – A DRAFT FLOWCHART FOR THE PROCESS OF MY VERNAM CIPHER

Caesar

To ensure that the non-alpha characters are not shifted accidentally into a character that the application cannot correctly display, I will add each of their ASCII values into a list and replace them with a “-1” in the array. Then when shifting I will not apply anything if the value being looked at is equal to -1. Finally, I will iterate through the list and add the old ASCII values where a “-1” is in place, before converting the values back to a character.

Note that appropriate validation is not mentioned above but before beginning the flowchart the following must be noted:

- The input must not be null and the number of chars inputted must be ≥ 1.
- The key must be an integer ≥ 1 and given by the user.
- The plain text can non-alpha characters, including numbers, punctuation or spaces.
- The output must also be a string of characters.

Equally, the Caesar cipher can be split up into the following steps:

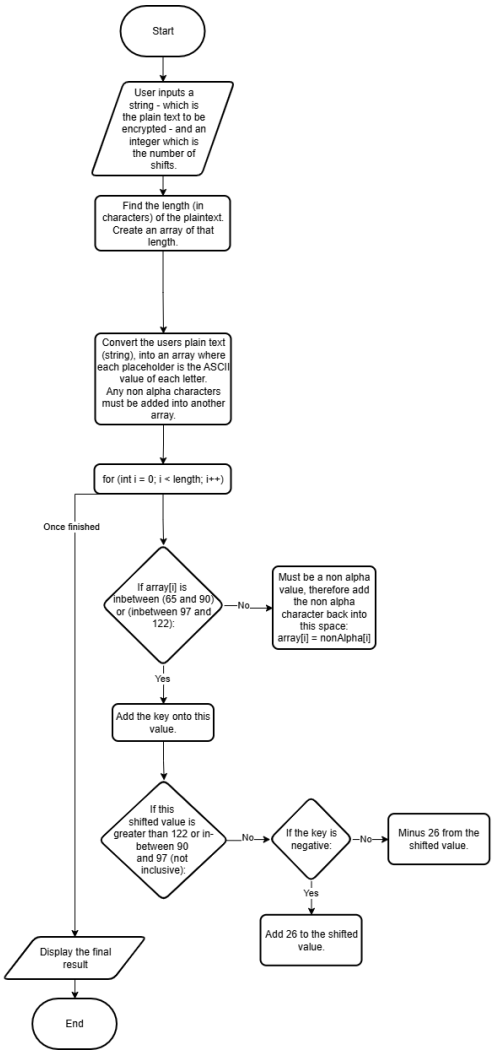
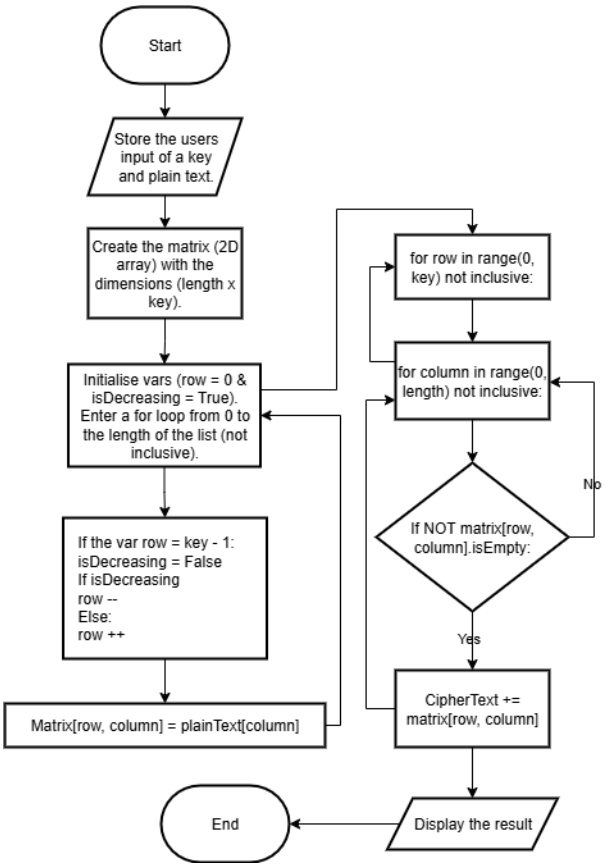


FIGURE 29 – A DRAFT FLOWCHART FOR THE PROCESS OF MY CAESAR CIPHER

RailFence



This cipher can be split into the following steps. Note that validation of the key size and plaintext length will be needed but has not been modelled. In addition, this algorithm has no requirement to add validation for the presence of non-alphabetic characters; this is because it revolves around manipulating each character's position in a string, not each character's type.

FIGURE 30 – A DRAFT FLOWCHART FOR THE PROCESS OF MY RAILFENCE CIPHER

Deep Dive on the SHA-256 Hash

This hashing algorithm will be fundamental to my application. Not only will it be used as a demo to the users, explaining how the hash works, but it will also be used to hash the user's passwords for the database. In this section I will be looking in depth at how the algorithm will work – but more specifically on the process of hashing the user's passwords during login and sign-up.

UI

This process all begins with the UI, where the user will call either the sign-in or sign-up methods.

When the sign-in method is called, the following will happen:
Firstly, the username and password will be taken from their appropriate text boxes and stored as variables (note that as soon as the password variable is no longer needed, it will be cleared). The details will be validated (for example, checking that neither is null) before an object of type SHA256 (my hashing algorithm's class) will be created. The password will be passed into this, and the hash will be created.

When the sign-up method is called, a similar process will happen:
All details will be taken from their appropriate text boxes, stored as variables and validated. Before an object of type SHA256 is created, the hash function is called with the passwords passed in as an argument.

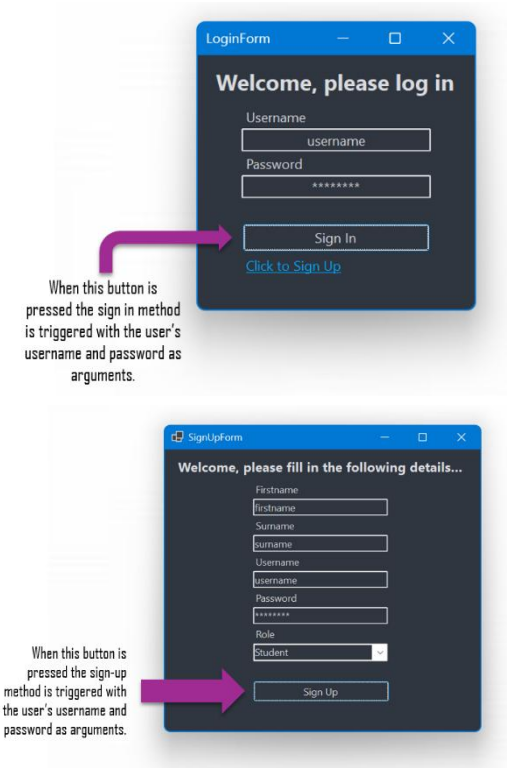


FIGURE 31 – THE USER INTERFACE BEHIND MY LOGIN AND SIGNUP PAGES IN RELATION TO THE HASH

Algorithm

I will now outline the algorithm, using a flow chart I created to aid the design process of my algorithm. The flowchart was made with inheritance from a generic cryptography algorithm class in mind. This class will contain methods that will be used across many algorithms (for example, an XOR method, a binary to hex method, etc).

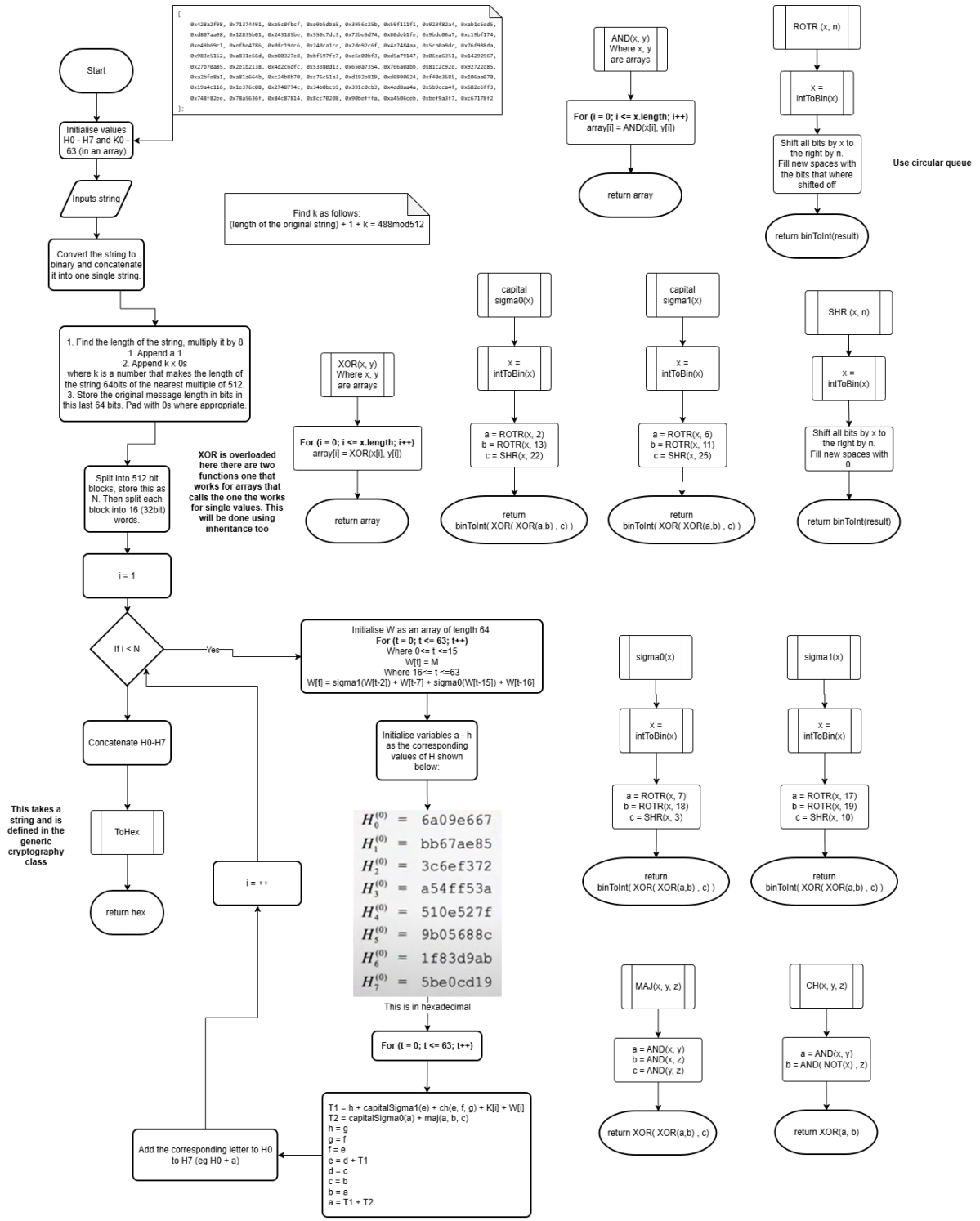


FIGURE 32 – A FLOWCHART FOR THE PROCESS OF MY SHA256 HASH. NOTE: DURING CODING AND TESTING I FOUND VARIOUS ISSUES/ERRORS WITH THIS FLOWCHART – THESE ISSUES WERE CORRECTED IN THE IMPLEMENTATION/CODE BUT NOT ON THE ABOVE IMAGE.

After creating this flow chart, I will be adding the following methods to my generic encryption class:

- AND(x,y) where x and y are of type int
- AND(x,y) where x and y are of type int[] (calls AND(int,int) for each item)
- XOR(x,y) where x and y are of type int
- XOR(x,y) where x and y are of type int[] (calls XOR(int,int) for each item)
- binToHex(x) where x is of type string (returns a string)
- hexToBin(x) where x is of type string (returns a string)
- binToInt(x) where x is of type int[] (returns an int)
- intToBin(x) where x is of type int (returns an int[])

Equally, in the hashing algorithm class I will have:

- capitalSigmaOne(x) where x is of type int (works with binary but returns an int)
- capitalSigmaZero(x) where x is of type int (works with binary but returns an int)
- sigmaOne(x) where x is of type int (works with binary but returns an int)
- sigmaZero(x) where x is of type int (works with binary but returns an int)
- SHR(x,n) where x and n are of type int (works with binary but returns an int)
- ROTR(x,n) where x and n are of type int (works with binary but returns an int)

Database

Continuing the sign-in method, once the hash has been created and stored, a connection to the database is created, and the HashedPass, UserID, Username, Firstname and Role are all selected from it where the username entered matches a username in the table. The two hashed passwords (the one entered by the user and the one stored in the database) are compared; if they match, the user is logged in. If they do not, then a suitable message is displayed to them.

Now, looking at the sign-up method, an entry to the Users database is created with the recently entered details as attributes, with one of them being the hashed password. The user is then returned to the log-in page, where they can now enter their details.

Data Security and Robust Programming

Parameterised Queries

SQL injection is a risk whenever insert or select statements are used. Therefore, to mitigate this, I will be using Parameters.AddWithValue. This function replaces a given substring inside the query string with the parameter's value, after running validation to ensure that the value will not pose a risk of injection.

Robustly Closing Connections and Scope Identities

When I need to open an SQL connection or command, I will use the using command, as this ensures that once the appropriate lines have been run, the connection is closed immediately. This proper disposal prevents resource leaks. The same applies to accessing files. When reading the data from the teacher's file into a binary array, I will make use of the using statement.

In addition to preventing connections from accidentally being left open, I will be aware of the number of connections and queries that I make. An example is using scope identity to reliably get the ID of the most recently inserted record, rather than re-querying for it.

UI Authorisation

If a student or teacher got access to the wrong dashboard, it would be likely that the SQL commands would crash or return nothing, and this would also allow for incorrect usage of the application. Therefore, I will ensure I have appropriate measures in place to only show a button if the current user is of the appropriate type.

Hashing Passwords

It would be unsafe to store passwords as plain text in the database. Therefore, I will be creating a SHA256 algorithm to hash the passwords before storing or comparing them.

Storing Keys

Simply defining the API keys as a variable (before runtime) is insecure. Instead, I will be storing keys using either environment variables or text files. The same may apply for the connection string. For this initial version, my database will be stored locally, thus no security benefit would be provided. Although if I continued development, adding remote servers, then I would certainly look to store the connection string, as I will store the API key.

Read-Only

I want to ensure there is no possibility that the initial hash values and round constants used in my SHA algorithm can change. Therefore, I will set the lists as read-only using the keyword in C#.

EG) private static readonly uint[] InitialValues = new uint[]

Unsigned Integers

During the SHA algorithm, I will be frequently converting between binary and integers. As a result, there could be logical errors (with sign bits) when converting between the two. Therefore, I will use the data type uint to represent 32-bit words and prevent sign issues during bitwise operations.

Validation

1 Pages

1.1 Sign-up fields

1.1.1 Presence check ~ this check should occur to ensure that all fields of username, password first name, etc are not empty or null.

1.1.2 Uniqueness check ~ the username inputted when creating an account must not already be in use

1.1.3 Password criteria ~ a check must occur to ensure that the password is of the correct strength:

- 1.1.3.1 Minimum length of 8 chars
- 1.1.3.2 Must contain at least one uppercase letter
- 1.1.3.3 Must contain at least one lowercase letter
- 1.1.3.4 Must contain at least one digit (0-9)
- 1.1.3.5 Must contain at least one symbol

1.1.4 SQL injection prevention ~ parameterised queries must be used to validate the input

1.2 Teacher Dashboard

1.2.1 The user's role must be validated before allowing access to the teacher dashboard

1.2.2 The username must be verified, ensuring that it exists, before adding to the assignment

1.2.3 Seeds must be used to prevent an assignment being created with a non-unique ID

1.2.4 The max mark must be a positive integer

1.3 Progress Tracking and Audit Logging

1.3.1 Outlier handling ~ if a user's total time spent is greater than 2x the average they should be plotted at the 100% mark rather than attempting to plot it at the 100+% mark.

2 Demonstrations & Teaching Tools

2.1 Caesar Cipher

2.1.1 Non-alpha chars ~ if found in the Caesar cipher's input, they must not be shifted

2.1.2 Key validation ~ the key should be properly accounted for regardless of its sign (positive or negative)

2.2 Vernam Cipher

2.2.1 Key and plain text ASCII char range ~ inform the user if and which char in the text or key has a value greater than 256

2.2.2 Ensure the key length is the same as the plain or cipher text length – if this is not the case the user should be told

2.2.3 Decrypting non-hexadecimal cipher text ~ if the user enters any values which are not hexadecimal in the cipher text, they should be told so.

2.3 Railfence Cipher

2.3.1 The key must be validated to ensure that only an integer key can be entered

2.3.2 Range check ~ the key must be greater than or equal to 1

2.4 SHA-256 Hash

2.4.1 Length check ~ ensure that the output is exactly 64 hexadecimal chars – if necessary, padding should be applied

Namespace Design

During the final stages of building the application, I came across a significant number of errors related to inconsistent namespaces across the project. My project was previously called 'InitialUI' – from the earlier stages of development – however sometimes it would appear as 'InitalUI' (missing the second 'i'). The result of this was two entirely separate namespace hierarchies existing within my project. After refactoring 'InitalUI' to 'InitialUI', I ended up with even more errors. Partial classes became broken as I changed the main code files' namespace because I did not think to change the designer, too. This led to the inheritance chain from Form being broken and issues with methods not being visible. To fix this issue, I had to systematically refactor the project, creating a new and refined namespace system. This was time-consuming and ultimately led to a greater understanding of the necessity of namespace design.

Here is the namespace design I created in the later stages – note '*InitialUI*' has changed to '*CryptographyTeachingTool*':

CryptographyTeachingTool

- CryptographyTeachingTool.MainClasses
    - Contains key classes like User and Assignment
- CryptographyTeachingTool.Data
    - Contains the DataClasses class (all repositories' parent class)
    - CryptographyTeachingTool.Data.Repositories

        - Contains all repositories
- CryptographyTeachingTool.Cryptography
    - Contains the CryptographicAlgorithm class (all cipher and hash parent classes)
    - CryptographyTeachingTool.Cryptography.Ciphers
        - Contains all the ciphers such as Vernam and Caesar
    - CryptographyTeachingTool.Cryptography.Hash
        - Contains the SHA256Hash class
- CryptographyTeachingTool.UI
    - CryptographyTeachingTool.UI.Basic
        - Contains all key Forms inheriting parent classes and the General Dashboard
        - CryptographyTeachingTool.UI.Teacher
            - Contains the teacher dashboard and assignment setter
        - CryptographyTeachingTool.UI.Student
            - Contains the assignment viewer and marks uploader
        - CryptographyTeachingTool.UI.Demonstrations
            - Contains all teaching tools and the base demonstration page
- CryptographyTeachingTool.Login
    - Contains the Login and Signup forms pages
- CryptographyTeachingTool.Help
    - Contains the Chatbot class and Help forms page

Use of AI in the Database

Creating the lessons for the database is a tedious process, and for this prototype, it does not need to be manually done; however, for deployment, it would be sensible to create the lessons manually to ensure the balance of correctness, conciseness and completeness. As a result, I used OpenAI's ChatGPT LLM to write SQL queries to fill the database below. I will show the query that I made to the LLM and the SQL queries that I ran as part of filling the database.

Note: this was before I had implemented some other columns (such as the images) and tables (for example the AuditLog). Not all entries to the database were made by AI – in the end it was only really the lessons.

The Query Sent

I will give you my current program for my NEA cryptography teaching tool, which is coded in C# using visual studio. I want you to create the data for my database tables. You must create the data in as much detail as possible, following the write up document below. You must create an SQL query to add the data to my databases. Here is the write up, the .zip of my project is attached, go through the code so that you get an understanding on the layout of the databases.

After this I then gave it my objectives…

The Response

Awesome — I dug into your code and pulled out the actual table/column names your forms query:

- **Users**: `UserID`, `Username`, `Password`, `Firstname`, `Lastname`, `Role` (from `LoginForm.cs`).
- **Lessons** (4 tables the app reads via `{section}_Lessons`): `GCSE_Lessons`, `Alevel_Lessons`, `Further_Lessons`, `Demos_Lessons` with columns `lessonID`, `lessonName`, `lessonText` (from `NavigationalPagesTemplate.cs` and `LessonViewer.cs`).
- **Assignments (Teacher view)**: `DefinedAssignments` with `AssignmentID`, `Title`, `DateSet`, `TotalMarks`, `TeacherID`, and `LinkedAssignments` with `UserID`, `AssignmentID`, `Mark` (plus a file column that can be NULL) — inferred from the `fetchAsignmentsSet` and `fetchResults` queries in `TeachersDashboard.cs`.

I've created a **full T-SQL seed script** that:

- Seeds **teachers + students** into `Users`.
- Populates **detailed lesson content** (titles + rich lesson text) across **GCSE, A-Level, Further, and Demos** tables, aligned to your write-up (plaintext vs ciphertext clarity, Caesar/Vernam/Rail Fence, symmetric/asymmetric, hashing, RSA, DH, PKI, attacks, etc.).
- Inserts **teacher-defined assignments** into `DefinedAssignments`.
- Links **student results** (marks) in `LinkedAssignments`, leaving any file/upload column **NULL**, as requested.
- Uses `OUTPUT inserted...` into table variables so foreign keys are wired up **without guessing identity values**.
- Wraps everything in a **single transaction** and includes a simple guard to avoid double-seeding.

You can download and run it as-is in SQL Server:

Download the SQL seed script

If you prefer to paste directly into SSMS, here's the script inline (identical to the file):

```
/* Seed data for NEA Cryptography Teaching Tool
   Targets tables seen in codebase:
   - Users
   - GCSE_Lessons
   - Alevel_Lessons
   - Further_Lessons
   - Demos_Lessons
   - DefinedAssignments
   - LinkedAssignments

   Notes:
   * File/Upload column in LinkedAssignments (if present) is left NULL as
requested.
```

```
   * Uses OUTPUT into table variables to capture generated IDs without
assuming IDENTITY settings.
   * Safe to run multiple times if you change the WHERE NOT EXISTS guards.
*/

BEGIN TRAN;

IF EXISTS (SELECT 1 FROM sys.tables WHERE name = 'Users')
BEGIN
    IF EXISTS (SELECT 1 FROM Users WHERE Username = N'teacher.alice')
    BEGIN
        PRINT 'Seed appears to have been applied already (found user
teacher.alice). Aborting seed.';
        ROLLBACK TRAN;
        RETURN;
    END
END

DECLARE @Users TABLE (UserID INT, Username NVARCHAR(100));

INSERT INTO Users (Username, Password, Firstname, Lastname, Role)
OUTPUT inserted.UserID, inserted.Username INTO @Users(UserID, Username)
VALUES
    (N'teacher.alice', N'Password123!', N'Alice', N'Nguyen', N'Teacher'),
    (N'teacher.ben',   N'Password123!', N'Ben',   N'Kaur',   N'Teacher'),
    (N'student.chloe', N'Pa55word!',    N'Chloe', N'Evans',  N'Student'),
    (N'student.dan',   N'Pa55word!',    N'Dan',   N'Patel',  N'Student'),
    (N'student.emma',  N'Pa55word!',    N'Emma',  N'Wong',   N'Student'),
    (N'student.finn',  N'Pa55word!',    N'Finn',  N'Jones',  N'Student'),
    (N'student.grace', N'Pa55word!',    N'Grace', N'Garcia', N'Student'),
    (N'student.hugo',  N'Pa55word!',    N'Hugo',  N'Brown',  N'Student');

-- GCSE lessons
INSERT INTO GCSE_Lessons (lessonName, lessonText) VALUES
(N'1. What is Cryptography?',
N'[Overview]
Cryptography is the practice of protecting information by transforming it
so only intended parties can read it.
You''ll learn: plaintext, ciphertext, keys, and algorithms.

[Key Ideas]
• Plaintext → original message.
• Ciphertext → unreadable output after encryption.
• Key → secret value that controls encryption/decryption.
• Algorithm → step-by-step method that performs encryption or decryption.

[Discussion Prompt]
Where in everyday life might encryption be used (phones, banking,
messaging)?'),
(N'2. Plaintext vs Ciphertext',
N'[Definition Check]
• Plaintext: "HELLO"
• Ciphertext: "KHOOR" (after a shift of +3)

[Why Separate Clearly?]
Your teacher emphasised students confuse these terms.
We''ll label them explicitly in every example.

[Try It]
Take "CAT" and imagine it becomes "FDW"—is that plaintext or ciphertext?'),
(N'3. Caesar (Shift) Cipher – Basics',
```

```
N'[Goal]
Encrypt uppercase alphabetic text by shifting each letter by a fixed
number.

[Rules]
• Accept A–Z only (no digits, spaces, punctuation).
• If a non-letter is entered, the tool shows an error (as per Objective
7.2).

[Example]
Plaintext: HELLO, Shift: +3 → Ciphertext: KHOOR'),
(N'4. Caesar – Step-by-Step Walkthrough',
N'[Demonstration]
We step through the first character to show how the cipher moves through
the alphabet:
1) Take H.
2) Map to index 7 (0-based A=0).
3) Add shift +3 → 10.
4) Wrap within 0–25 → 10 corresponds to K.
Repeat per character.

[Discussion Prompt]
What happens with a negative shift?'),
(N'5. Frequency Analysis (GCSE)',
N'[Concept]
Some ciphers (like Caesar) leak letter patterns. English text has common
letters (E, T, A).

[Task]
Given a ciphertext, which letter occurs most? Guess the likely shift that
produced it.'),
(N'6. Keys and Brute Force',
N'[Keyspace]
Caesar has only 26 possible keys (shifts).

[Brute Force]
Try all shifts and pick the one that makes sense. Why is this weak compared
to modern crypto?'),
(N'7. Ethical & Legal Use',
N'[Safeguards]
Encryption protects privacy but can be misused.
Know school policies and the law regarding data protection (e.g., GDPR
principles).'),
(N'8. Recap & Quiz',
N'[Quick Check]
1) Define plaintext and ciphertext.
2) What does a key control?
3) How many Caesar keys exist?
4) Why is frequency analysis useful?');

-- A-Level lessons
INSERT INTO Alevel_Lessons (lessonName, lessonText) VALUES
(N'1. Symmetric vs Asymmetric',
N'[Definitions]
• Symmetric: same key encrypts and decrypts (e.g., AES).
• Asymmetric: public key encrypts, private key decrypts (e.g., RSA).

[Pros/Cons]
Symmetric is fast but needs secure key sharing.
Asymmetric simplifies key distribution but is slower.'),
(N'2. Formal Shift (Caesar) Cipher',
```

```
N'[Algorithm]
E_k(x) = (x + k) mod 26
D_k(y) = (y - k) mod 26

[Exercise]
Encrypt "ATTACKATDAWN" with k=5.'),
(N'3. Vernam (One-Time Pad) – Theory',
N'[Core Idea]
Bitwise XOR with a truly random key of equal length to the message. Key
must never be reused.

[Properties]
• Perfect secrecy if keys are random, secret, and single-use.
• In practice, generating/distributing true random keys is hard.'),
(N'4. Vernam – Worked Example',
N'[Walkthrough]
Plaintext: "A" → 01000001
Random key:  10111010
XOR result:  11111011 → Ciphertext byte

[UI Tie-in]
Tool shows ASCII, binary, and XOR step-by-step.'),
(N'5. Rail Fence (Transposition)',
N'[Idea]
Rearrange (permute) character order using rows (rails).

[Mini-Task]
Encrypt "CRYPTOGRAPHY" with 3 rails.'),
(N'6. Stream vs Block Ciphers',
N'[Compare]
Stream: encrypts symbol-by-symbol (like Vernam stream).
Block: encrypts fixed-size blocks (like AES).

[Why Care?]
Mode selection impacts security (see ECB vs CBC).'),
(N'7. Modes of Operation (ECB/CBC concept)',
N'[Warning]
ECB reveals patterns (identical blocks → identical ciphertext blocks).
CBC uses an IV to randomise first block and chains subsequent blocks.'),
(N'8. Key Management & Distribution',
N'[Problem]
How do parties share keys securely? Options: pre-shared, Diffie–Hellman,
hybrid (RSA + AES).'),
(N'9. Exam Practice – Short Answers',
N'[Questions]
1) Why does reusing a Vernam key break security?
2) Explain one weakness of ECB mode.
3) Outline one benefit of asymmetric encryption.');

-- Further/Post A-Level lessons
INSERT INTO Further_Lessons (lessonName, lessonText) VALUES
(N'1. Cryptographic Hash Functions',
N'[Properties]
Deterministic, preimage-resistant, second-preimage-resistant, collision-
resistant.

[Use Cases]
Password storage (with salt), integrity checks, digital signatures.'),
(N'2. SHA Family & Collisions',
N'[Discussion]
SHA-1 is deprecated due to practical collision attacks.
```

```
Modern systems use SHA-256/512 (or SHA-3).'),
(N'3. Modular Arithmetic Basics',
N'[Maths]
a ≡ b (mod n). Fast exponentiation is critical for RSA.'),
(N'4. RSA – Key Generation',
N'[Steps]
1) Choose primes p, q.
2) n = p·q ; φ(n) = (p−1)(q−1).
3) Choose e coprime with φ(n).
4) Compute d ≡ e^{-1} (mod φ(n).)'),
(N'5. RSA – Encrypt/Decrypt',
N'[Formulas]
Ciphertext C = M^e mod n
Plaintext  M = C^d mod n

[Security]
Depends on factoring difficulty.'),
(N'6. Diffie–Hellman Key Exchange',
N'[Idea]
Two parties agree a shared secret over an insecure channel using g^ab mod
p.'),
(N'7. Digital Signatures',
N'[Concept]
Sign with private key, verify with public key.
Provides authenticity and non-repudiation.'),
(N'8. PKI & Certificates',
N'[Overview]
Certificate Authorities bind identities to public keys.
X.509 certificates underpin TLS.'),
(N'9. Attacks & Threat Models',
N'[Examples]
Chosen-plaintext, chosen-ciphertext, timing/side-channel, man-in-the-
middle.');

-- Demos (Tools) lessons
INSERT INTO Demos_Lessons (lessonName, lessonText) VALUES
(N'Caesar Tool – Usage',
N'[Input Rules]
• Uppercase letters only A–Z.
• No digits or punctuation.
[What You See]
• Step-by-step of first character.
• Both encryption and decryption.'),
(N'Vernam Tool – Usage',
N'[Key Generation]
• Random binary key of same length (0/1 only).
[Walkthrough]
• ASCII → binary → XOR → ciphertext.
• Show both directions (enc/dec).'),
(N'Rail Fence Tool – Usage',
N'[How To]
• Choose number of rails.
• See transposition step-by-step.
• Decrypt by reconstructing the rails.'),
(N'Comparing Outputs',
N'[Activity]
Encrypt the same message with Caesar vs Rail Fence.
Which leaks letter frequency more obviously?');

-- DefinedAssignments
DECLARE @Assignments TABLE (AssignmentID INT, Title NVARCHAR(200));
```

```
DECLARE @TeacherAlice INT = (SELECT TOP 1 UserID FROM @Users WHERE Username
= N'teacher.alice');
DECLARE @TeacherBen   INT = (SELECT TOP 1 UserID FROM @Users WHERE Username
= N'teacher.ben');

INSERT INTO DefinedAssignments (Title, DateSet, TotalMarks, TeacherID)
OUTPUT inserted.AssignmentID, inserted.Title INTO
@Assignments(AssignmentID, Title)
VALUES
    (N'GCSE Worksheet: Caesar Cipher Basics', CONVERT(date, '2025-09-10'),
20, @TeacherAlice),
    (N'A-Level Coursework: Vernam Implementation', CONVERT(date, '2025-10-
01'), 40, @TeacherAlice),
    (N'Further Task: RSA & Key Management Mini-Report', CONVERT(date,
'2025-10-15'), 30, @TeacherBen);

-- LinkedAssignments
DECLARE @sChloe INT = (SELECT TOP 1 UserID FROM @Users WHERE Username =
N'student.chloe');
DECLARE @sDan   INT = (SELECT TOP 1 UserID FROM @Users WHERE Username =
N'student.dan');
DECLARE @sEmma  INT = (SELECT TOP 1 UserID FROM @Users WHERE Username =
N'student.emma');
DECLARE @sFinn  INT = (SELECT TOP 1 UserID FROM @Users WHERE Username =
N'student.finn');
DECLARE @sGrace INT = (SELECT TOP 1 UserID FROM @Users WHERE Username =
N'student.grace');
DECLARE @sHugo  INT = (SELECT TOP 1 UserID FROM @Users WHERE Username =
N'student.hugo');

DECLARE @aCaesar INT = (SELECT AssignmentID FROM @Assignments WHERE Title =
N'GCSE Worksheet: Caesar Cipher Basics');
DECLARE @aVernam INT = (SELECT AssignmentID FROM @Assignments WHERE Title =
N'A-Level Coursework: Vernam Implementation');
DECLARE @aRSA    INT = (SELECT AssignmentID FROM @Assignments WHERE Title =
N'Further Task: RSA & Key Management Mini-Report');

INSERT INTO LinkedAssignments (UserID, AssignmentID, Mark) VALUES
    (@sChloe, @aCaesar, 18),
    (@sDan,   @aCaesar, 15),
    (@sEmma,  @aCaesar, 20),
    (@sFinn,  @aCaesar, 13),
    (@sGrace, @aCaesar, 19),
    (@sHugo,  @aCaesar, 16);

INSERT INTO LinkedAssignments (UserID, AssignmentID, Mark) VALUES
    (@sChloe, @aVernam, 34),
    (@sDan,   @aVernam, 27),
    (@sEmma,  @aVernam, 36),
    (@sGrace, @aVernam, 31);

INSERT INTO LinkedAssignments (UserID, AssignmentID, Mark) VALUES
    (@sDan,   @aRSA, 24),
    (@sEmma,  @aRSA, 28),
    (@sFinn,  @aRSA, 22),
    (@sHugo,  @aRSA, 25);

COMMIT TRAN;

PRINT 'Seed completed successfully.';
```

The Query Used

I did not run the entirety of this query, given by the LLM, simply because I was only really concerned about the lessons. I ended up inserting all demo lessons and users myself. Below is the query that I ran…

```
-- GCSE lessons
INSERT INTO GCSE_Lessons (lessonName, lessonText) VALUES
(N'1. What is Cryptography?',
N'[Overview]
Cryptography is the practice of protecting information by transforming it
so only intended parties can read it.
You''ll learn: plaintext, ciphertext, keys, and algorithms.

[Key Ideas]
• Plaintext → original message.
• Ciphertext → unreadable output after encryption.
• Key → secret value that controls encryption/decryption.
• Algorithm → step-by-step method that performs encryption or decryption.

[Discussion Prompt]
Where in everyday life might encryption be used (phones, banking,
messaging)?'),
(N'2. Plaintext vs Ciphertext',
N'[Definition Check]
• Plaintext: "HELLO"
• Ciphertext: "KHOOR" (after a shift of +3)

[Why Separate Clearly?]
Your teacher emphasised students confuse these terms.
We''ll label them explicitly in every example.

[Try It]
Take "CAT" and imagine it becomes "FDW"—is that plaintext or ciphertext?'),
(N'3. Caesar (Shift) Cipher – Basics',
N'[Goal]
Encrypt uppercase alphabetic text by shifting each letter by a fixed
number.

[Rules]
• Accept A–Z only (no digits, spaces, punctuation).
• If a non-letter is entered, the tool shows an error (as per Objective
7.2).

[Example]
Plaintext: HELLO, Shift: +3 → Ciphertext: KHOOR'),
(N'4. Caesar – Step-by-Step Walkthrough',
N'[Demonstration]
We step through the first character to show how the cipher moves through
the alphabet:
1) Take H.
2) Map to index 7 (0-based A=0).
3) Add shift +3 → 10.
4) Wrap within 0–25 → 10 corresponds to K.
Repeat per character.

[Discussion Prompt]
What happens with a negative shift?'),
(N'5. Frequency Analysis (GCSE)',
N'[Concept]
```

```
Some ciphers (like Caesar) leak letter patterns. English text has common
letters (E, T, A).

[Task]
Given a ciphertext, which letter occurs most? Guess the likely shift that
produced it.'),
(N'6. Keys and Brute Force',
N'[Keyspace]
Caesar has only 26 possible keys (shifts).

[Brute Force]
Try all shifts and pick the one that makes sense. Why is this weak compared
to modern crypto?'),
(N'7. Ethical & Legal Use',
N'[Safeguards]
Encryption protects privacy but can be misused.
Know school policies and the law regarding data protection (e.g., GDPR
principles).'),
(N'8. Recap & Quiz',
N'[Quick Check]
1) Define plaintext and ciphertext.
2) What does a key control?
3) How many Caesar keys exist?
4) Why is frequency analysis useful?');

-- A-Level lessons
INSERT INTO Alevel_Lessons (lessonName, lessonText) VALUES
(N'1. Symmetric vs Asymmetric',
N'[Definitions]
• Symmetric: same key encrypts and decrypts (e.g., AES).
• Asymmetric: public key encrypts, private key decrypts (e.g., RSA).

[Pros/Cons]
Symmetric is fast but needs secure key sharing.
Asymmetric simplifies key distribution but is slower.'),
(N'2. Formal Shift (Caesar) Cipher',
N'[Algorithm]
E_k(x) = (x + k) mod 26
D_k(y) = (y - k) mod 26

[Exercise]
Encrypt "ATTACKATDAWN" with k=5.'),
(N'3. Vernam (One-Time Pad) – Theory',
N'[Core Idea]
Bitwise XOR with a truly random key of equal length to the message. Key
must never be reused.

[Properties]
• Perfect secrecy if keys are random, secret, and single-use.
• In practice, generating/distributing true random keys is hard.'),
(N'4. Vernam – Worked Example',
N'[Walkthrough]
Plaintext: "A" → 01000001
Random key:  10111010
XOR result:  11111011 → Ciphertext byte

[UI Tie-in]
Tool shows ASCII, binary, and XOR step-by-step.'),
(N'5. Rail Fence (Transposition)',
N'[Idea]
Rearrange (permute) character order using rows (rails).
```

```
[Mini-Task]
Encrypt "CRYPTOGRAPHY" with 3 rails.'),
(N'6. Stream vs Block Ciphers',
N'[Compare]
Stream: encrypts symbol-by-symbol (like Vernam stream).
Block: encrypts fixed-size blocks (like AES).

[Why Care?]
Mode selection impacts security (see ECB vs CBC).'),
(N'7. Modes of Operation (ECB/CBC concept)',
N'[Warning]
ECB reveals patterns (identical blocks → identical ciphertext blocks).
CBC uses an IV to randomise first block and chains subsequent blocks.'),
(N'8. Key Management & Distribution',
N'[Problem]
How do parties share keys securely? Options: pre-shared, Diffie–Hellman,
hybrid (RSA + AES).'),
(N'9. Exam Practice – Short Answers',
N'[Questions]
1) Why does reusing a Vernam key break security?
2) Explain one weakness of ECB mode.
3) Outline one benefit of asymmetric encryption.');

-- Further/Post A-Level lessons
INSERT INTO Further_Lessons (lessonName, lessonText) VALUES
(N'1. Cryptographic Hash Functions',
N'[Properties]
Deterministic, preimage-resistant, second-preimage-resistant, collision-
resistant.

[Use Cases]
Password storage (with salt), integrity checks, digital signatures.'),
(N'2. SHA Family & Collisions',
N'[Discussion]
SHA-1 is deprecated due to practical collision attacks.
Modern systems use SHA-256/512 (or SHA-3).'),
(N'3. Modular Arithmetic Basics',
N'[Maths]
a ≡ b (mod n). Fast exponentiation is critical for RSA.'),
(N'4. RSA – Key Generation',
N'[Steps]
1) Choose primes p, q.
2) n = p·q ; φ(n) = (p−1)(q−1).
3) Choose e coprime with φ(n).
4) Compute d ≡ e^{-1} (mod φ(n).)'),
(N'5. RSA – Encrypt/Decrypt',
N'[Formulas]
Ciphertext C = M^e mod n
Plaintext  M = C^d mod n

[Security]
Depends on factoring difficulty.'),
(N'6. Diffie–Hellman Key Exchange',
N'[Idea]
Two parties agree a shared secret over an insecure channel using g^ab mod
p.'),
(N'7. Digital Signatures',
N'[Concept]
Sign with private key, verify with public key.
Provides authenticity and non-repudiation.'),
```

```
(N'8. PKI & Certificates',
N'[Overview]
Certificate Authorities bind identities to public keys.
X.509 certificates underpin TLS.'),
(N'9. Attacks & Threat Models',
N'[Examples]
Chosen-plaintext, chosen-ciphertext, timing/side-channel, man-in-the-
middle.');
```

Technical Solution

Cryptography Classes

CryptographicAlgorithm.cs

```
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Numerics;
using System.Web;
namespace CryptographyTeachingTool.Cryptography
{
    public class CryptographicAlgorithm
    {
        /// <summary>
        /// This is a parent class to all algorithms that I will have a demo on.
        /// The class will contain methods for all generic and frequently used
computations,
        /// for example XORs on binary arrays or converting denary to
hexadecimal.
        /// Therefore, all algorithms will inherit from it and be able to access
or override certain
        /// methods to improve code reusability and maintainability.
        /// </summary>

        // The following constants are values used in general cryptographic
algorithms. They are stored as constants
        // to provide a placeholder or identifier for the value and a central
place to store the value.
        protected int BITS_PER_HEX_DIGIT = 4;
        protected int BITS_PER_BYTE = 8;
        protected ulong MAX_NUMERIC_VAL = 10; // this is the threashold for
hexadecimal where numbers are no longer used (eg 10 = A)
        protected ulong NUMBER_TO_ALPHA_ASCII_SHIFT = 87; // eg 10 + 87 = the
letter A (as above)
        protected int LOWERCASE_LOWER_BOUND = 97;   // These demonstrate the
bounds for the ASCII
        protected int LOWERCASE_UPPER_BOUND = 122;  // values of uppercase and
lowercase symbols
        protected int UPPERCASE_LOWER_BOUND = 65;
        protected int UPPERCASE_UPPER_BOUND = 90;

        public CryptographicAlgorithm()
        {

        }

        public interface IHashingAlgorithm
        {
            // This interface creates a Hash method for hashing classes that
inherit from this,
            // like the SHA-256 class.
            string Hash(string input);
        }

        public virtual string Encrypt(int key, string plainText)
        {
            // This method will be overridden by child classes to run their own
version,
            // ultimately returning the text encrypted.
            return "";
        }
```

```
        public virtual string Decrypt(int key, string cipherText)
        {
            // This method calls the Encrypt method with a negative key, because
most decryption processes
            // are simply the same as encryption but with the key being used in a
reverse manner.
            // This method can be overridden.
            return Encrypt(-key, cipherText);
        }


        // All numbers are represented using the type ulong to avoid conversion
errors during bitwise
        // operations. This is memory inefficient however completely nessisary to
ensure the correct
        // result is achieved.

        protected ulong[] AND(ulong[] a, ulong[] b)
        {
            // This is a stock cryptography process and thus the method performs
an AND operation between two ulong arrays.
            // Both arrays will only ever contain 1s and 0s in each index, but is
of type ulong to prevent conversion errors
            // later in, for example, the hashing algorithm.
            ulong[] result = new ulong[a.Length];
            for (int i = 0; i < a.Length; i++)
            {

                result[i] = AND(a[i], b[i]);
            }
            return result;
        }

        protected ulong AND(ulong a, ulong b)
        {
            // This overload of AND takes two ulongs as opposed to a list of
them.
            // It simply performs an AND operation between the two.
            if ((a == 1) && (b == 1)) { return 1; }
            else { return 0; }
        }

        protected int[] OR(int[] a, int[] b)
        {
            // This is a stock cryptography process and thus the method performs
an OR operation between two ulong arrays.
            // Both arrays will only ever contain 1s and 0s in each index, but is
of type ulong to prevent conversion errors
            // later in, for example, the hashing algorithm.
            int[] result = new int[a.Length];
            for (int i = 0; i < a.Length; i++)
            {

                result[i] = OR(a[i], b[i]);
            }
            return result;
        }

        protected int OR(int a, int b)
        {
            // This overload of OR takes two ulongs as opposed to a list of them.
            // It simply performs an OR operation between the two.
            if ((a == 1) || (b == 1)) { return 1; }
```

```
            else { return 0; }
        }

        protected ulong[] XOR(ulong[] a, ulong[] b)
        {
            // This is a stock cryptography process and thus the method performs
an XOR operation between two ulong arrays.
            // Both arrays will only ever contain 1s and 0s in each index, but is
of type ulong to prevent conversion errors
            // later in, for example, the hashing algorithm.
            ulong[] result = new ulong[a.Length];
            for (int i = 0; i < a.Length; i++)
            {

                result[i] = XOR(a[i], b[i]);
            }
            return result;
        }

        protected ulong XOR(ulong a, ulong b)
        {
            // This overload of XOR takes two ulongs as opposed to a list of
them.
            // It simply performs an XOR operation between the two.
            if ((a == 1) && (b == 1)) { return 0; }
            else if ((a == 0) && (b == 0)) { return 0; }
            else { return 1; }
        }

        protected string XOR(string ascii, string key)
        {
            // This overload of XOR takes two strings where each character is
only a 1 or 0.
            // This method has been created specifically for cryptography
processes such as the Vernam Cipher,
            // where ascii values are XORed with a key.
            string result = "";
            for (int i = 0; i < ascii.Length; i++)
            {
                if (ascii[i] != key[i])
                {
                    result += "1";
                }
                else
                {
                    result += "0";
                }
            }
            return result;
        }

        protected ulong[] NOT(ulong[] a)
        {
            // This is a stock cryptography process and thus the method performs
a NOT operation on every bit in a ulong array.
            // The array will only ever contain 1s and 0s in each index, but is
of type ulong to prevent conversion errors
            // later in, for example, the hashing algorithm.
            ulong[] result = new ulong[a.Length];
            for (int i = 0; i < a.Length; i++)
            {
                result[i] = NOT(a[i]);
            }
```

```
            return result;
        }

        protected ulong NOT(ulong a)
        {
            // This overload of NOT takes asingle ulong as opposed to a list of
them.
            // It simply performs an OR operation on it.
            if (a == 1) { return 0; }
            else { return 1; }
        }

        protected ulong[] IntToBin(ulong x, int length)
        {
            // This method is used to convert a positive integer to a binary
value.
            // It takes a value to convert and then a number of bits to express
it with.
            // The method does not use a sign convention - only positive values
are represented.
            ulong[] bin = new ulong[length];

            for (int i = length - 1; i >= 0; i--)
            {
                bin[i] = x % 2;
                x = x / 2;
            }

            return bin;
        }

        protected ulong BinToInt(ulong[] bin)
        {
            // This method converts an binary array of any given length to a
single ulong (positive)
            // value. The value can only ever be positive.
            ulong x = 0;
            int length = bin.Length;

            for (int i = 0; i < length; i++)
            {
                x += bin[i] * Convert.ToUInt64(Math.Pow(2, length - 1 - i));
            }

            return x;
        }

        protected ulong[] HexToBin(string hex)
        {
            // This method is used to convert a string of any number of
hexadecimal digits to an unsigned binary array.
            ulong[] bits = new ulong[hex.Length * BITS_PER_HEX_DIGIT]; // one hex
val is 4 bits
            for (int i = 0; i < hex.Length; i++)
            {
                ulong val = Convert.ToUInt64(hex[i].ToString(), 16);
                ulong[] binVal = IntToBin(val, BITS_PER_HEX_DIGIT);

                for (int j = 0; j < BITS_PER_HEX_DIGIT; j++)
                {
                    bits[i * BITS_PER_HEX_DIGIT + j] = binVal[j];
                }
            }
```

```
            return bits;
        }

        protected string BinToHex(ulong[] bin)
        {
            // This method is used to convert an unsigned binary array of any
given length
            // into a string of hexadecimal digits which represent the value of
the binary array.
            string hex = "";
            for (int i = 0; i < bin.Length; i += BITS_PER_HEX_DIGIT) // asumes
its a mult of 4
            {
                ulong[] bits = new ulong[BITS_PER_HEX_DIGIT];
                for (int j = 0; j < BITS_PER_HEX_DIGIT; j++)
                {
                    bits[j] = bin[i + j];
                }

                ulong val = BinToInt(bits);

                if (val >= MAX_NUMERIC_VAL)
                {
                    ulong asciiVal = val + NUMBER_TO_ALPHA_ASCII_SHIFT;
                    hex += Convert.ToChar(asciiVal);
                }

                else { hex += val.ToString(); }
            }

            return hex;
        }

        protected char BinaryToChar(string binary)
        {
            // This method is used to convert an unsigned binary array of 8 chars
            // into a single character, based on its ASCII code.
            int result = 0;
            int columnValue = 1;
            for (int i = binary.Length - 1; i >= 0; i--)
            {
                result += (columnValue * Convert.ToInt16(binary[i]));
                columnValue = columnValue * 2;
            }
            return Convert.ToChar(result);
        }

    }
}
```

CaesarCipher.cs

```
using CryptographyTeachingTool;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CryptographyTeachingTool.Cryptography.Ciphers
{
    public class CaesarCipher : CryptographicAlgorithm
    {
```

```
        /// <summary>
        /// I will be using polymorphism to override the CryptographicAlgorithm
classes encrypt
        /// and decrypt methods with this shift cipher. The class will be used in
the demos to
        /// for encrypting and decrypting strings when given a key.
        /// </summary>

        // Constants
        private int ALPHABET_LENGTH = 26;
        public override string Encrypt(int key, string plainText)
        {
            // This static method is used to process and string of text and key.
            // The original Encrypt method is overridden.
            // It performs a Caesar cipher on the string, shifting each char the
given
            // number of places in the alphabet. The result is returned as a
string.
            // Note: any symbols (such as punctuation) are not shifted but
instead left in their place.
            // This is because the symbols do not have a native alphabet and
would still be convient
            // to be represented by this cipher.
            key = ((key % ALPHABET_LENGTH) + ALPHABET_LENGTH) % ALPHABET_LENGTH;
// a massive key is an issue (when going from a-z) therefore can be simplified
            bool isNegativeKey = false; // this var ensures that the code works
for negative keys
            if (key < 0)
            {
                isNegativeKey = true;
            }
            int length = plainText.Length;
            char[] plainTextArray = plainText.ToArray(); // this is an array
where each item is a char in the plain text
            int[] asciiArray = new int[length]; // this array will later contain
the ascii values of certain characters
            int[] nonAlpha = new int[length]; // this array will contain all
nonalpha characters which shouldn't be shifted
            string cipherText = "";

            for (int i = 0; i < length; i++) // creates the shifted ascii array
and the non alpha array
            {
                int currentASCII = Convert.ToInt16(plainTextArray[i]);
                if ((currentASCII >= LOWERCASE_LOWER_BOUND && currentASCII <=
LOWERCASE_UPPER_BOUND) || (currentASCII >= UPPERCASE_LOWER_BOUND && currentASCII
<= UPPERCASE_UPPER_BOUND))
                {
                    int shiftedVal = currentASCII + key; // this stores the
shifted values
                    if (currentASCII >= UPPERCASE_LOWER_BOUND && currentASCII <=
UPPERCASE_UPPER_BOUND)
                    {
                        if (shiftedVal > UPPERCASE_UPPER_BOUND)
                            shiftedVal = shiftedVal - ALPHABET_LENGTH;
                    }
                    else if (currentASCII >= LOWERCASE_LOWER_BOUND &&
currentASCII <= LOWERCASE_UPPER_BOUND)
                    {
                        if (shiftedVal > LOWERCASE_UPPER_BOUND)
                            shiftedVal = shiftedVal - ALPHABET_LENGTH;
                    }
```

```
                    asciiArray[i] = shiftedVal;
                }
                else
                {
                    nonAlpha[i] = currentASCII;
                    asciiArray[i] = -1;
                }
            }

            for (int i = 0; i < length; i++) // places the two lists back
together, converts back to a char
                                             // and then concatenates to a var
called cipher text
            {
                if (asciiArray[i] == -1)
                {
                    asciiArray[i] = nonAlpha[i];
                }
                cipherText = cipherText + Convert.ToChar(asciiArray[i]);
            }

            return cipherText;
        }
    }

}
```

VernamCipher.cs

```
using CryptographyTeachingTool;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CryptographyTeachingTool.Cryptography.Ciphers
{
    public class VernamCipher : CryptographicAlgorithm
    {
        /// <summary>
        /// This class inherits from the CryptographicAlgorithm class and will
use methods
        /// from it relating to XORing strings, converting denary into binary or
denary to
        /// hexadecimal. All binary results will be displayed as hexadecimal.
        /// </summary>

        public int[] GenerateKey(string plainText)
        {
            // This static method is used to generate a random ASCII key of the
correct
            // length. It is used when the user does not enter a key.
            Random random = new Random();
            int length = plainText.Length;
            int[] key = new int[length];
            for (int i = 0; i < plainText.Length; i++)
            {
                int k = random.Next(LOWERCASE_LOWER_BOUND,
LOWERCASE_UPPER_BOUND);
                key[i] = k;
            }
            return key;
```

```
        }

        public string Encrypt(int[] key, string plainText)
        {
            // This static method overrides the original Encrypt method in
CryptographicAlgorithm.
            // It is used to perfrom a Vernam cipher on a string. It returns the
result in hexadecimal.
            string ConvertToBinary(int denary)
            {
                // This method is used to convert a denary number to binary and
                // return the result as a string, unlike other conversion
methods.
                if (denary == 0)
                {
                    return "00000000";
                }

                string result;
                if (denary == 1) // this ensures that if the denary value 0 is
being converted it remains as "0" not ""
                {
                    return "1";
                }
                else
                {
                    result = ConvertToBinary(denary / 2) +
Convert.ToString(denary % 2);
                }
                return result.PadLeft(BITS_PER_BYTE, '0'); // ensures the length
is 8 bits
            }

            if (key.Length != plainText.Length)
            {
                return "Key was not the same length as cipher text";
            }

            char[] plainTextArray = plainText.ToArray();
            List<ulong> binResult = new List<ulong>();
            for (int i = 0; i < plainTextArray.Length; i++)
            {
                // This loop goes through each character in the plain text
converts it to ASCII,
                // performs an XOR operation on the two and concatenates the
result onto the overall
                // plainTextArray for returning

                int PlainTextASCIIValue = Convert.ToInt16(plainTextArray[i]);
                string binaryASCIIValue = ConvertToBinary(PlainTextASCIIValue);
                string k = ConvertToBinary(key[i]);
                string result = XOR(binaryASCIIValue, k);

                foreach (char bit in result) // this concatenates the values onto
the binary result list
                {
                    if (bit == '1')
                    {
                        binResult.Add(1);
                    }
                    else
                    {
                        binResult.Add(0);
```

```
                    }
                }
            }
            while (binResult.Count % BITS_PER_HEX_DIGIT != 0)
            {
                binResult.Add(0); // pads the array with zeros to ensure the
length is a multiple of 4
            }
            ulong[] bitArray = binResult.ToArray();
            return BinToHex(bitArray); // the bit array is converted to
hexadecimal and returned
        }

        public string Decrypt(int[] key, string cipherText)
        {
            // This static method overrides the original Decrypt method in
CryptographicAlgorithm
            // A negative key cannot be created or used to decrypt the cipher
text. Instead a
            // decryption can be done by performing the exact encryption process
with the cipher
            // text and the same key.

            // Firstly the hexdecimal must be converted to a string
            ulong[] hexBits = HexToBin(cipherText);
            string cipherTextString = "";

            for (int i = 0; i < hexBits.Length; i += BITS_PER_BYTE) // gets every
8 bits
            {
                string stingOfBitsInByte = "";
                for (int j = 0; j < BITS_PER_BYTE; j++) // adds the 8 bits to the
sting
                {
                    stingOfBitsInByte += hexBits[i + j];
                }
                cipherTextString += BinaryToChar(stingOfBitsInByte);
            }

            return Encrypt(key, cipherTextString); // The encryption process is
now just the same process
        }
    }
}
```

RailFenceCipher.cs

```
using CryptographyTeachingTool;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CryptographyTeachingTool.Cryptography.Ciphers
{
    public class RailFenceCipher : CryptographicAlgorithm
    {
        /// <summary>
        /// The CryptographicAlgorithm will be a parent to this class.
        /// Railfence is a transposition cipher that will need its own methods
        /// therefore extra will be added into this class.
```

```
        /// </summary>

        public override string Encrypt(int key, string plainText)
        {
            // This static method overrides the original Encrypt method in
CryptographicAlgorithm.
            // It is used to perform a railfence cipher on a string using a given
key.
            // The result is returned as a single string.

            char[,] matrix = new char[key, plainText.Length]; // the matrix is
created where the number of columns
                                                              // equals the
length of the plain text and the number
                                                              // of rows equals
the key

            int row = 0; // this is needed to keep track of the current row for
each stage
            bool isDecreasing = true;
            for (int column = 0; column < plainText.Length; column++) // We
interate through the columns (thus the plain text)
            {
                matrix[row, column] = plainText[column]; // the letter is placed
in the appropriate cell

                if (row == key - 1) // if on the bottom row then begin going back
up
                {
                    isDecreasing = false;
                }
                else if (row == 0) // if on the top row then begin going back
down
                {
                    isDecreasing = true;
                }

                // Alter the row based on the current direction
                if (isDecreasing)
                {
                    row++;
                }
                else
                {
                    row--;
                }
            }

            // Read the text from left to right and then top to bottom
            string cipherText = "";
            for (row = 0; row < key; row++)
            {
                for (int column = 0; column < plainText.Length; column++)
                {
                    if (matrix[row, column] != '\0')
                    {
                        cipherText += matrix[row, column];
                    }
                }
            }
            return cipherText;
        }
```

```
        public override string Decrypt(int key, string cipherText)
        {
            // This static method overrides the original Decrypt method in
CryptographicAlgorithm.
            // It then decrypts a cipher text given the key and cipher text as a
string.
            // The encryption process is different to the Encrypt process which
is why a negative key
            // cannot be used. The result is returned as a single string.

            char[,] matrix = new char[key, cipherText.Length];

            int row = 0;
            bool isDecreasing = true;
            for (int column = 0; column < cipherText.Length; column++)
            {
                // Begin by creating a matrix with the symbol '*' in place of
where digits should be
                matrix[row, column] = '*';

                if (row == key - 1)
                {
                    isDecreasing = false;
                }
                else if (row == 0)
                {
                    isDecreasing = true;
                }

                if (isDecreasing)
                {
                    row++;
                }
                else
                {
                    row--;
                }
            }

            // Then replace each '*' with the appropriate letter
            int count = 0;
            for (row = 0; row < key; row++)
            {
                for (int column = 0; column < cipherText.Length; column++)
                {
                    if (matrix[row, column] == '*')
                    {
                        matrix[row, column] = cipherText[count];
                        count++;
                    }
                }
            }

            // Then read in the same zig zag pattern to get the plain text out
            row = 0;
            isDecreasing = true;
            string plainText = "";
            for (int column = 0; column < cipherText.Length; column++)
            {
                plainText += matrix[row, column];

                if (row == key - 1)
                {
```

```
                    isDecreasing = false;
                }
                else if (row == 0)
                {
                    isDecreasing = true;
                }

                if (isDecreasing)
                {
                    row++;
                }
                else
                {
                    row--;
                }
            }

            return plainText;
        }
    }

}
```

SHAHash.cs

```
using CryptographyTeachingTool;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using static CryptographyTeachingTool.Cryptography.CryptographicAlgorithm;

namespace CryptographyTeachingTool.Cryptography.Hash
{
    public class SHA256Hash : CryptographicAlgorithm, IHashingAlgorithm
    {
        /// <summary>
        /// This algorithm is highly compicated and therefore I will be
decomposing
        /// the overall algorithms into many smaller tasks. I will use methods
from
        /// the CryptographicAlgorithm class (its parent) to improve the ease of
        /// writing and debugging this algorithm.
        /// </summary>

        // Constants
        private int N_BITS = 32; // This defines the word length of the algorithm
        // (the number of bits that each process occurs on)

        // The following array of strings contains the round constants used,
which
        // are the fractional parts of the cube roots of the first 64 prime
numbers.
        // Each value is stored in hexadecimal.
        private static readonly string[] K_CONSTANTS =
        {
            // H0-H7
            "0x428a2f98", "0x71374491", "0xb5c0fbcf", "0xe9b5dba5",
            "0x3956c25b", "0x59f111f1", "0x923f82a4", "0xab1c5ed5",

            // Later vals
            "0xd807aa98", "0x12835b01", "0x243185be", "0x550c7dc3",
```

```
            "0x72be5d74", "0x80deb1fe", "0x9bdc06a7", "0xc19bf174",
            "0xe49b69c1", "0xefbe4786", "0x0fc19dc6", "0x240ca1cc",
            "0x2de92c6f", "0x4a7484aa", "0x5cb0a9dc", "0x76f988da",
            "0x983e5152", "0xa831c66d", "0xb00327c8", "0xbf597fc7",
            "0xc6e00bf3", "0xd5a79147", "0x06ca6351", "0x14292967",
            "0x27b70a85", "0x2e1b2138", "0x4d2c6dfc", "0x53380d13",
            "0x650a7354", "0x766a0abb", "0x81c2c92e", "0x92722c85",
            "0xa2bfe8a1", "0xa81a664b", "0xc24b8b70", "0xc76c51a3",
            "0xd192e819", "0xd6990624", "0xf40e3585", "0x106aa070",
            "0x19a4c116", "0x1e376c08", "0x2748774c", "0x34b0bcb5",
            "0x391c0cb3", "0x4ed8aa4a", "0x5b9cca4f", "0x682e6ff3",
            "0x748f82ee", "0x78a5636f", "0x84c87814", "0x8cc70208",
            "0x90befffa", "0xa4506ceb", "0xbef9a3f7", "0xc67178f2"
        };

        // The following array of strings contains the initial hash values, which
        // are the fractional parts of the square roots of the first prime
numbers.
        // Each value is stored in hexadecimal.
        private static readonly string[] INITIAL_HASH_VALUES =  // these are H0-7
        {
            "0x6a09e667", "0xbb67ae85", "0x3c6ef372", "0xa54ff53a",
            "0x510e527f", "0x9b05688c", "0x1f83d9ab", "0x5be0cd19"
        };

        // The following constants define various helper constants. They are
stored as constants
        // to provide a placeholder or identifier for the value and a central
place to store the value.
        private int N_H_VALUES = 8;
        private int N_K_VALUES = 64;
        private int BLOCK_SIZE = 512;
        private int PADDING = 448;
        private int LENGTH_BITS = 64;
        private int HEX_BASE = 16;
        private int TOTAL_ROUNDS = 16;

        // Hash methods
        private ulong[] ROTR(ulong[] a, ulong n)
        {
            // This method is used to rotate a string of bits to the right by a
given
            // number of places.
            ulong length = Convert.ToUInt64(a.Length);
            ulong[] result = new ulong[length];

            // shift all bits to the right by n filling in the new spaces with
the bits removed
            for (ulong i = 0; i < length; i++)
            {
                result[i] = a[(i + length - n) % length];
            }

            return result;
        }

        private ulong[] SHR(ulong[] a, ulong n)
        {
            // This method is used to shift all of the bits in a binary array to
the
            // right by a given number of places the new space is filled with
zeros.
            ulong length = Convert.ToUInt64(a.Length);
```

```
            ulong[] result = new ulong[length]; // this already allocated the
array with zeros

            // shift all bits to the right by n filling in the new spaces with 0
            for (ulong i = 0; i < length - n; i++)
            {
                result[i + n] = a[i];
            }

            return result;
        }

        private ulong MAJ(ulong x, ulong y, ulong z)
        {
            // This method is used to perform a majority operation on three
ulongs.
            // The operation can be broken down to:
            // (X AND Y) XOR (X AND Z) XOR (Y AND Z)
            ulong[] xBin = IntToBin(x, N_BITS);
            ulong[] yBin = IntToBin(y, N_BITS);
            ulong[] zBin = IntToBin(z, N_BITS);
            ulong[] a = AND(xBin, yBin);
            ulong[] b = AND(xBin, zBin);
            ulong[] c = AND(yBin, zBin);
            ulong[] result = XOR(XOR(a, b), c);
            return BinToInt(result);
        }

        private ulong CH(ulong x, ulong y, ulong z)
        {
            // This method is used to perfrom a choose operation on three ulongs.
            // The operation can be broken down to:
            // (X AND Y) XOR (NOT X AND Z)
            ulong[] xBin = IntToBin(x, N_BITS);
            ulong[] yBin = IntToBin(y, N_BITS);
            ulong[] zBin = IntToBin(z, N_BITS);
            ulong[] a = AND(xBin, yBin);
            ulong[] b = AND(NOT(xBin), zBin);
            ulong[] result = XOR(a, b);
            return BinToInt(result);
        }

        private ulong SigmaZero(ulong x)
        {
            // This method performs the lowercase sigma zero function unique to
the SHA-256 hash
            // The operation can be broken down to:
            // ROTR(X, 7) XOR ROTR(X, 18) XOR SHR(X, 3)
            ulong[] xBin = IntToBin(x, N_BITS);
            ulong[] a = ROTR(xBin, 7);
            ulong[] b = ROTR(xBin, 18);
            ulong[] c = SHR(xBin, 3);
            ulong[] result = XOR(XOR(a, b), c);
            return BinToInt(result);
        }

        private ulong SigmaOne(ulong x)
        {
            // This method performs the lowercase sigma one function unique to
the SHA-256 hash.
            // The operation can be broken down to:
            // ROTR(X, 17) XOR ROTR(X, 19) XOR SHR(X, 10)
            ulong[] xBin = IntToBin(x, N_BITS);
```

```
            ulong[] a = ROTR(xBin, 17);
            ulong[] b = ROTR(xBin, 19);
            ulong[] c = SHR(xBin, 10);
            ulong[] result = XOR(XOR(a, b), c);
            return BinToInt(result);
        }

        private ulong CapitalSigmaZero(ulong x)
        {
            // This method performs the uppercase sigma one function unique to
the SHA-256 hash.
            // The operation can be broken down to:
            // ROTR(X, 2) XOR ROTR(X, 13) XOR SHR(X, 22)
            ulong[] xBin = IntToBin(x, N_BITS);
            ulong[] a = ROTR(xBin, 2);
            ulong[] b = ROTR(xBin, 13);
            ulong[] c = SHR(xBin, 22);
            ulong[] result = XOR(XOR(a, b), c);
            return BinToInt(result);
        }

        private ulong CapitalSigmaOne(ulong x)
        {
            // This method performs the uppercase sigma one function unique to
the SHA-256 hash
            // The operation can be broken down to:
            // ROTR(X, 6) XOR ROTR(X, 11) XOR SHR(X, 25)
            ulong[] xBin = IntToBin(x, N_BITS);
            ulong[] a = ROTR(xBin, 6);
            ulong[] b = ROTR(xBin, 11);
            ulong[] c = SHR(xBin, 25);
            ulong[] result = XOR(XOR(a, b), c);
            return BinToInt(result);
        }

        // hashing
        public string Hash(string input)
        {
            // This static method overrides the interface and Hash method of the
CryptographicAlgorithm class.
            // It is used to perform the overall hashing process for SHA-256. The
result is returned
            // as a hexadecimal value.

            // initalise the hash vals H0-7
            ulong[] H = new ulong[N_H_VALUES];
            for (int i = 0; i < N_H_VALUES; i++)
            {
                H[i] = Convert.ToUInt64(INITIAL_HASH_VALUES[i], HEX_BASE); //
uses built in function rather than converting to binary then denary
            }
            // initalise the k constants
            ulong[] K = new ulong[N_K_VALUES];
            for (int i = 0; i < N_K_VALUES; i++)
            {
                K[i] = Convert.ToUInt64(K_CONSTANTS[i], HEX_BASE);
            }

            // convert the input to one ulong binary string
            string binInput = "";
            for (int i = 0; i < input.Length; i++)
            {
```

```
                ulong[] binArray = IntToBin(input[i], BITS_PER_BYTE); // 8 bits
per char
                foreach (int digit in binArray)
                {
                    binInput += digit;
                }
            }

            // alter length
            int l = binInput.Length;
            int k = (PADDING - ((l + 1) % BLOCK_SIZE) + BLOCK_SIZE) % BLOCK_SIZE;
            binInput += '1';
            for (int i = 0; i < k; i++) // adds padding
            {
                binInput += '0';
            }
            ulong[] lengthInBits = IntToBin(Convert.ToUInt64(l), LENGTH_BITS); //
stores the length in the final 64 bits
            foreach (int digit in lengthInBits)
            {
                binInput += digit;
            }

            // split into blocks and words
            int N = binInput.Length / BLOCK_SIZE;
            string[] blocks = new string[N];
            for (int i = 0; i < N; i++)
            {
                blocks[i] = binInput.Substring(i * BLOCK_SIZE, BLOCK_SIZE); //
creates the nessisary number of blocks
            }
            ulong[][][] words = new ulong[N][][]; // N blocks, 16 words, 32 bits

            for (int i = 0; i < N; i++) // fills the words array
            {
                words[i] = new ulong[TOTAL_ROUNDS][];
                for (int j = 0; j < TOTAL_ROUNDS; j++)
                {
                    string wordString = blocks[i].Substring(j * N_BITS, N_BITS);
                    words[i][j] = new ulong[N_BITS];

                    for (int bit = 0; bit < N_BITS; bit++)
                    {
                        words[i][j][bit] =
Convert.ToUInt64(char.GetNumericValue(wordString[bit])); // convert characters to
integers
                    }
                }
            }

            // loop through each block
            for (int i = 0; i < N; i++)
            {
                ulong[] W = new ulong[TOTAL_ROUNDS];
                for (int t = 0; t < TOTAL_ROUNDS; t++) // initalise vals for W
                {
                    if (t >= 0 && t <= 15) // first 15
                    {
                        W[t] = Convert.ToUInt64(BinToInt(words[i][t]));
                    }
                    else if (t >= 16 && t <= 63) // above 15
                    {
```

```
                        W[t] = Convert.ToUInt64(SigmaOne(W[t - 2]) + W[t - 7] +
SigmaZero(W[t - 15]) + W[t - 16]);
                    }
                }

                // initalise a-h
                ulong a = H[0];
                ulong b = H[1];
                ulong c = H[2];
                ulong d = H[3];
                ulong e = H[4];
                ulong f = H[5];
                ulong g = H[6];
                ulong h = H[7];

                // calculate a to h
                for (int t = 0; t < TOTAL_ROUNDS; t++)
                {
                    ulong TOne = Convert.ToUInt64(h + CapitalSigmaOne(e) + CH(e,
f, g)) + K[t] + W[t];
                    ulong TTwo = Convert.ToUInt64(CapitalSigmaZero(a) + MAJ(a, b,
c));

                    h = g;
                    g = f;
                    f = e;
                    e = d + TOne;
                    d = c;
                    c = b;
                    b = a;
                    a = TOne + TTwo;
                }

                // add corrosponding a-h onto H0-7
                H[0] += a;
                H[1] += b;
                H[2] += c;
                H[3] += d;
                H[4] += e;
                H[5] += f;
                H[6] += g;
                H[7] += h;
            }

            string result = "";
            for (int i = 0; i < BITS_PER_BYTE; i++)
            {
                result += H[i].ToString("x8"); // using built in prevents needing
to do binToHex(intToBin(result, nBits));
            }

            return result.Substring(0, 64);
        }
    }

}
```

Main Classes

User.cs

```
using CryptographyTeachingTool.Data;
using CryptographyTeachingTool.Data.Repositories;
using Microsoft.Data.SqlClient;
```

```
using System;
using System.Collections.Generic;
using System.IO.Compression;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CryptographyTeachingTool.MainClasses
{
    public class User
    {
        /// <summary>
        ///  This generic class will hold all essential information need by the
algorithm.
        ///  When fetching data from the database things like the connection
string and the usersID are needed,
        ///  which is why these are stored in the class.
        /// </summary>

        // The attributes below correspond to each column in the Users table of
the database and
        // are part of this object, so they can be sent around the application
together (in an encapsulated manner).
        public int userID;
        public string username;
        public string firstName;
        public string role;
        private int loginID;

        public User(int uID, string Uname, string fName, string role, int login)
        {
            // This is the only constructor for the user class.
            this.userID = uID;
            this.username = Uname;
            this.firstName = fName;
            this.role = role;
            this.loginID = login;
        }

        public void SignOut()
        {
            // This method is used to provide a centralised place for uploading a
Log-out time to the AuditLog database
            UserRepo.SignOut(userID, loginID);
        }
    }
}
```

Lesson.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CryptographyTeachingTool.MainClasses
{
    public class Lesson
    {
        /// <summary>
```

```
        /// This class will be used to hold details of currently retrieved
lessons -
        /// this is especially useful as a list of lesson objects can be returned
from various
        /// methods in the Lesson Repo class.
        /// </summary>

        // The attributes below correspond to each column in the DefinedLessons
table of
        // the database and are part of this object, so they can be sent around
        // the application together (in an encapsulated manner).
        public int lessonID;
        public string text;
        public string title;
        public string workPrompt;
        public byte[] imageData;
        public int lessonNo;

        public Lesson(int lessonID, string text, string title, string workPrompt)
        {
            // This constructor is used when adding a lesson to the lesson viewer
when the lesson does not have an image.
            this.lessonID = lessonID;
            this.text = text;
            this.title = title;
            this.workPrompt = workPrompt;
        }
        public Lesson(int lessonID, string text, string title, string workPrompt,
byte[] imageData)
        {
            // This constructor is used when adding a lesson to the lesson viewer
when the lesson does have an image.
            this.lessonID = lessonID;
            this.text = text;
            this.title = title;
            this.workPrompt = workPrompt;
            this.imageData = imageData;
        }
        public Lesson(int lessonID, int lessonNo, string title)
        {
            // This constructor is used to create a lesson instance for each
lesson to be displayed in the lesson selector page.
            this.lessonID = lessonID;
            this.lessonNo = lessonNo;
            this.title = title;
        }
    }
}
```

Assignment.cs

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CryptographyTeachingTool.MainClasses
{
    public class Assignment
    {
        /// <summary>
```

```
        /// This class will be used similarly to the above Lesson class -
        /// it will be used to hold details of assignments retrieved from the
Assignment Repo class.
        /// This will also prevent numerous visits to the database.
        /// </summary>

        // The attributes below are collected from each column in the
DefinedLessons,
        // SyllabusLessons and Users table of the database and are part of this
object, so
        // they can be sent around the application together (in an encapsulated
manner).
        // This is particularly useful when assignment details or results need to
be displayed.
        public string title;
        public int assignmentID;
        public DateTime dateSet;
        public int totalMarks;
        public int actualMarks;
        public string fullname;
        public string firstname;
        public string surname;
        public byte[] binFile; // this stores the binary data of the file sent by
the teacher
        public string fileName;

        public Assignment(string title, DateTime dateSet, int totalMarks)
        {
            // This constructor is used create an Assignment instance that stores
the details of each assignment set by the current teacher for displaying on the
teacher dashboard.
            this.title = title;
            this.dateSet = dateSet;
            this.totalMarks = totalMarks;
        }
        public Assignment(string title, DateTime dateSet, int totalMarks, int
actualMarks, string fullname)
        {
            // This constructor is used to create an Assignment instance that
stores the details of their
            // student's recent results for displaying on the teacher dashboard.
            this.title = title;
            this.dateSet = dateSet;
            this.totalMarks = totalMarks;
            this.actualMarks = actualMarks;
            this.fullname = fullname;
        }
        public Assignment(string title, DateTime dateSet, int totalMarks, byte[]
binFile, string fileName)
        {
            // This constructor is used to create an Assignment instance that
stores the details of an
            // assignment that is to be set by a teacher.
            this.title = title;
            this.dateSet = dateSet;
            this.totalMarks = totalMarks;
            this.binFile = binFile;
            this.fileName = fileName;
        }
        public Assignment(string title, DateTime dateSet, int totalMarks, string
firstname, string surname, int assignmentID)
        {
```

```
            // This constructor is used to create an Assignment instance that
stores the details of an
            // assignment set by a teacher for viewing on a student's assignment
viewer.
            this.title = title;
            this.dateSet = dateSet;
            this.totalMarks = totalMarks;
            this.firstname = firstname;
            this.surname = surname;
            this.assignmentID = assignmentID;
        }
    }
}
```

Data

DataClasses.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CryptographyTeachingTool.Data
{
    public class DataClasses
    {
        /// <summary>
        /// This class will be used to hold the connection string for all classes
that need to access the database.
        /// As I am designing my application so that only classes that inherit
from this will make the queries
        /// to the database, which will keep the database part of my application
separate from other
        /// areas (like the user interface). The repository classes will inherit
from this class,
        /// which will ensure that if the connection string ever needs to be
changed, it can be done
        /// through this class in one place, thus improving data integrity.
        /// </summary>

        // This string is used for creating a link to the database. It shows
Microsoft.Data.SqlClient where the database is stored, in this case locally.
        static protected string CONNECTION_STRING = "Server=DESKTOP-
TIHP928;Database=NEA;Trusted_Connection=True;Encrypt=False;";
    }
}
```

UserRepo.cs

```
using CryptographyTeachingTool.MainClasses;
using Microsoft.Data.SqlClient;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using static System.Windows.Forms.VisualStyles.VisualStyleElement.StartPanel;

namespace CryptographyTeachingTool.Data.Repositories
{
    class UserRepo : DataClasses
    {
        /// <summary>
        /// Instead of running queries on forms pages I will modularise the
process
        /// of querying the users table (on my database) through this class.
        /// This will result in better encapsulation, code reuse and
maintainability.
        /// </summary>

        static public User GetUser(string username, string hashedInput)
        {
            // This static method is used to return get a user's details from the
database and
            // then return an instance of the User object with their details.

            // Note: when null is returned the result will be caught at a lower
layer and processed accordingly.
            try
            {
                using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
                {
                    connection.Open();
                    string query = "SELECT HashedPass, UserID, Username,
Firstname, Role FROM Users WHERE Username = @username";
                    using (SqlCommand command = new SqlCommand(query,
connection))
                    {
                        command.Parameters.AddWithValue("@username", username);
                        using (SqlDataReader reader = command.ExecuteReader())
                        {
                            if (reader.Read())
                            {
                                string hashedPass = reader.GetString(0);

                                if (hashedInput == hashedPass)
                                {
                                    int uID = reader.GetInt32(1);
                                    string uName = reader.GetString(2);
                                    string fName = reader.GetString(3);
                                    string role = reader.GetString(4);
                                    int loginTime = SaveLoginTime(uID);
                                    User currentUser = new User(uID, uName,
fName, role, loginTime);
                                    return currentUser;
                                }
                                else
                                {
                                    MessageBox.Show("Incorrect Username or
Password.");
                                    return null;
```

```
                                }
                            // Note: when null is returned the result will be
caught at a lower layer and processed accordingly.
                            }
                            else
                            {
                                MessageBox.Show("There is no account with this
username please sign up first.");
                                return null;
                            }

                        }
                    }
                }
            }
            catch (SqlException ex)
            {
                // Note: when null is returned the result will be caught at a
lower layer and processed accordingly.
                if (ex.Number == 40 || ex.Number == 53 || ex.Number == -1 ||
ex.Number == -2 || ex.Number == 2)
                {
                    MessageBox.Show($"Our database could not be reached - please
check your internet and try again.");
                    return null;
                }
                else
                {
                    MessageBox.Show($"A database related error has occured:
\n{ex.Message}");
                    return null;
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
                return null;
            }
        }

        static public void SignOut(int userID, int loginID)
        {
            // This method is called whenever a forms page that inherits from the
BaseFormsPage closes.
            // It adds a log-out time to the row associated with the user's
current log-in.

            try
            {
                using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
                {
                    connection.Open();
                    string query = "UPDATE AuditLog SET LogoutTime = @logoutTime
WHERE LogoutTime is NULL and LoginID = @loginID;";
                    using (SqlCommand command = new SqlCommand(query,
connection))
                    {
                        command.Parameters.AddWithValue("@uID", userID);
                        command.Parameters.AddWithValue("@loginID", loginID);
                        command.Parameters.AddWithValue("@logoutTime",
DateTime.Now);
                        command.ExecuteNonQuery();
```

```
                    }
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
            }
        }

        static public bool UserExists(string username)
        {
            // This static method is used to check for a username in the Users
table, if it exists it returns True and otherwise False
            try
            {
                using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
                {
                    connection.Open();
                    string query = "SELECT Username FROM Users WHERE Username =
@uname;";
                    using (SqlCommand command = new SqlCommand(query,
connection))
                    {
                        command.Parameters.AddWithValue("@uname", username);
                        using (SqlDataReader reader = command.ExecuteReader())
                        {
                            if (reader.Read())
                            { return true; }
                            else
                            { return false; }
                        }
                    }
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
                return false;
            }
        }

        static public bool CreateUser(string username, string hashedPassword,
string firstname, string secondname, string role)
        {
            // This method is used to create an entry to the Users table with the
required information passed in as arguments.
            // It returns True if the result was a success
            try
            {
                using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
                {
                    connection.Open();
                    string query = "INSERT INTO Users (Username, HashedPass,
Firstname, Surname, Role)" +
                        "VALUES (@uname, @pword, @fname, @sname, @role);";
                    using (SqlCommand command = new SqlCommand(query,
connection))
                    {
                        try
                        {
                            command.Parameters.AddWithValue("@uname", username);
```

```
                            command.Parameters.AddWithValue("@pword",
hashedPassword);
                            command.Parameters.AddWithValue("@fname", firstname);
                            command.Parameters.AddWithValue("@sname",
secondname);
                            command.Parameters.AddWithValue("@role", role);
                            command.ExecuteNonQuery();
                            MessageBox.Show("Thank you for signing up, please now
log in to continue");
                            return true;
                        }
                        catch (Exception ex)
                        {
                            MessageBox.Show($"There has been an error, please try
again soon:\n{ex.Message}");
                            return false;
                        }
                    }

                }
            }
            catch (SqlException ex)
            {
                if (ex.Number == 40)
                {
                    MessageBox.Show($"Our database could not be reached - please
check your internet a try again.");
                    return false;
                }
                else
                {
                    MessageBox.Show($"An unexpected error occured:
\n{ex.Message}");
                    return false;
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
                return false;
            }
        }

        static private int SaveLoginTime(int userID)
        {
            // This method is used to create an inital entrance to the LoginAudit
table, which saves the time a specific user logs-in.
            // The SignOut method in the User object is used to add the log-out
time to this entry.
            DateTime dateTime = DateTime.Now;
            int loginID = 0;
            try
            {
                using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
                {
                    connection.Open();
                    string query = "INSERT INTO AuditLog(UserID, LoginTime)
Values(@uID, @time);" +
                    "SELECT CAST(SCOPE_IDENTITY() as int);"; // this is a simple
way of getting the assignmentID instead of requerying
                    using (SqlCommand command = new SqlCommand(query,
connection))
```

```
                    {
                        command.Parameters.AddWithValue("@uID", userID);
                        command.Parameters.AddWithValue("@time", dateTime);
                        loginID = Convert.ToInt32(command.ExecuteScalar()); //
the execute scalar acts as the execute non query in this case
                    }
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
            }
            return loginID;
        }

        static public int GetCurrentUserTotal(User currentUser)
        {
            // This static method returns the total time spent by a single user
on the application.
            // It does this through the audit log, with the user's ID being used
to filter the table down.
            try
            {
                int total = 0;
                using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
                {
                    connection.Open();
                    string query = "SELECT COALESCE(SUM(DATEDIFF(MINUTE,
LoginTime, LogoutTime)), 0) " + // COALESCE means if there are no entries 0 is
used
                        "FROM AuditLog WHERE UserID = @uID AND LogoutTime IS NOT
NULL;";
                    using (SqlCommand command = new SqlCommand(query,
connection))
                    {
                        command.Parameters.AddWithValue("@uID",
currentUser.userID);
                        using (SqlDataReader reader = command.ExecuteReader())
                        {
                            if (reader.Read())
                            {
                                total = reader.GetInt32(0);
                            }
                        }
                    }
                }
                return total;
            }
            catch (Exception ex)
            {
                MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
                return 0;
            }
        }
        static public int GetAvgUserTotal()
        {
            // This static method is used to return the average total time spent
by each user on the application.
            // It works this out using the audit table and the number of users on
the application.
            try
            {
```

```
                int total = 0;
                int count = 0;
                using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
                {
                    connection.Open();
                    string query = "SELECT COALESCE(SUM(DATEDIFF(MINUTE,
LoginTime, LogoutTime)), 0), COALESCE(COUNT(DISTINCT UserID), 0) " + // distinct
ensures we don't count only
                        "FROM AuditLog WHERE LogoutTime IS NOT NULL;";
// the number of entries (instead users)
                    using (SqlCommand command = new SqlCommand(query,
connection))
                    {
                        using (SqlDataReader reader = command.ExecuteReader())
                        {
                            if (reader.Read())
                            {
                                total = reader.GetInt32(0);
                                count = reader.GetInt32(1);
                            }
                        }
                    }
                }
                if (total > 0 && count > 0) { return total / count; }
                else { return 0; }
            }

            catch (Exception ex)
            {
                MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
                return 0;
            }
        }
    }
}
```

LessonRepo.cs

```
using CryptographyTeachingTool.MainClasses;
using Microsoft.Data.SqlClient;
using System;
using System.Collections.Generic;
using System.Configuration;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using static System.Collections.Specialized.BitVector32;

namespace CryptographyTeachingTool.Data.Repositories
{
    class LessonRepo : DataClasses
    {
        /// <summary>
        /// Similarly to the User Repo this class will be used to perform
necessary select or insert into queries,
        /// but this time relating to the Lesson Syllabus and Lesson Details
tables.
        /// </summary>

        static public Lesson GetLessonByID(int lessonID)
        {
```

```
            // This static method is used to obtain, create and return the
information needed for a lesson inside an instance of the Lesson object.
            // The information is then used to display the lesson to the user in
the LessonViewer page.
            try
            {
                using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
                {
                    connection.Open();
                    string query = $"SELECT Text, Title, WorkPrompt, Image FROM
DefinedLessons WHERE lessonID = @lessonID";
                    using (SqlCommand command = new SqlCommand(query,
connection))
                    {
                        command.Parameters.AddWithValue("@lessonID", lessonID);
                        using (SqlDataReader reader = command.ExecuteReader())
                        {
                            if (reader.Read())
                            {
                                string text = reader.GetString(0);
                                string title = reader.GetString(1);
                                string workPrompt = reader.GetString(2);

                                if (!reader.IsDBNull(reader.GetOrdinal("Image")))
                                {
                                    byte[] imageData = (byte[])reader["Image"];
                                    Lesson lesson = new Lesson(lessonID, text,
title, workPrompt, imageData);
                                    return lesson;
                                }
                                else
                                {
                                    Lesson lesson = new Lesson(lessonID, text,
title, workPrompt);
                                    return lesson;
                                }
                            }
                            else { return null; }
                        }
                    }
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
                return null;
            }
        }
        static public List<Lesson> GetLessonsBySyllabus(string section, string
examBoard, Dictionary<string, int> lessonLevels)
        {
            // This static method is used to obtain, create and return a list of
instances of the Lesson object
            // which contain the lessons that are to be displayed to the user
given their selected exam board.
            List<Lesson> allLessons = new List<Lesson>();
            try
            {
                using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
                {
                    connection.Open();
```

```
                    string query = "SELECT SyllabusLessons.LessonID,
SyllabusLessons.LessonNo, DefinedLessons.Title " +
                        $"FROM SyllabusLessons, DefinedLessons " +
                        $"WHERE SyllabusLessons.LessonID =
DefinedLessons.LessonID AND SyllabusLessons.LessonLevel = @level AND
SyllabusLessons.ExamBoard = @exam " +
                        $"ORDER BY lessonNO";
                    using (SqlCommand command = new SqlCommand(query,
connection))
                    {
                        command.Parameters.AddWithValue("@level",
lessonLevels[section]);
                        command.Parameters.AddWithValue("@exam", examBoard);

                        using (SqlDataReader reader = command.ExecuteReader())
                        {
                            while (reader.Read())
                            {
                                int lessonID =
reader.GetInt32(reader.GetOrdinal("lessonID"));
                                int lessonNo =
reader.GetInt32(reader.GetOrdinal("LessonNo"));
                                string title =
reader.GetString(reader.GetOrdinal("Title"));
                                Lesson lesson = new Lesson(lessonID, lessonNo,
title);
                                allLessons.Add(lesson);
                            }
                        }
                    }
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
            }
            return allLessons;
        }
    }
}
```

AssignmentRepo.cs

```
using CryptographyTeachingTool.MainClasses;
using Microsoft.Data.SqlClient;
using System;
using System.Collections.Generic;
using System.IO.Compression;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms.Design;

namespace CryptographyTeachingTool.Data.Repositories
{
    class AssignmentRepo : DataClasses
    {
        /// <summary>
        /// This repository class will have the greatest number of methods, it
will be used to modularise
        /// and abstract the process of creating assignments, uploading and
downloading files,
```

```
        /// uploading marks and getting the average time spent by users on the
application.
        /// </summary>

        static public List<Assignment> GetAssignmentsForTeacher(User currentUser)
        {
            // This static method is used to obtain, create and return a list of
assignments that have been set by the current teacher.
            // The assignments will then be displayed on the teacher's dashboard.
            List<Assignment> allAssignments = new List<Assignment>();
            try
            {
                using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
                {
                    connection.Open();
                    string query = $"SELECT Title, DateSet, TotalMarks FROM
DefinedAssignments WHERE TeacherID = @teacherID ORDER BY DateSet DESC";
                    using (SqlCommand command = new SqlCommand(query,
connection))
                    {
                        command.Parameters.AddWithValue("@teacherID",
currentUser.userID);
                        using (SqlDataReader reader = command.ExecuteReader())
                        {
                            while (reader.Read())
                            {
                                string title =
reader.GetString(reader.GetOrdinal("Title"));
                                DateTime dateSet =
reader.GetDateTime(reader.GetOrdinal("DateSet"));
                                int totalMarks =
reader.GetInt32(reader.GetOrdinal("TotalMarks"));

                                Assignment assignment = new Assignment(title,
dateSet, totalMarks);
                                allAssignments.Add(assignment);
                            }
                        }
                    }
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
            }
            return allAssignments;
        }

        static public List<Assignment> GetAssignmentsForStudent(User currentUser)
        {
            // This static method is used to obtain, create and return a list of
assignments that have been set for the current student.
            // The assignments will then be displayed on the student's assignment
viewer page.
            List<Assignment> allAssignments = new List<Assignment>();
            try
            {
                using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
                {
                    connection.Open();
```

```
                    // where user id matches get teacher fname . surname set by,
date, TotalMarks
                    string query = $"SELECT DefinedAssignments.Title,
DefinedAssignments.DateSet, DefinedAssignments.TotalMarks,
DefinedAssignments.AssignmentID, Users.Firstname, Users.Surname " +
                        $"FROM DefinedAssignments " +
                        $"INNER JOIN Users ON DefinedAssignments.TeacherID =
Users.UserID " +
                        $"INNER JOIN LinkedAssignments ON
LinkedAssignments.AssignmentID = DefinedAssignments.AssignmentID " +
                        $"WHERE LinkedAssignments.UserID = @userID " +
                        $"ORDER BY DateSet";
                    using (SqlCommand command = new SqlCommand(query,
connection))
                    {
                        command.Parameters.AddWithValue("@userID",
currentUser.userID);
                        using (SqlDataReader reader = command.ExecuteReader())
                        {
                            while (reader.Read())
                            {
                                string title =
reader.GetString(reader.GetOrdinal("Title"));
                                DateTime dateSet =
reader.GetDateTime(reader.GetOrdinal("DateSet"));
                                int totalMarks =
reader.GetInt32(reader.GetOrdinal("TotalMarks"));
                                string firstname =
reader.GetString(reader.GetOrdinal("Firstname"));
                                string surname =
reader.GetString(reader.GetOrdinal("Surname"));
                                int assignmentID =
reader.GetInt32(reader.GetOrdinal("AssignmentID"));

                                Assignment assignment = new Assignment(title,
dateSet, totalMarks, firstname, surname, assignmentID);
                                allAssignments.Add(assignment);
                            }
                        }
                    }
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
            }
            return allAssignments;
        }

        static public List<Assignment> GetResults(User currentUser)
        {
            // This static method is used to obtain, create and return a list of
all assignments set by the current teacher
            // and their results.
            List<Assignment> allAssignments = new List<Assignment>();
            try
            {
                using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
                {
                    connection.Open();
```

```
                    string query = $"SELECT Mark, DefinedAssignments.Title,
DefinedAssignments.DateSet, Users.Firstname, Users.Surname,
DefinedAssignments.TotalMarks " +
                        "FROM LinkedAssignments " +
                        "INNER JOIN Users ON Users.UserID =
LinkedAssignments.UserID " +
                        "INNER JOIN DefinedAssignments ON
DefinedAssignments.AssignmentID = LinkedAssignments.AssignmentID " +
                        "WHERE DefinedAssignments.TeacherID = @teacherid AND Mark
IS NOT NULL " +
                        "ORDER BY DefinedAssignments.DateSet";

                    using (SqlCommand command = new SqlCommand(query,
connection))
                    {
                        command.Parameters.AddWithValue("@teacherid",
currentUser.userID);
                        using (SqlDataReader reader = command.ExecuteReader())
                        {

                            while (reader.Read())
                            {
                                string title =
reader.GetString(reader.GetOrdinal("Title"));
                                DateTime dateSet =
reader.GetDateTime(reader.GetOrdinal("DateSet"));
                                int actualMarks =
reader.GetInt32(reader.GetOrdinal("Mark"));
                                int totalMarks =
reader.GetInt32(reader.GetOrdinal("TotalMarks"));
                                string fullname =
$"{reader.GetString(reader.GetOrdinal("Firstname"))}
{reader.GetString(reader.GetOrdinal("Surname"))}";

                                Assignment assignment = new Assignment(title,
dateSet, totalMarks, actualMarks, fullname);
                                allAssignments.Add(assignment);
                            }
                        }
                    }
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
            }
            return allAssignments;
        }

        static public void CreateAssignment(User currentUser, Assignment
assignment, string[] users)
        {
            // This static method is used to create an entry to the Assignments
table, thus creating an assignment
            // for the current teacher. Along with storing the appropriate data
it also assigns the assignment to all necessary students.
            try
            {
                using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING)) // uploads the asignment data for both tables
                {
                    int assignmentID = -1;
                    connection.Open();
```

```
                    string query = "INSERT INTO DefinedAssignments (TeacherID,
Title, DateSet, OriginalFile, TotalMarks, FileName) " +
                        "VALUES (@userID, @title, @today, @binFile, @maxMarks,
@filename); " +
                        "SELECT CAST(SCOPE_IDENTITY() as int);"; // this is a
simple way of getting the assignmentID instead of requerying
                    using (SqlCommand command = new SqlCommand(query,
connection))
                    {
                        command.Parameters.AddWithValue("@userID",
currentUser.userID);
                        command.Parameters.AddWithValue("@title",
assignment.title);
                        command.Parameters.AddWithValue("@today",
assignment.dateSet);
                        command.Parameters.AddWithValue("@binFile",
assignment.binFile);
                        command.Parameters.AddWithValue("@maxMarks",
assignment.totalMarks);
                        command.Parameters.AddWithValue("@filename",
assignment.fileName);
                        assignmentID = Convert.ToInt32(command.ExecuteScalar());
// the execute scalar acts as the execute non query in this case
                    }

                    int[] IDs = new int[users.Length];
                    for (int i = 0; i < users.Length; i++)
                    {
                        // This loop goes through all of the associated users
                        // and creates a link between the assignment and them
                        string user = users[i];
                        query = $"SELECT UserID FROM Users WHERE Username =
@user";
                        using (SqlCommand command = new SqlCommand(query,
connection))
                        {
                            command.Parameters.AddWithValue("@user", user);
                            using (SqlDataReader reader =
command.ExecuteReader())
                            {
                                if (reader.Read())
                                {
                                    IDs[i] = reader.GetInt32(0);
                                }
                                else
                                {
                                    MessageBox.Show($"Could not find {user}");
                                }
                            }
                        }
                    }

                    query = "INSERT INTO LinkedAssignments (AssignmentID, UserID)
" +
                        "VALUES (@assignmentID, @userID)";
                    foreach (int id in IDs)
                    {
                        using (SqlCommand command = new SqlCommand(query,
connection))
                        {
                            command.Parameters.AddWithValue("@assignmentID",
assignmentID);
                            command.Parameters.AddWithValue("@userID", id);
```

```
                        command.ExecuteNonQuery();
                    }
                }

                if (assignmentID <= 0)
                {
                    MessageBox.Show("Assignment setting failed - could not
obtain AssignmentID.");
                }
                else
                {
                    MessageBox.Show("Assignment set.");
                }
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
        }
    }

    static public void UpdateMarks(int marks, int uID)
    {
        // This static method is used to update the Marks field of the
LinkedAssignments table,
        // thus allowing a student to share their marks with their teacher.
        using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
        {
            connection.Open();
            string query = $"UPDATE LinkedAssignments SET Mark = @marks WHERE
assignmentID = @id";
            using (SqlCommand command = new SqlCommand(query, connection))
            {
                command.Parameters.AddWithValue("@marks", marks);
                command.Parameters.AddWithValue("@id", uID);
                command.ExecuteNonQuery();
            }
        }
    }

    static public bool CanAddToAssignment(string username, ListBox
StudentsList)
    {
        // This static method is used to validate each username added to the
list of people
        // an assignment is to be set to. It validates them to ensure that
their username
        // exists, is a student rather than a teacher and has not already
been added.
        try
        {
            using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
            {
                connection.Open();
                string query = $"SELECT Role FROM Users WHERE Username =
'{username}'";

                using (SqlCommand command = new SqlCommand(query,
connection))
                {
                    using (SqlDataReader reader = command.ExecuteReader())
```

```
                        {
                            if (reader.Read())
                            {
                                if (reader.GetString(0) != "Student")
                                {
                                    MessageBox.Show("This is a teacher - you
cannot set assignments to your teacher!");
                                }
                                else if (StudentsList.Items.Contains(username))
                                {
                                    MessageBox.Show("This student is already on
the list.");
                                }
                                else
                                {
                                    return true;
                                }
                            }
                            else
                            {
                                MessageBox.Show("This username cannot be found
please check it before trying again.");
                            }
                        }
                    }
                    return false;
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"An unexpected error occured: \n{ex.Message}");
                return false;
            }
        }

        static public void DownloadFile(int assignmentID)
        {
            // This static method is used to save the file uploaded by a teacher
to an assignment.
            using (SqlConnection connection = new
SqlConnection(CONNECTION_STRING))
            {
                connection.Open();
                string query = $"SELECT OriginalFile, FileName FROM
DefinedAssignments " +
                    $"WHERE DefinedAssignments.AssignmentID = @ID ";
                using (SqlCommand command = new SqlCommand(query, connection))
                {
                    command.Parameters.AddWithValue("@ID", assignmentID);
                    using (SqlDataReader reader = command.ExecuteReader())
                    {
                        if (reader.Read())
                        {
                            byte[] fileBytes = (byte[])reader["OriginalFile"]; //
gets a byte array of the file
                            string filename = reader.GetString(1);
                            string downloadPath = Path.Combine(

Environment.GetFolderPath(Environment.SpecialFolder.UserProfile),
                                "Downloads", filename); // this finds the users
profile path then append the
                                                        // dowloads folder and
file name onto the string
```

```
                            File.WriteAllBytes(downloadPath, fileBytes);
                            MessageBox.Show("Sucessfully saved to your downloads
folder.");
                        }
                    }
                }
            }

        }
    }
}
```

User Interface

BaseFormsPage.cs

```
using CryptographyTeachingTool.Help;
using CryptographyTeachingTool.MainClasses;
using CryptographyTeachingTool.UI;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CryptographyTeachingTool.UI.Basic
{
    public class BaseFormsPage : Form
    {
        /// <summary>
        /// The help button will be visible in almost all pages therefore instead
of recreating the code for each page it is present;
        /// I will create a class that inherits from ‘Form’ and initializes the
help button.
        /// Then for form that will have the help button, I can make that form
inherit from this class.
        /// </summary>

        protected User currentUser; // This attribute holds all of the current
users details so
                                    // they can be accessed by the UI and then
reacted to on this level

        protected Button helpBtn;   // This stores the help button so that it can
be added to the screen dynamically

        // The following dictionary stores the mappings/encodings used for each
level in the code below.
        // It provides a fast way of converting between the string Level and then
a integer representing the level.
        // This dictionary will allow for lesson levels to easily be added and
interpretted from the database correctly.
        protected private Dictionary<string, int> LESSON_LEVELS = new
Dictionary<string, int>
        {
            { "FURTHER", 3 },
            { "ALEVEL", 2 },
            { "GCSE", 1 },
            { "DEMOS", 0 }
        };

        // The following dictionary holds all of the Exam Boards that are
individually recoginsed by the application
```

```
        protected string[] EXAM_BOARDS = { "AQA", "Edexcel", "OCR" };

        protected int BUTTON_HEIGHTS = 40;  // Provides a single constant for the
height of buttons which is then used
                                            // when creating the button or
padding for the display

        public BaseFormsPage(User user, bool isHelpBtnVis)
        {
            // This is the base (and only) constructor which adds the help button
to the
            // page and assigns the ClosingApplication method to action of
FormClosed
            currentUser = user;
            InitializeComponent();
            if ( isHelpBtnVis ) { helpBtn = InitalizeHelpButton(); }
            this.FormClosed += ClosingApplication;
        }

        protected virtual void ClosingApplication(object sender,
FormClosedEventArgs e)
        {
            // This method is used to call the SignOut method of the user
            // (thus adding the signout time to an appropriate entry of the
AuditLog table).
            currentUser.SignOut();
            Application.Exit();
        }

        protected Button InitalizeHelpButton()
        {
            // This method is used to create and add a help button into the
botton-left corner of the pages.
            Button btn = new Button();
            btn.Text = "Help";
            btn.Size = new Size(150, BUTTON_HEIGHTS);
            btn.Location = new Point(10, this.ClientSize.Height - btn.Height -
10); // bottom-left corner
            btn.Anchor = AnchorStyles.Bottom | AnchorStyles.Left;
            btn.Click += HelpButtonClick;
            btn.BackColor = Color.FromArgb(47, 54, 64);
            btn.ForeColor = Color.FromArgb(220, 221, 225);
            Controls.Add(btn);
            return btn;
        }

        private void InitializeComponent()
        {
            // This method creates a page of size 1000, 600 in the center of the
screen
            SuspendLayout();
            //
            // BaseFormsPage
            //
            BackColor = Color.FromArgb(47, 54, 64);
            ClientSize = new Size(1000, 600);
            FormBorderStyle = FormBorderStyle.FixedSingle;
            MinimumSize = new Size(720, 432);
            Name = "BaseFormsPage";
            ShowIcon = false;
            StartPosition = FormStartPosition.CenterScreen;
            ResumeLayout(false);
        }
```

```
        private void HelpButtonClick(object? sender, EventArgs e)
        {
            // This method creates and shows the help form page, it does not
close the current page.
            HelpForm helpForm = new HelpForm();
            helpForm.Show();
        }
    }
}
```

GeneralDashboardForm.cs

```
using CryptographyTeachingTool.Data.Repositories;
using CryptographyTeachingTool.Login;
using CryptographyTeachingTool.MainClasses;
using CryptographyTeachingTool.UI;
using CryptographyTeachingTool.UI.Student;
using CryptographyTeachingTool.UI.Teacher;
using Microsoft.Data.SqlClient;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Security.AccessControl;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace CryptographyTeachingTool.UI.Basic
{
    public partial class GeneralDashboardForm : BaseFormsPage
    {
        // Base forms contains currentUserUser object
        public GeneralDashboardForm(User user) : base(user, false)
        {
            InitializeComponent();
            InitalizeHelpButton();
            if (currentUser.role == "Teacher") { TeacherDashboardbtn.Visible =
true; AssignmentsButton.Visible = false; }
            else { TeacherDashboardbtn.Visible = false; AssignmentsButton.Visible
= true; }
            WelcomeMessage.Text = $"Welcome, {currentUser.firstName}!";
            notUserTxt.Text = $"not {currentUser.firstName}?";
        }

        private void GeneralDashboardFormClosed(object sender,
FormClosedEventArgs e)
        {
            currentUser.SignOut();
            Application.Exit();
        }

        // These following buttons are used for creating an instance of
NavigationalPagesTemplate
        // and adding it to the screen. When the instance is created the
appropraite level (eg GCSE)
        // is passed in as a parameter - along side the current user object.

        private void GCSEButtonClick(object sender, EventArgs e)
```

```
        {
            NavigationalPagesTemplate gcse = new
NavigationalPagesTemplate(currentUser, "GCSE");
            gcse.Show();
            this.Hide();
        }

        private void AlevelButtonClick(object sender, EventArgs e)
        {
            NavigationalPagesTemplate alevel = new
NavigationalPagesTemplate(currentUser, "ALEVEL");
            alevel.Show();
            this.Hide();
        }

        private void DemoButtonClick(object sender, EventArgs e)
        {
            NavigationalPagesTemplate demos = new
NavigationalPagesTemplate(currentUser, "DEMOS");
            demos.Show();
            this.Hide();
        }

        private void FurtherButtonClick(object sender, EventArgs e)
        {
            NavigationalPagesTemplate further = new
NavigationalPagesTemplate(currentUser, "FURTHER");
            further.Show();
            this.Hide();
        }

        private void TeacherDashboardButtonClick(object sender, EventArgs e)
        {
            TeachersDashboard tDash = new TeachersDashboard(currentUser);
            tDash.Show();
            this.Hide();
        }

        private void GeneralDashboardFormLoad(object sender, EventArgs e)
        {
            // This method queries for and renders the appropriate progress
information
            // of the current user into a progress bar

            int currentUserTotal = UserRepo.GetCurrentUserTotal(currentUser);
            int avgUserTotal = UserRepo.GetAvgUserTotal();

            progressBar.Maximum = Math.Max(avgUserTotal * 2, 1); // this is just
incase when desigining the application there are no entries
            progressBar.Value = Math.Min(currentUserTotal, progressBar.Maximum);
// this min statement is for the rare occasions when an individual is double the
average

            userTime.Text = "Your time spent: " + currentUserTotal + "mins";
            avgTime.Text = "Average user time spent: " + avgUserTotal + "mins";

        }

        private void AssignmentsButtonClick(object sender, EventArgs e)
        {
            ViewAssignments assignments = new ViewAssignments(currentUser);
            assignments.Show();
        }
```

```
        private void SignOutButtonClick(object sender, EventArgs e)
        {
            // This method logs a user out - calling the SignOut method to
            // add the signout time to the audit log

            currentUser.SignOut();
            LoginForm login = new LoginForm();
            login.Show();
            this.Hide();
        }
    }
}
```

Lessons

InsertingImagesQuery.txt

```
UPDATE DefinedLessons
SET [Image] = (
    SELECT BulkColumn
    FROM OPENROWSET(
        BULK 'C:\Users\User\Downloads\ceaserCipher.png',
        SINGLE_BLOB
    ) AS img
)
WHERE LessonID = 1;
```

NavigationalPagesTemplate.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Collections.Specialized.BitVector32;
using Microsoft.Data.SqlClient;
using System.Speech.Synthesis;
using static System.Windows.Forms.VisualStyles.VisualStyleElement.StartPanel;
using System.Security.Cryptography.X509Certificates;
using Microsoft.IdentityModel.Tokens;
using CryptographyTeachingTool.MainClasses;
using CryptographyTeachingTool.Data.Repositories;

namespace CryptographyTeachingTool.UI.Basic
{
    public partial class LessonPage : NavigationalPagesTemplate
    {
        /// <summary>
        /// This class inherits from the GeneralForms class. There will be
        /// many pages that will need a back button and a flow layout panel in
the same space,
        /// for example a lesson on the Caesar Cipher. Similarly to the
GeneralForms class this is a
        /// parent to many different lesson screens (this could in include
viewing the syllabus or a lesson).
        /// </summary>

        private Lesson lesson; // This lesson object stores the details to be
displayed on the page
```

```
        private SpeechSynthesizer synth = new SpeechSynthesizer();  // The synth
attribute is used to store and object of
                                                                    // type
SpeechSynthesizer which is later used to read the
                                                                    // text out
loud.
        public LessonPage(User user, Lesson lesson, string level) : base(user,
level, false)
        {
            // This is the only constructor for this page, it is used to add
lesson information based on the lessonID given.
            this.lesson = lesson;
            this.lesson = LessonRepo.GetLessonByID(lesson.lessonID);
            AddComponents();
            DisplayLesson(lesson.text, lesson.workPrompt);
            if (!lesson.imageData.IsNullOrEmpty())
            {
                AddImage(lesson.imageData);
            }
        }

        private void AddImage(byte[] imageData)
        {
            // This method is used to load and add an image to the lesson page
when given the image in a byte array.
            // It does this in a manner that ensures the thread does not freeze.
            using (var ms = new MemoryStream(imageData))
            {
                PictureBox picture = new PictureBox();
                picture.Image = Image.FromStream(ms);
                picture.SizeMode = PictureBoxSizeMode.Zoom;
                picture.Width = flowLayoutPanel.ClientSize.Width - 30;
                picture.Height = 300;
                picture.Margin = new Padding(10);
                picture.Dock = DockStyle.Top;

                flowLayoutPanel.Controls.Add(picture);
            }
        }

        private void AddComponents()
        {
            // This method is used to initalize a flowlayout pannel before
initalizing the rest of the window.
            this.components = new System.ComponentModel.Container();
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.Text = lesson.title;
            flowLayoutPanel.Controls.Clear();
            InitializeComponent();
        }

        private void TextToSpeech(object sender, EventArgs e)
        {
            // This method is called when the text to speech button is pressed
and reads the text on the page asynchronously.
            synth.SpeakAsyncCancelAll(); // prevents having multiple voices at
once
            synth.SpeakAsync(lesson.text); // this async version prevents the
thread from
                                           // freezing so the user can stop the
synthesizer
        }
```

```
        private void StopSpeech(object sender, EventArgs e)
        {
            // This method is called when the stop button is pressed, and
resultingly stops the voice from speaking.
            synth.SpeakAsyncCancelAll();
        }
    }
}
```

LessonPage.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Collections.Specialized.BitVector32;
using Microsoft.Data.SqlClient;
using System.Speech.Synthesis;
using static System.Windows.Forms.VisualStyles.VisualStyleElement.StartPanel;
using System.Security.Cryptography.X509Certificates;
using Microsoft.IdentityModel.Tokens;
using CryptographyTeachingTool.MainClasses;
using CryptographyTeachingTool.Data.Repositories;

namespace CryptographyTeachingTool.UI.Basic
{
    public partial class LessonPage : NavigationalPagesTemplate
    {
        /// <summary>
        /// This class inherits from the GeneralForms class. There will be
        /// many pages that will need a back button and a flow layout panel in
the same space,
        /// for example a lesson on the Caesar Cipher. Similarly to the
GeneralForms class this is a
        /// parent to many different lesson screens (this could in include
viewing the syllabus or a lesson).
        /// </summary>

        private Lesson lesson; // This lesson object stores the details to be
displayed on the page
        private SpeechSynthesizer synth = new SpeechSynthesizer();  // The synth
attribute is used to store and object of
                                                                    // type
SpeechSynthesizer which is later used to read the
                                                                    // text out
loud.
        public LessonPage(User user, Lesson lesson, string level) : base(user,
level, false)
        {
            // This is the only constructor for this page, it is used to add
lesson information based on the lessonID given.
            this.lesson = lesson;
            this.lesson = LessonRepo.GetLessonByID(lesson.lessonID);
            AddComponents();
            DisplayLesson(lesson.text, lesson.workPrompt);
            if (!lesson.imageData.IsNullOrEmpty())
            {
```

```
                AddImage(lesson.imageData);
            }
        }

        private void AddImage(byte[] imageData)
        {
            // This method is used to load and add an image to the lesson page
when given the image in a byte array.
            // It does this in a manner that ensures the thread does not freeze.
            using (var ms = new MemoryStream(imageData))
            {
                PictureBox picture = new PictureBox();
                picture.Image = Image.FromStream(ms);
                picture.SizeMode = PictureBoxSizeMode.Zoom;
                picture.Width = flowLayoutPanel.ClientSize.Width - 30;
                picture.Height = 300;
                picture.Margin = new Padding(10);
                picture.Dock = DockStyle.Top;

                flowLayoutPanel.Controls.Add(picture);
            }
        }

        private void AddComponents()
        {
            // This method is used to initalize a flowlayout pannel before
initalizing the rest of the window.
            this.components = new System.ComponentModel.Container();
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.Text = lesson.title;
            flowLayoutPanel.Controls.Clear();
            InitializeComponent();
        }

        private void TextToSpeech(object sender, EventArgs e)
        {
            // This method is called when the text to speech button is pressed
and reads the text on the page asynchronously.
            synth.SpeakAsyncCancelAll(); // prevents having multiple voices at
once
            synth.SpeakAsync(lesson.text); // this async version prevents the
thread from
                                    // freezing so the user can stop the
synthesizer
        }

        private void StopSpeech(object sender, EventArgs e)
        {
            // This method is called when the stop button is pressed, and
resultingly stops the voice from speaking.
            synth.SpeakAsyncCancelAll();
        }
    }
}
```

Demos

DemosBasePage.cs

```
using CryptographyTeachingTool.Help;
using CryptographyTeachingTool.MainClasses;
using CryptographyTeachingTool.UI;
using CryptographyTeachingTool.UI.Basic;
using System;
```

```
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace CryptographyTeachingTool.UI.Demonstrations
{
    public partial class DemosBasePage : Form
    {
        protected User currentUser;
        protected Button helpbtn;
        protected Button backbtn;

        public DemosBasePage(User user)
        {
            this.currentUser = user;
            ClientSize = new Size(1142, 750);
            helpbtn = InitalizeHelpButton();
            backbtn = InitalizeBacktoDashBtn();
            helpbtn.BringToFront();
            backbtn.BringToFront();
            this.FormClosed += ClosingWindow;
        }

        private Button InitalizeHelpButton()
        {
            // This method creates the help button for all forms pages that run
this method.
            // It adds it to the bottom left corner and assigns the
HelpButtonClick method at runtime

            Button btn = new Button();
            btn.Text = "Help";
            btn.Size = new Size(150, 40);
            btn.Location = new Point(10, this.ClientSize.Height - btn.Height -
10); // bottom left corner
            btn.Anchor = AnchorStyles.Bottom | AnchorStyles.Left;
            btn.Click += HelpButtonClick;
            btn.BackColor = Color.FromArgb(47, 54, 64);
            btn.ForeColor = Color.FromArgb(220, 221, 225);
            Controls.Add(btn);
            return btn;
        }
        private void HelpButtonClick(object? sender, EventArgs e)
        {
            // This method is run when the help button is pressed.
            // It simply opens the help form as a new window

            HelpForm helpForm = new HelpForm();
            helpForm.Show();
        }

        private Button InitalizeBacktoDashBtn()
        {
            // This method adds a back to dashboard button into the bottom right
corner of the window
            // It assigns the BackToDashboardButton method at runtime
```

```
            Button btn = new Button();
            btn.Text = "Back";
            btn.Size = new Size(150, 40);
            btn.BackColor = Color.FromArgb(47, 54, 64);
            btn.ForeColor = Color.FromArgb(220, 221, 225);
            btn.Location = new Point(
                this.ClientSize.Width - btn.Width - 10,
                this.ClientSize.Height - btn.Height - 10); // bottom right corner
            Controls.Add(btn);
            btn.Anchor = AnchorStyles.Bottom | AnchorStyles.Right;
            btn.Click += BackToDashboardButton;
            return btn;
        }
        private void BackToDashboardButton(object? sender, EventArgs e)
        {
            // This method is called when the back to dashboard button is pressed
            // It opens the general dashboard and closes the current page.

            GeneralDashboardForm dash = new GeneralDashboardForm(currentUser);
            dash.Show();
            this.Hide();
        }

        protected void ClosingWindow(object sender, FormClosedEventArgs e)
        {
            // This method opens the general dashboard when the current window is
closed via the cross
            // in the top right hand corner of the window
            GeneralDashboardForm dash = new GeneralDashboardForm(currentUser);
            dash.Show();
            this.Hide();
        }

    }
}
```

CaesarDemo.cs

```
using CryptographyTeachingTool.Cryptography;
using CryptographyTeachingTool.Cryptography.Ciphers;
using CryptographyTeachingTool.Help;
using CryptographyTeachingTool.MainClasses;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace CryptographyTeachingTool.UI.Demonstrations
{
    public partial class CaesarDemo : DemosBasePage
    {
        public CaesarDemo(User user) : base(user)
        {
            InitializeComponent();
        }

        private void EncryptButtonClick(object sender, EventArgs e)
        {
```

```
            // This method gets the key and plain text a user entered and
displays the result of
            // running the Encrypt method for the caesar cipher object
            try
            {
                string plainText = PlainTextBox.Text;
                int key = Convert.ToInt32(KeySelector.Value);
                CaesarCipher caesar = new CaesarCipher();
                string result = caesar.Encrypt(key, plainText);
                CipherTextBox.Text = result;
            }
            catch (Exception ex)
            {
                MessageBox.Show($"There has been an error:\n{ex.Message}");
            }
        }

        private void DecryptButtonClick(object sender, EventArgs e)
        {
            // This method gets the key and cipher text a user entered and
displays the result of
            // running the Decrypot method for the caesar cipher object
            try
            {
                string plainText = CipherTextBox.Text;
                int key = Convert.ToInt32(KeySelector.Value);
                CaesarCipher caesar = new CaesarCipher();
                string result = caesar.Decrypt(key, plainText);
                PlainTextBox.Text = result;
            }
            catch (Exception ex)
            {
                MessageBox.Show($"There has been an error:\n{ex.Message}");
            }
        }
    }
}
```

VernamDemo.cs

```
using CryptographyTeachingTool.Cryptography;
using CryptographyTeachingTool.Cryptography.Ciphers;
using CryptographyTeachingTool.Help;
using CryptographyTeachingTool.MainClasses;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace CryptographyTeachingTool.UI.Demonstrations
{
    public partial class CaesarDemo : DemosBasePage
    {
        public CaesarDemo(User user) : base(user)
        {
            InitializeComponent();
        }
```

```
        private void EncryptButtonClick(object sender, EventArgs e)
        {
            // This method gets the key and plain text a user entered and
displays the result of
            // running the Encrypt method for the caesar cipher object
            try
            {
                string plainText = PlainTextBox.Text;
                int key = Convert.ToInt32(KeySelector.Value);
                CaesarCipher caesar = new CaesarCipher();
                string result = caesar.Encrypt(key, plainText);
                CipherTextBox.Text = result;
            }
            catch (Exception ex)
            {
                MessageBox.Show($"There has been an error:\n{ex.Message}");
            }
        }

        private void DecryptButtonClick(object sender, EventArgs e)
        {
            // This method gets the key and cipher text a user entered and
displays the result of
            // running the Decrypot method for the caesar cipher object
            try
            {
                string plainText = CipherTextBox.Text;
                int key = Convert.ToInt32(KeySelector.Value);
                CaesarCipher caesar = new CaesarCipher();
                string result = caesar.Decrypt(key, plainText);
                PlainTextBox.Text = result;
            }
            catch (Exception ex)
            {
                MessageBox.Show($"There has been an error:\n{ex.Message}");
            }
        }
    }
}
```

RailFenceDemo.cs

```
using CryptographyTeachingTool.Help;
using CryptographyTeachingTool;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using CryptographyTeachingTool.MainClasses;
using CryptographyTeachingTool.Cryptography;
using CryptographyTeachingTool.UI.Demonstrations;
using CryptographyTeachingTool.Cryptography.Ciphers;

namespace CryptographyTeachingTool.UI.Demos
{
    public partial class RailfenceDemo : DemosBasePage
```

```
    {

        public RailfenceDemo(User user) : base(user)
        {
            InitializeComponent();
        }

        private void EncryptButtonClick(object sender, EventArgs e)
        {
            // This method gets the key and plain text a user entered and
displays the result of
            // running the Encrypt method for the railfense cipher object

            try
            {
                string plainText = PlainTextBox.Text;
                int key = Convert.ToInt32(numericUpDown.Value);
                if (key <= 1)
                {
                    MessageBox.Show("Enter a key that is greater than 1");
                }
                else
                {
                    RailFenceCipher rail = new RailFenceCipher();
                    string result = rail.Encrypt(key, plainText);
                    CipherTextBox.Text = result;
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"There has been an error:\n{ex.Message}");
            }
        }

        private void DecryptButtonClick(object sender, EventArgs e)
        {
            // This method gets the key and plain text a user entered and
displays the result of
            // running the Decrypt method for the railfence cipher object

            try
            {
                string plainText = CipherTextBox.Text;
                int key = Convert.ToInt32(numericUpDown.Value);
                if (key <= 1)
                {
                    MessageBox.Show("Enter a key that is greater than 1");
                }
                else
                {
                    RailFenceCipher rail = new RailFenceCipher();
                    string result = rail.Decrypt(key, plainText);
                    PlainTextBox.Text = result;
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"There has been an error:\n{ex.Message}");
            }
        }
    }

}
```

SHADemo.cs

```
using CryptographyTeachingTool.Help;
using CryptographyTeachingTool;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using CryptographyTeachingTool.MainClasses;
using CryptographyTeachingTool.Cryptography;
using CryptographyTeachingTool.UI.Demonstrations;
using CryptographyTeachingTool.Cryptography.Ciphers;

namespace CryptographyTeachingTool.UI.Demos
{
    public partial class RailfenceDemo : DemosBasePage
    {

        public RailfenceDemo(User user) : base(user)
        {
            InitializeComponent();
        }

        private void EncryptButtonClick(object sender, EventArgs e)
        {
            // This method gets the key and plain text a user entered and
displays the result of
            // running the Encrypt method for the railfense cipher object

            try
            {
                string plainText = PlainTextBox.Text;
                int key = Convert.ToInt32(numericUpDown.Value);
                if (key <= 1)
                {
                    MessageBox.Show("Enter a key that is greater than 1");
                }
                else
                {
                    RailFenceCipher rail = new RailFenceCipher();
                    string result = rail.Encrypt(key, plainText);
                    CipherTextBox.Text = result;
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"There has been an error:\n{ex.Message}");
            }
        }

        private void DecryptButtonClick(object sender, EventArgs e)
        {
            // This method gets the key and plain text a user entered and
displays the result of
            // running the Decrypt method for the railfence cipher object
```

```
            try
            {
                string plainText = CipherTextBox.Text;
                int key = Convert.ToInt32(numericUpDown.Value);
                if (key <= 1)
                {
                    MessageBox.Show("Enter a key that is greater than 1");
                }
                else
                {
                    RailFenceCipher rail = new RailFenceCipher();
                    string result = rail.Decrypt(key, plainText);
                    PlainTextBox.Text = result;
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"There has been an error:\n{ex.Message}");
            }
        }
    }

}
```

Student

InputMarksForm.cs

```
using CryptographyTeachingTool.Data.Repositories;
using CryptographyTeachingTool.UI.Basic;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace CryptographyTeachingTool.UI.Student
{
    public partial class InputMarksForm : Form
    {
        private int ID;
        public InputMarksForm(ViewAssignments assign, int id)
        {
            this.ID = id;
            InitializeComponent();
        }

        private void UpdateMarksButtonClick(object sender, EventArgs e)
        {
            // This method validates the number of marks inputted by the user
before querying the database
            // to update the new number of marks. Note: it does this through the
UpdateMarks method of the AssignmentRepo
            int marksEntered = Convert.ToInt16(marksInput.Value);
            if (marksEntered < 0 )
            {
                MessageBox.Show("Please enter a mark that is greater than or
equal to 0");
            }
```

```
            else
            {
                AssignmentRepo.UpdateMarks(marksEntered, ID);
            }
            this.Hide();
        }
    }
}
```

ViewAssignments.cs

```
using CryptographyTeachingTool.Data.Repositories;
using CryptographyTeachingTool.MainClasses;
using Microsoft.Data.SqlClient;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.IO.Compression;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement.Tab;
using CryptographyTeachingTool.UI.Basic;

namespace CryptographyTeachingTool.UI.Student
{
    public partial class ViewAssignments : Form
    {
        private User currentUser;

        public ViewAssignments(User user)
        {
            currentUser = user;
            InitializeComponent();
            AssignmentPanel.AutoScroll = true;
            FetchAsignmentsSet();
        }

        private void FetchAsignmentsSet()
        {
            // This method gets the assignments returned by the
GetAssignmentsForStudent method of the AssignmentRepo
            // it then displays each of these on the page

            List<Assignment> allAssignments =
AssignmentRepo.GetAssignmentsForStudent(currentUser);
            foreach(Assignment assignment in allAssignments)
            {
                Panel bubble = CreateAssignmentBubble(assignment.title,
assignment.dateSet, assignment.totalMarks, assignment.firstname,
assignment.surname, assignment.assignmentID);
                bubble.Width = AssignmentPanel.ClientSize.Width -
bubble.Margin.Horizontal;
                AssignmentPanel.Controls.Add(bubble); // adding to the flow panel
            }
        }
```

```
        private Panel CreateAssignmentBubble(string title, DateTime dateSet, int
totalMarks, string fName, string sName, int id) // using similar code to the help
page bubbles
        {
            // This method is run to create an instance of a panel object that
can be added to the
            // screen for displaying each assignment in a bubble like manner

            Panel bubble = new Panel();
            bubble.AutoSize = true;
            bubble.BackColor = Color.FromArgb(39, 60, 117);
            bubble.ForeColor = Color.FromArgb(245, 246, 250);
            bubble.BorderStyle = BorderStyle.None;
            bubble.Margin = new Padding(10);
            bubble.Padding = new Padding(15);

            bubble.Paint += (s, e) => // use paint to create the rounded corners
            {
                using (GraphicsPath path = new GraphicsPath())
                {
                    int radius = 20; // controlling how round the cornersare
                    Rectangle rect = bubble.ClientRectangle;
                    path.AddArc(rect.X, rect.Y, radius, radius, 180, 90);
                    path.AddArc(rect.Right - radius, rect.Y, radius, radius, 270,
90);
                    path.AddArc(rect.Right - radius, rect.Bottom - radius,
radius, radius, 0, 90);
                    path.AddArc(rect.X, rect.Bottom - radius, radius, radius, 90,
90);
                    path.CloseFigure();

                    bubble.Region = new Region(path);
                    e.Graphics.SmoothingMode = SmoothingMode.AntiAlias;
                    using (Pen pen = new Pen(Color.FromArgb(245, 246, 250), 5))
                    {
                        e.Graphics.DrawPath(pen, path);
                    }
                }
            };

            // write all the text
            Label lblTitle = new Label();
            lblTitle.Text = title;
            lblTitle.Font = new Font("Segoe UI", 13, FontStyle.Bold);
            lblTitle.AutoSize = true;
            Label lblDate = new Label();
            lblDate.Text = $"{fName[0]}.{sName}, Date: {dateSet:dd MMM yyyy}";
            lblDate.Font = new Font("Segoe UI", 10, FontStyle.Regular);
            lblDate.AutoSize = true;
            Label lblMarks = new Label();
            lblMarks.Text = $"Total Marks: {totalMarks}";
            lblMarks.Font = new Font("Segoe UI", 8, FontStyle.Regular);
            lblMarks.AutoSize = true;

            FlowLayoutPanel FlowLayout = new FlowLayoutPanel(); // this controls
how the info in the bubble is layed out
            FlowLayout.FlowDirection = FlowDirection.TopDown;
            FlowLayout.AutoSize = false;
            FlowLayout.Dock = DockStyle.Fill;
            FlowLayout.WrapContents = false;
            FlowLayout.Controls.Add(lblTitle);
            FlowLayout.Controls.Add(lblDate);
            FlowLayout.Controls.Add(lblMarks);
```

```
            // create the buttons
            Button addMarksBtn = new Button
            {
                Text = "Upload Marks",
                BackColor = Color.FromArgb(47, 54, 64),
                ForeColor = Color.FromArgb(220, 221, 225),
                AutoSize = true,
                FlatStyle = FlatStyle.Flat,
                Margin = new Padding(3, 3, 3, 3),
                Dock = DockStyle.Fill
            };
            Button downloadBtn = new Button
            {
                Text = "Download File",
                BackColor = Color.FromArgb(47, 54, 64),
                ForeColor = Color.FromArgb(220, 221, 225),
                AutoSize = true,
                FlatStyle = FlatStyle.Flat,
                Margin = new Padding(3, 3, 3, 3),
                Dock = DockStyle.Fill
            };

            addMarksBtn.Click += (s, e) => UploadMarks(s, e, id);
            downloadBtn.Click += (s, e) => DownloadFile(s, e, id);

            TableLayoutPanel buttons = new TableLayoutPanel // this is to ensure
the buttons are equal and side by side
            {
                ColumnCount = 2,
                AutoSize = true,
                Dock = DockStyle.Top
            };
            buttons.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 50F));
            buttons.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 50F));
            buttons.Controls.Add(addMarksBtn);
            buttons.Controls.Add(downloadBtn);

            FlowLayoutPanel bubbleLayout = new FlowLayoutPanel // add all
components into one final layout panel
            {
                FlowDirection = FlowDirection.TopDown,
                AutoSize = true,
                WrapContents = false,
                Dock = DockStyle.Fill
            };

            bubbleLayout.Controls.Add(FlowLayout); // adds the text inside the
bubble
            bubbleLayout.Controls.Add(buttons);    // this adds the butons table
layout into the bubble

            bubble.Controls.Add(bubbleLayout);
            return bubble;
        }

        private void UploadMarks(object sender, EventArgs e, int id)
        {
            // This method opens the input marks form where the user can select
their marks.

            InputMarksForm marks = new InputMarksForm(this, id);
            marks.Show();
```

```
        }

        private void DownloadFile(object sender, EventArgs e, int assignmentID)
        {
            // This method is assigned to the download button click event at run
time.
            // Thus is called when the button is pressed

            AssignmentRepo.DownloadFile(assignmentID);
        }
    }
}
```

Teacher

SetAssignments.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using CryptographyTeachingTool.UI;
using Microsoft.Data.SqlClient;
using static System.Windows.Forms.VisualStyles.VisualStyleElement.StartPanel;
using CryptographyTeachingTool.UI.Basic;
using CryptographyTeachingTool.Data.Repositories;
using CryptographyTeachingTool.MainClasses;

namespace CryptographyTeachingTool.UI.Teacher
{
    public partial class SetAssignments : Form
    {
        private User currentUser;
        private TeachersDashboard teacherDash;

        public SetAssignments(User user, TeachersDashboard dash)
        {
            currentUser = user;
            teacherDash = dash;
            InitializeComponent();
        }

        private void AddStudentButtonClick(object sender, EventArgs e)
        {
            // This method adds the students username - as entered in the
username box
            // - to the list of students that the assignment is to be set to

            try
            {
                string username = StudentNameText.Text;
                if (AssignmentRepo.CanAddToAssignment(username, StudentsList))
                {
                    StudentsList.Items.Add(username);
                }
            }
            catch (Exception ex)
            {
```

```
                MessageBox.Show($"There has been an error:\n{ex.Message}");
            }
        }

        private void RemoveStudentButtonClick(object sender, EventArgs e)
        {
            // This method removes a student's username from the StudentsList
            // after validating to ensure that it does exist

            try
            {
                string username = StudentNameText.Text;
                if (StudentsList.Items.Contains(username))
                {
                    StudentsList.Items.Remove(username);
                }
                else
                {
                    MessageBox.Show("That student is not on the list");
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"There has been an error:\n{ex.Message}");
            }
        }

        private void BrowseButtonClick(object sender, EventArgs e)
        {
            // This method opens a browsing menu for teachers to select their
file easily.
            // It saves only the path.
            try
            {
                using (OpenFileDialog dialog = new OpenFileDialog())
                {
                    dialog.Title = "Select the assignments file";
                    // using varbinary allows for around 2gb to be stored on my
dataframe
                    // therefore there is no need to filter the files.
                    dialog.InitialDirectory =
Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
                    if (dialog.ShowDialog() == DialogResult.OK)
                    {
                        string path = dialog.FileName;
                        PathText.Text = path;
                    }
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"There has been an error:\n{ex.Message}");
            }
        }

        private void AddAssignmentButtonClick(object sender, EventArgs e)
        {
            // This method validates and stores all assignment information,
calling the
            // CreateAssignment method from the AssignmentRepo to add the
information to the database
            try
            {
```

```
                string[] users = StudentsList.Items.Cast<string>().ToArray();
                string title = TitleTextBox.Text;
                string path = PathText.Text;
                int userID = currentUser.userID;
                string name = Path.GetFileName(path); // this includes the
'.extention'
                byte[] binFile;
                if (name.Length <= 0) // only reads the file if one is attached
                {
                    return;
                }
                else
                {
                    binFile = File.ReadAllBytes(path); // converts the selected
file to bytes and stores it
                }
                int maxMarks = Convert.ToInt16(TotalMarks.Value);
                DateTime today = DateTime.Now;

                Assignment assignment = new Assignment(title, today, maxMarks,
binFile, name);
                AssignmentRepo.CreateAssignment(currentUser, assignment, users);
                teacherDash.FetchAsignmentsSet();
            }
            catch (Exception ex)
            {
                MessageBox.Show($"There has been an error:\n{ex.Message}");
            }
        }

        private void CloseButtonClick(object sender, EventArgs e)
        {
            this.Close();
        }
    }
}
```

TeacherDashboard.cs

```
using CryptographyTeachingTool.Data.Repositories;
using CryptographyTeachingTool.MainClasses;
using CryptographyTeachingTool.UI.Basic;
using Microsoft.Data.SqlClient;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Linq;
using System.Numerics;
using System.Reflection;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.ComponentModel.Design.ObjectSelectorEditor;

namespace CryptographyTeachingTool.UI.Teacher
{
    public partial class TeachersDashboard : Form
    {
        private Button dash;
```

```
        private User currentUser;
        public TeachersDashboard(User user)
        {
            FormClosed += BackToDashboardbtn_Click;
            currentUser = user;
            InitializeComponent();
            dash = InitalizeBacktoDashBtn();
            dash.BringToFront();
            TeacherName.Text = $"Teacher: {user.username}";
            FetchAsignmentsSet();
            FetchResults();
        }

        public void FetchAsignmentsSet()
        {
            // This method is used to get the assignments set by the currently
logged in teacher
            // and then display them on the screen

            RecentAsignmentsPanel.Controls.Clear();

            List<Assignment> assignmentList =
AssignmentRepo.GetAssignmentsForTeacher(currentUser);
            foreach (Assignment assignment in assignmentList)
            {
                Panel bubble = CreateAssignmentBubble(assignment.title,
assignment.dateSet, assignment.totalMarks);
                bubble.Width = RecentAsignmentsPanel.ClientSize.Width -
bubble.Margin.Horizontal;
                RecentAsignmentsPanel.Controls.Add(bubble); // adding to the flow
panel
            }
        }

        private void FetchResults()
        {
            // This method is used to get the marks from students with
assignments associated to
            // the currently logged in teacher and then display them on the
screen

            List<Assignment> assignments =
AssignmentRepo.GetResults(currentUser);
            foreach (Assignment assignment in assignments)
            {
                Panel bubble = CreateResultsBubble(assignment.title,
assignment.dateSet, assignment.actualMarks, assignment.totalMarks,
assignment.fullname);
                bubble.Width = RecentResultsPanel.ClientSize.Width -
bubble.Margin.Horizontal;
                RecentResultsPanel.Controls.Add(bubble); // adding to the flow
panel
            }
        }

        private Panel CreateAssignmentBubble(string title, DateTime dateSet, int
totalMarks)
        {
            // This method creates a a bubble which displays all of the key
information of currently set assignments

            Panel bubble = new Panel();
            bubble.AutoSize = false;
```

```
            bubble.BackColor = Color.FromArgb(39, 60, 117);
            bubble.ForeColor = Color.FromArgb(245, 246, 250);
            bubble.BorderStyle = BorderStyle.None;
            bubble.Margin = new Padding(10);
            bubble.Padding = new Padding(15);

            bubble.Paint += (s, e) => // use paint to create the rounded corners
            {
                using (GraphicsPath path = new GraphicsPath())
                {
                    int radius = 20; // controlling how round the cornersare
                    Rectangle rect = bubble.ClientRectangle;
                    path.AddArc(rect.X, rect.Y, radius, radius, 180, 90);
                    path.AddArc(rect.Right - radius, rect.Y, radius, radius, 270,
90);
                    path.AddArc(rect.Right - radius, rect.Bottom - radius,
radius, radius, 0, 90);
                    path.AddArc(rect.X, rect.Bottom - radius, radius, radius, 90,
90);
                    path.CloseFigure();

                    bubble.Region = new Region(path);
                    e.Graphics.SmoothingMode = SmoothingMode.AntiAlias;
                    using (Pen pen = new Pen(Color.FromArgb(245, 246, 250), 5))
                    {
                        e.Graphics.DrawPath(pen, path);
                    }
                }
            };

            // write all the text
            Label lblTitle = new Label();
            lblTitle.Text = title;
            lblTitle.Font = new Font("Segoe UI", 15, FontStyle.Bold);
            lblTitle.AutoSize = true;
            Label lblDate = new Label();
            lblDate.Text = $"Date: {dateSet:dd MMM yyyy}";
            lblDate.Font = new Font("Segoe UI", 12, FontStyle.Regular);
            lblDate.AutoSize = true;
            Label lblMarks = new Label();
            lblMarks.Text = $"Total Marks: {totalMarks}";
            lblMarks.Font = new Font("Segoe UI", 12, FontStyle.Regular);
            lblMarks.AutoSize = true;

            FlowLayoutPanel FlowLayout = new FlowLayoutPanel(); // this controls
how the info in the bubble is layed out
            FlowLayout.FlowDirection = FlowDirection.TopDown;
            FlowLayout.AutoSize = false;
            FlowLayout.Dock = DockStyle.Fill;
            FlowLayout.WrapContents = false;
            FlowLayout.Controls.Add(lblTitle);
            FlowLayout.Controls.Add(lblDate);
            FlowLayout.Controls.Add(lblMarks);

            bubble.Controls.Add(FlowLayout); // adds the text inside the bubble
            bubble.MaximumSize = new Size(RecentAsignmentsPanel.ClientSize.Width
- 50, 999);
            return bubble;
        }

        private Panel CreateResultsBubble(string title, DateTime dateSet, int
actualMarks, int totalMarks, string name) // using similar code to the help page
bubbles
```

```
        {
            // This method creates a bubble which displays a result from a
student on a recently set assignment

            Panel bubble = new Panel();
            bubble.AutoSize = false;
            bubble.BackColor = Color.FromArgb(39, 60, 117);
            bubble.ForeColor = Color.FromArgb(245, 246, 250);
            bubble.BorderStyle = BorderStyle.None;
            bubble.Margin = new Padding(10);
            bubble.Padding = new Padding(15);

            bubble.Paint += (s, e) => // use paint to create the rounded corners
            {
                using (GraphicsPath path = new GraphicsPath())
                {
                    int radius = 20; // controlling how round the cornersare
                    Rectangle rect = bubble.ClientRectangle;
                    path.AddArc(rect.X, rect.Y, radius, radius, 180, 90);
                    path.AddArc(rect.Right - radius, rect.Y, radius, radius, 270,
90);
                    path.AddArc(rect.Right - radius, rect.Bottom - radius,
radius, radius, 0, 90);
                    path.AddArc(rect.X, rect.Bottom - radius, radius, radius, 90,
90);
                    path.CloseFigure();

                    bubble.Region = new Region(path);
                    e.Graphics.SmoothingMode = SmoothingMode.AntiAlias;
                    using (Pen pen = new Pen(Color.FromArgb(245, 246, 250), 5))
                    {
                        e.Graphics.DrawPath(pen, path);
                    }
                }
            };

            // write all the text
            Label lblTitle = new Label();
            lblTitle.Text = $"{name} : {dateSet:dd MMM yyyy}";
            lblTitle.Font = new Font("Segoe UI", 10, FontStyle.Bold);
            lblTitle.AutoSize = true;
            Label lblDate = new Label();
            lblDate.Text = $"{title}";
            lblDate.Font = new Font("Segoe UI", 9, FontStyle.Regular);
            lblDate.AutoSize = true;
            Label lblMarks = new Label();
            lblMarks.Text = $"Marks: {actualMarks}/{totalMarks}";
            lblMarks.Font = new Font("Segoe UI", 8, FontStyle.Regular);
            lblMarks.AutoSize = true;

            FlowLayoutPanel FlowLayout = new FlowLayoutPanel(); // this controls
how the info in the bubble is layed out
            FlowLayout.FlowDirection = FlowDirection.TopDown;
            FlowLayout.AutoSize = false;
            FlowLayout.Dock = DockStyle.Fill;
            FlowLayout.WrapContents = false;
            FlowLayout.Controls.Add(lblTitle);
            FlowLayout.Controls.Add(lblDate);
            FlowLayout.Controls.Add(lblMarks);

            bubble.Controls.Add(FlowLayout); // adds the text inside the bubble
            bubble.MaximumSize = new Size(RecentResultsPanel.ClientSize.Width -
50, 999);
```

```
            return bubble;
        }

        private Button InitalizeBacktoDashBtn()
        {
            // This method creates the back to dashboard button

            Button btn = new Button();
            btn.Text = "Back";
            btn.Size = new Size(150, 40);
            btn.BackColor = Color.FromArgb(47, 54, 64);
            btn.ForeColor = Color.FromArgb(220, 221, 225);
            btn.Location = new Point(
                this.ClientSize.Width - btn.Width - 10,
                this.ClientSize.Height - btn.Height - 10); // bottom right corner
            Controls.Add(btn);
            btn.Anchor = AnchorStyles.Bottom | AnchorStyles.Right;
            btn.Click += BackToDashboardbtn_Click;
            return btn;
        }

        private void BackToDashboardbtn_Click(object? sender, EventArgs e)
        {
            // This method is assigned to the press of the back to dashboard
button
            // it simply closes the current form and opens the dashboard

            GeneralDashboardForm dashboard = new
GeneralDashboardForm(currentUser);
            dashboard.Show();
            this.Hide();
        }

        private void SetAssignments_Click(object sender, EventArgs e)
        {
            // This method opens up the SetAssignment window over the top of the
teacher dashboard

            SetAssignments assignments = new SetAssignments(currentUser, this);
            assignments.Show();
        }
    }
}
```

Log in & Sign up

LoginForm.cs

```
using CryptographyTeachingTool.Cryptography;
using CryptographyTeachingTool.Cryptography.Hash;
using CryptographyTeachingTool.Data.Repositories;
using CryptographyTeachingTool.MainClasses;
using CryptographyTeachingTool.UI.Basic;
using Microsoft.Data.SqlClient;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Runtime.Intrinsics.Arm;
using System.Security.Cryptography;
using System.Text;
```

```
using System.Threading.Tasks;
using System.Windows.Forms;

namespace CryptographyTeachingTool.Login
{
    public partial class LoginForm : Form
    {

        public LoginForm()
        {
            InitializeComponent();
        }

        private void SignInButtonClick(object sender, EventArgs e)
        {
            // This method takes the username and password, validates them,
            // hashes the password and calls the GetUser method in the UserRepo
            // to see if a user exists with these correct details.

            string username = usernameBox.Text;
            string inputPass = passwordBox.Text;
            if (username != "" && inputPass != "")
            {
                SHA256Hash SHA = new SHA256Hash();
                string hashedPass = SHA.Hash(inputPass);

                User currentUser = UserRepo.GetUser(username, hashedPass);
                if (currentUser == null)
                {
                    // This branch is only reached if there is an issue with the
database beyond the user's details being incorrect
                    // Messages for issues are displayed inside the GetUser
method
                    passwordBox.Text = ""; // This resets the password box
                }
                else
                {
                    GeneralDashboardForm dashboardForm = new
GeneralDashboardForm(currentUser);
                    dashboardForm.Show();
                    this.Hide();
                }

            }
        }

        private void SignUpLabelClick(object sender,
LinkLabelLinkClickedEventArgs e)
        {
            // This method opens up the sign up window and hides the current
login page

            SignUpForm signupForm = new SignUpForm(this);
            signupForm.Show();
            this.Hide();
        }

    }
}
```

SignUpForm.cs

```
using System;
```

```
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using Microsoft.Data.SqlClient;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.ComponentModel.Design.ObjectSelectorEditor;
using static System.Windows.Forms.VisualStyles.VisualStyleElement.StartPanel;
using CryptographyTeachingTool;
using CryptographyTeachingTool.Cryptography;
using CryptographyTeachingTool.Data.Repositories;
using CryptographyTeachingTool.Cryptography.Hash;

namespace CryptographyTeachingTool.Login
{
    public partial class SignUpForm : Form
    {
        private LoginForm loginSender;

        private int NUMBER_LOWER_LIMIT = 48;
        private int NUMBER_UPPER_LIMIT = 57;
        private int UPPERCASE_LOWER_LIMIT = 65;
        private int UPPERCASE_UPPER_LIMIT = 90;
        private int LOWERCASE_LOWER_LIMIT = 97;
        private int LOWERCASE_UPPER_LIMIT = 122;

        private int MIN_PASS_LENGTH = 8;
        private int MIN_UPPER_COUNT = 1;
        private int MIN_LOWER_COUNT = 1;
        private int MIN_NUMB_COUNT = 1;
        private int MIN_SPECIAL_COUNT = 1;

        public SignUpForm(LoginForm loginForm)
        {
            loginSender = loginForm;
            InitializeComponent();
            this.FormClosed += OnFormClosed; // this ensures that if the form is
closed via the red cross the login page is returned to
        }

        private void OnFormClosed(object? sender, FormClosedEventArgs e)
        {
            loginSender.Show();
        }

        private void SignUpButtonClick(object sender, EventArgs e)
        {
            // This method takes all the user's input, validates it
            // (including calling the UserExists method as part of the
            // UserRepo class to check the username is unique) and then if
suitable
            // an entry to the Users database is created with the added details.
            // Note the password is also validated as part of this window.

            string fname = firstnameBox.Text;
            string sname = surnameBox.Text;
            string uname = usernameBox.Text;
            SHA256Hash SHA = new SHA256Hash();
            string hashedPass = SHA.Hash(Password.Text);
            string role = roleSelect.Text;
```

```
            // validation
            bool isValid = true;

            int upperCount = 0;
            int lowerCount = 0;
            int numbCount = 0;
            int specialCount = 0;
            foreach (char c in Password.Text) // This counts and checks each
character to ensures that each password meets the correct criterion
            {
                if (Convert.ToInt16(c) >= NUMBER_LOWER_LIMIT &&
Convert.ToInt16(c) <= NUMBER_UPPER_LIMIT)
                {
                    numbCount++;
                }
                else if (Convert.ToInt16(c) >= UPPERCASE_LOWER_LIMIT &&
Convert.ToInt16(c) <= UPPERCASE_UPPER_LIMIT)
                {
                    upperCount++;
                }
                else if (Convert.ToInt16(c) >= LOWERCASE_LOWER_LIMIT &&
Convert.ToInt16(c) <= LOWERCASE_UPPER_LIMIT)
                {
                    lowerCount++;
                }
                else
                {
                    specialCount++;
                }
            }

            if (fname == "" || sname == "" || uname == "" || Password.Text == "")
            {
                MessageBox.Show("Please fill in all details before submitting,
none should be blank.");
                isValid = false;
            }
            // If the counts do no match the criterion then the data is not
stored and a message is displayed explaining why
            else if (Password.Text.Length < MIN_PASS_LENGTH) {
MessageBox.Show("Your password must be at least 8 characters in length"); isValid
= false; }
            else if (upperCount < MIN_UPPER_COUNT) { MessageBox.Show("Your
password must have an uppercase letter in"); isValid = false; }
            else if (lowerCount < MIN_LOWER_COUNT) { MessageBox.Show("Your
password must have a lowercase letter in"); isValid = false; }
            else if (numbCount < MIN_NUMB_COUNT) { MessageBox.Show("Your password
must have a number in"); isValid = false; }
            else if (specialCount < MIN_SPECIAL_COUNT) { MessageBox.Show("Your
password must have a special character in"); isValid = false; }

            // storing the data
            if (isValid)
            {
                if (!UserRepo.UserExists(uname))
                {
                    if (UserRepo.CreateUser(uname, hashedPass, fname, sname,
role)) // this returns true if the user has been sucessfully created
                    {
// any errors are displayed as part of the CreateUser method
                        loginSender.Show();
                        this.Close();
```

```
                    }
                }
                else
                {
                    MessageBox.Show("The username you have entered is already in
use");
                }
            }
        }
    }
}
```

Help Page

ChatBot.cs

```
using OpenAI.Chat;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net.NetworkInformation;
using System.Text;
using System.Threading;
using System.Threading.Tasks;
using static System.Runtime.InteropServices.JavaScript.JSType;

namespace CryptographyTeachingTool.Help
{
    internal class ChatBot
    {
        /// <summary>
        /// This class will be used for making calls to OpenAI's LLM API. When
creating an instance
        /// of the object a connection will be established using a securely
stored API key, then the getResponse method
        /// may be called with the query as an input.
        /// This class will be defined as an async class, which will mean that
the thread will not freeze during the API call.
        /// </summary>

        private ChatClient chatClient; // This attribute stores OpenAI's
ChatClient object (which serves as
                                       // a link between their LLM and my code)

        // The WELCOME_MESSAGE constant defines what is to be displayed on the
user's screen as they first open the application.
        private string WELCOME_MESSAGE = "Hello, Welcome to the help page. I am
an expert in all things cryptography related. " +
            "Please ask any questions you have here and I will answer them. " +
            "I can also suggest articles or YouTube videos to watch - just ask!";

        // This PATH directs the program to the location of the txt file with the
API key inside.
        // It serves as a method of securely storing the key.
        private string PATH = Path.Combine(
            AppDomain.CurrentDomain.BaseDirectory,
            "Help Page",
            "APIKey.txt"
        );

        public ChatBot()
        {
```

```
            // This constructor is used to initialize the chat bot and create a
ChatClient object,
            // as per OpenAI's API.
            string apiKey = GetAPIKey();
            chatClient = new(model: "gpt-5-nano", apiKey: apiKey); // this
creates a connection with the cheapest model (being gpt5nano)
        }

        public string GetWelcomeMessage()
        {
            return WELCOME_MESSAGE;
        }
        private string GetAPIKey()
        {
            // This method is used to securely get the api key which is securely
stored in
            // a given path. The key is not stored as plain text in a variable
because this
            // could allow others to get hold of and abuse the key.
            using (StreamReader sr = new StreamReader(PATH, false))
            {
                string line = sr.ReadLine();
                if (line == null) { return ""; }
                else { return line; }
            }
        }

        public async Task<string> GetResponse(string input)
        {
            // This method creates an api call to obtain the LLM's response,
given a message as input.
            // It has been defined as async because it prevents the thread from
being frozen during api calls.
            // This is cruital as API calls to this particual service can be slow
and computationally intesive.

            if (input == "") { return "You did not say anything"; }
            var result = await chatClient.CompleteChatAsync(input); // this gets
the response with await preventing the thread being frozen during the process

            var completion = result.Value;
            string reply = "(no reply)";
            if (completion.Content != null && completion.Content.Count > 0) //
checks the reponse
            {
                reply = completion.Content[0].Text;
            }

            return reply;
        }
    }

}
```

HelpForm.cs

```
using Azure;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
```

```
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace CryptographyTeachingTool.Help
{
    public partial class HelpForm : Form
    {
        private ChatBot bot;
        public HelpForm()
        {
            InitializeComponent();
            bot = new ChatBot(); // creates the help bot object
            flowLayoutPanel.Controls.Add(CreateBubble(bot.GetWelcomeMessage(),
false)); // add the welcome message to the screen
        }

        private async void SendButtonClick(object sender, EventArgs e) // async
because it prevents the thread from being frozen during api calls
        {
            // This async method calls the GetResponse method of the ChatBot
class
            // it also displays the users input and the reply from the LLM
            string input = userInput.Text;
            userInput.Text = ""; // clears the text box

            flowLayoutPanel.Controls.Add(CreateBubble(input, true)); // add the
message to the screen
            string response = await bot.GetResponse(input); // get the reponse
            flowLayoutPanel.Controls.Add(CreateBubble(response, false)); // add
the response to the screen
        }

        private Panel CreateBubble(string message, bool isUserMessage)
        {
            // This method is used to create display any text message.
            // It uses blue for the user's messages and grey for the bot's

            Label label = new Label();
            label.Text = message;
            label.AutoSize = true;
            label.MaximumSize = new Size(400, 0);
            if (isUserMessage) { label.BackColor = Color.LightBlue; }
            else { label.BackColor = Color.LightGray; }
            label.Padding = new Padding(10);
            label.Margin = new Padding(5);
            label.Font = new Font("Segoe UI", 10);

            Panel bubble = new Panel();
            bubble.AutoSize = true;
            bubble.Controls.Add(label);
            bubble.Dock = DockStyle.Top;

            return bubble;
        }
    }
}
```

Program.cs

```
using CryptographyTeachingTool.Login;

namespace CryptographyTeachingTool
{
    internal static class Program
    {
        /// <summary>
        ///  The main entry point for the application.
        /// </summary>

        [STAThread]
        static void Main()
        {
            ApplicationConfiguration.Initialize();
            Application.Run(new LoginForm());
        }

    }
}
```

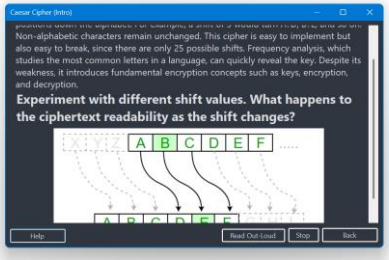
Testing

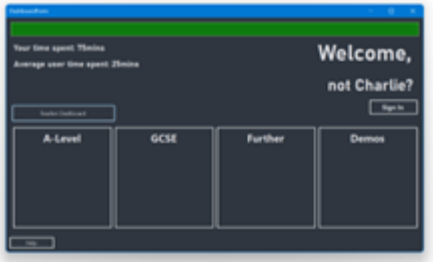
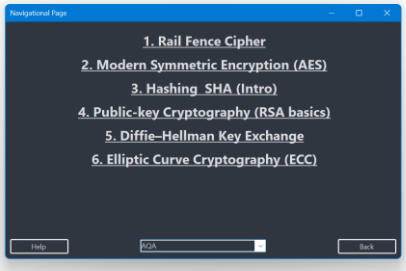
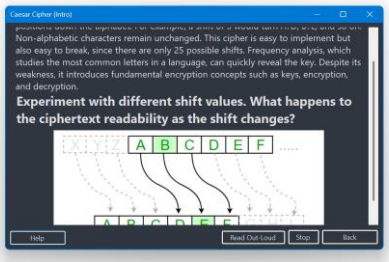
In the following section, I will be evaluating my program against the objectives I created in the design section. In addition to these, I will also be looking for data security and robust programming.

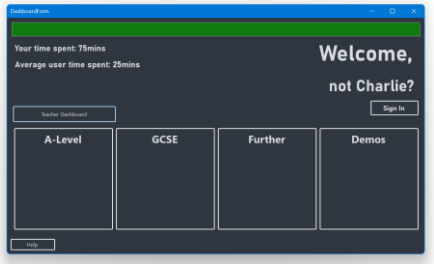
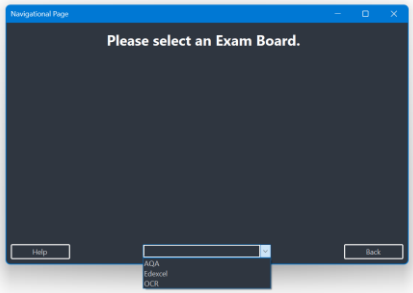
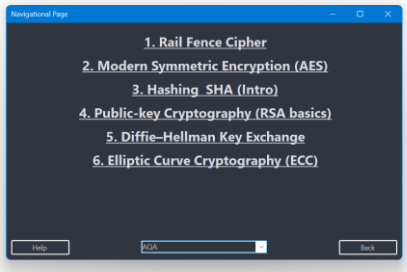
The video associated with all these tests can be found on the following link:
https://www.youtube.com/watch?v=22pPTWr38Ak

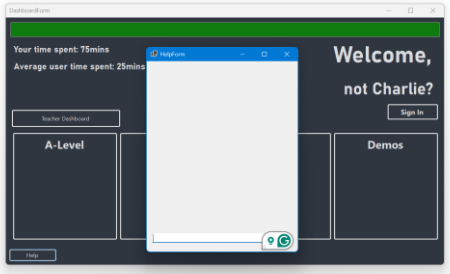
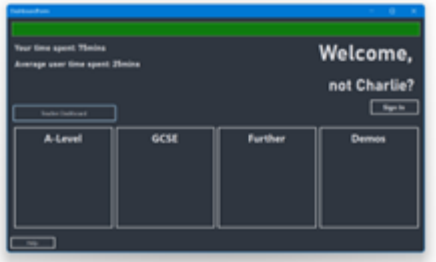
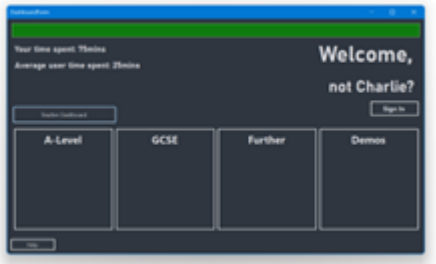
The video showcases each test in more depth and detail. Note that the passwords used in the video were manually added to the database and would not pass the validations, they are short and simple for ease of testing. These testing users would be removed before deployment.

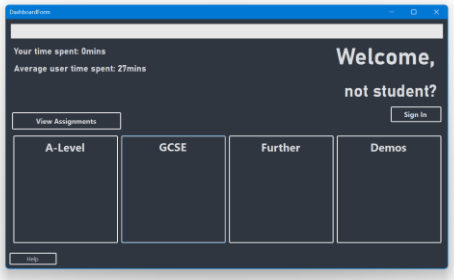
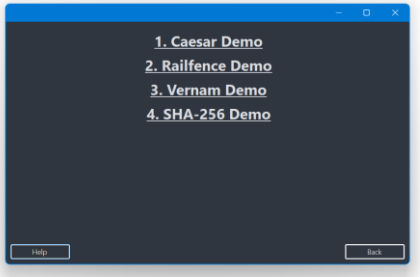
Generic and Robustness Tests

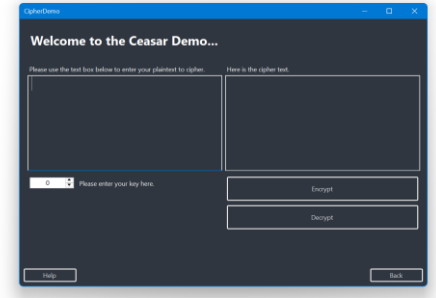
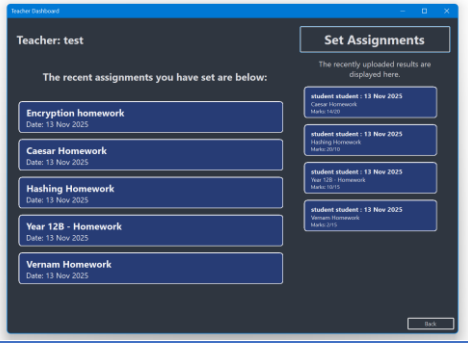
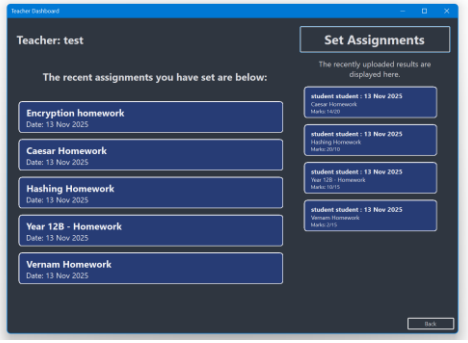
Test No.		User interface	Classes	Reason for test:	Data used:	Expected result:	Objectives met:	Evidence:	Pass / Fail:
1	OVERVIEW OBJECTIVE	Lesson Pages & Teaching Tools	Lesson Page	This application must cater for all students – no matter how they learn.	A student's login details.	Objectives 6.2.1-4 should be met, with lessons containing the following: - Images to aid visual learners - A text-to-speech system (includes start/stop) - A help button for the chatbot - A work prompt	1.1, 6.2.1, 6.2.2, 6.2.3, 6.2.4	 See objectives 6.2.1-4 for actual tests.	Pass

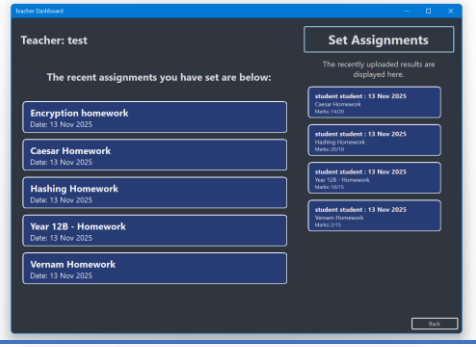
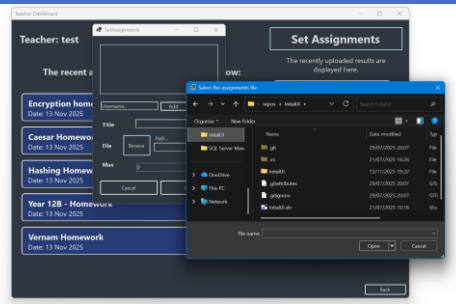
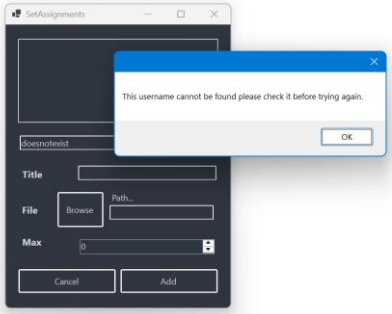
2		Dashboard & Lesson Selection	General Dashboard & Navigational Pages Template	It can be easy to quickly create a UI without thinking about what the user really needs to see. A minimalistic and logical approach will prevent this.	A teacher's and a student's login details. Plus, various exam board configurations.	The application should be easy and intuitive to use, with the inappropriate information being hidden (abstracted) from the user.	1.2, 1.3	 	**Pass**
3		Lesson Pages	Lesson Page & Navigational Pages Template	Both teachers and students should be able to access the lesson pages. It may be useful for teachers to look through the information before getting students to read through it, or perhaps the teacher may find a	A teacher's login details.	Teachers should be able to access the lesson library and view lessons in the same way as students.	1.4		**Pass**

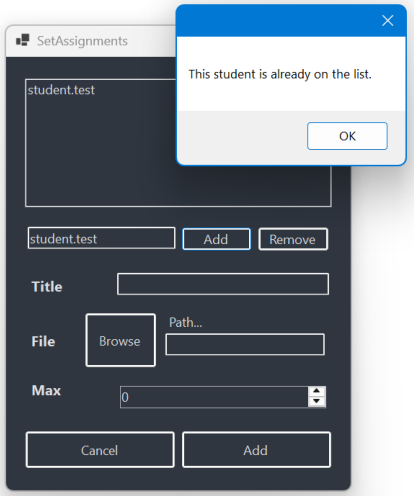
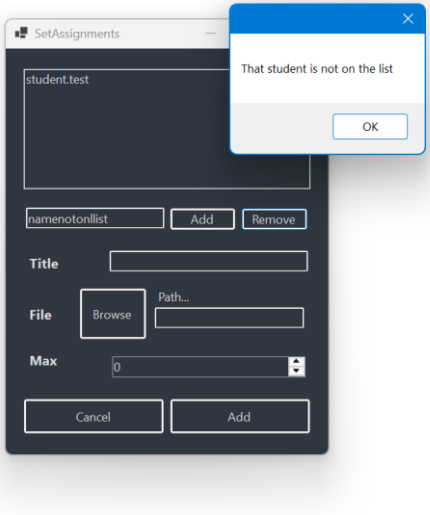
	GENERAL DASHBOARD			diagram/visual of use for in-lesson display.					
4		Progress bar	General Dashboard Form	A user can fill the progress bar, because it is simply a display of them compared to the average user – where the max is double the time of an average user.	I created a user with a time spent of greater than 2x the average, to see if the progress bar is filled to a suitable level.	I expected the progress bar to fill to the max level without any errors.	2.1		**Pass**
5		Exam boards	General Dashboard Form	It is important that any user type can quickly find their appropriate syllabus.	A teacher's login details (to view the general dashboard).	The exam board should be listed, under the appropriate level.	2.2	 	**Pass**

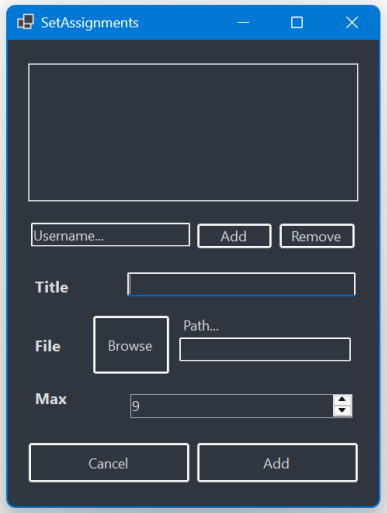
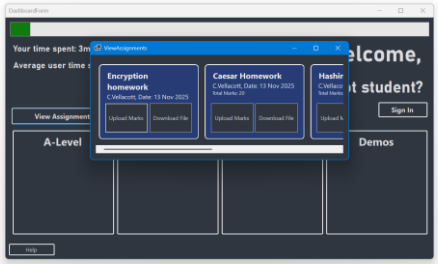
6		Help button	Base Page Forms	The dashboard is one of the most appropriate places for a user to find the help button, thus must be visible and work here.	A teacher's login details (to view the general dashboard).	The buttons should open the help page correctly and be placed in the bottom left corner.	2.3		**Pass**
7		Login message	General Dashboard Form	This is essential so that an individual can see who is currently logged in, because sharing resources could result in someone not being logged into the correct account.	A teacher's login details (to view the general dashboard).	The name and message should be displayed just below the progress bar, in a clear, easy-to-read font.	2.4		**Pass**
8		Teacher dashboard and View assignments button	General Dashboard Form	For a teacher to set assignments, they must be able to access the teacher dashboard.	A teacher and a student's login details (to view the general dashboard).	The teacher dashboard button should only be visible to the teacher and not the student, and likewise with the student's assignment button.	2.5, 3.3, 4.1		**Pass**

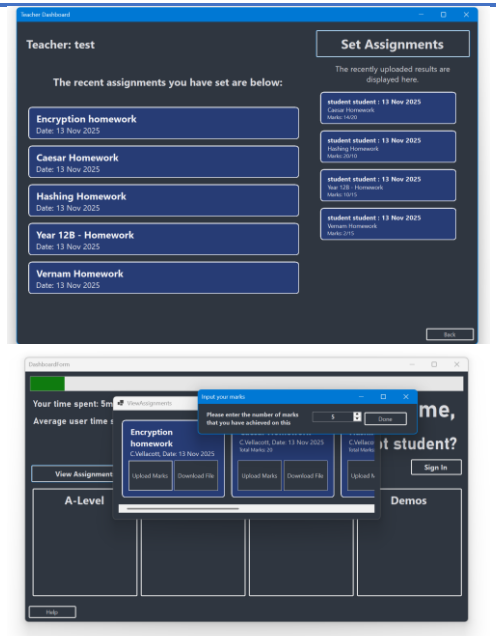
									
9		Demos button	General Dashboard Form	Any user must be able to access these features.	A teacher's and a student's login details (to view the general dashboard).	Any user should also be able to locate the encryption tools demos.	2.6	See the demos button in both above… 	**Pass**

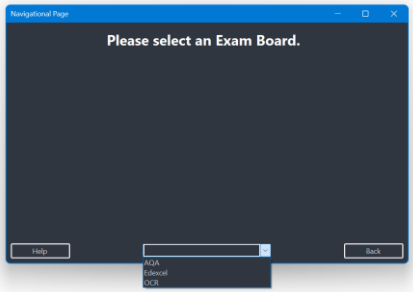
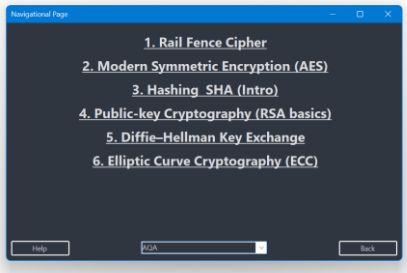
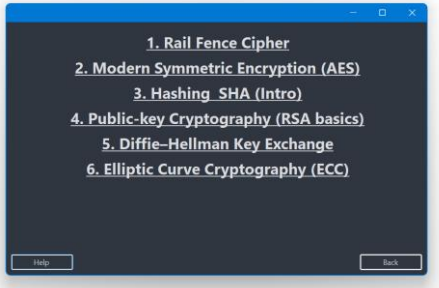
10	**TEACHER DASHBOARD**	All UI	Base Page Forms	Most teachers will use the demo on an interactive whiteboard thus this is crucial for teachers to be able to use this application.	A teacher and student's login details (to view all windows).	Buttons will be suitable for an interactive whiteboard.	2.7	 	**Pass**
11		Assignment bubbles	Teacher Dashboard	If a teacher cannot view the assignments, it will mean they will have to keep track of things on their own (when the application should do this).	A teacher's account which has assignments set on.	All assignments should be visible with information such as the date set and title.	3.1, 3.2, 3.3, 3.4,		**Pass**

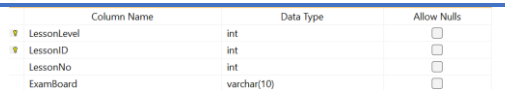
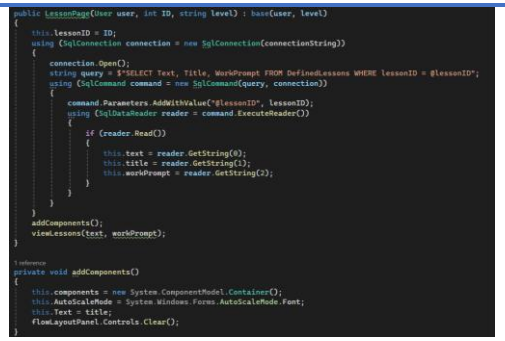
12	**ASSIGNMENT PAGES**	Assignment marks bubbles	Teacher Dashboard	A teacher should be able to track a student's progress. This can be done by displaying their marks to the teacher.	A teacher's account which has assignments set on.	The bubbles should be visible and contain the student's name, mark and the assignment title.	3.5		**Pass**
13		File browser and Assignment setting window	Assignment Setter	To set a task, the teacher must be able to send the work. This application allows for that to happen.	A teacher's login details plus a file to upload.	The upload browser should be opened after pressing the upload button and the files path stored in the path textbox. This file's name and binary data should then be stored on the database.	4.2		**Pass**
14		Student selection	Assignment Creator and User	The teacher must be able to select students based on username in a simple manner.	A teacher's login details, plus valid and invalid students' usernames.	The students' usernames should be validated when they are added; if they don't exist, the teacher must be told.	4.3,4.4		**Pass**

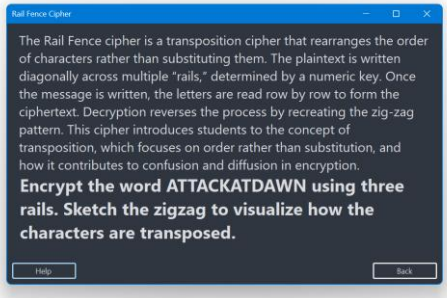
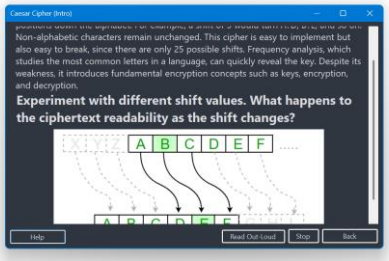
15		Student Selection	Assignment Creator and User	Measures should be taken to prevent the task being set twice for one individual; therefore, if the teacher enters a user's name twice, they should be told, and the name should only be added once.	A teacher's login details plus valid students' usernames.	If a username is mentioned more than once on the table, then the teacher should be made aware of it, and the name should not be added.	4.5		**Pass**
16		Student Selection	Assignment Creator and User	It must be possible for teachers to remove students if they are accidentally added to a task. Appropriate validation must occur so that if a teacher tries to remove a student who is not on the list they are told so.	A teacher's login details plus valid students' usernames.	The remove button should function as mentioned, and then a message should pop up to tell the teacher they tried to remove a username not on the list.	4.6		**Pass**

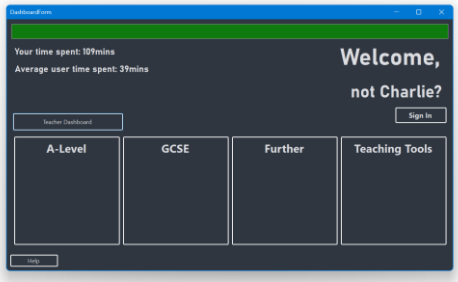
17		Assignment creation window	Assignment Creator and User	Along with the task it is appropriate to set the max number of marks that a student can get on the assignment.	A teacher's login details.	A numeric selector element should appear which allows the teacher to select the max marks and then once submitted this correct value should be placed into the database.	4.7		**Pass**
18		Assignment viewing window	Assignment Viewer	A student should be able to see all the tasks that have been set for them. If this is not the case, the assignment section of this application becomes redundant. Part of the viewing process is downloading	A student's login details.	A window will open for the student that has each recent assignment stored in a bubble with buttons such as uploading marks and downloading the file inside it.	4.8, 4.9		**Pass**

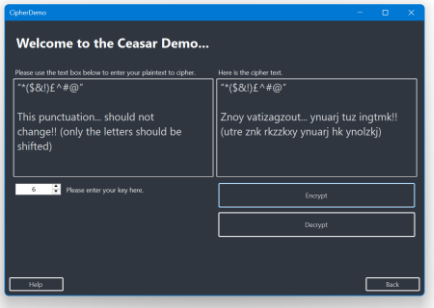
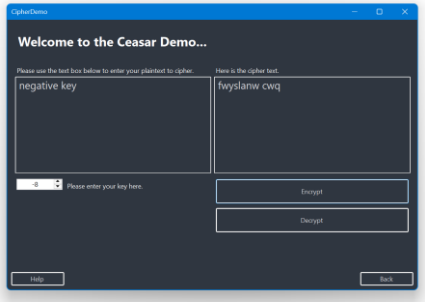
				the file to a suitable location; this is an error-prone process, so the result must be checked.					
19		Uploading marks window	Upload Marks	It must be so that a user can upload their marks without an error caused by the SQL code. In addition, users should be prompted if their inputted marks are greater than the maximum marks.	A teacher's login details.	The uploading marks window will open after pressing the upload marks button on the assignment window. On this page, the student should be able to select their marks and then for the teacher to be able to see that same number.	4.9, 4.10		**Pass**

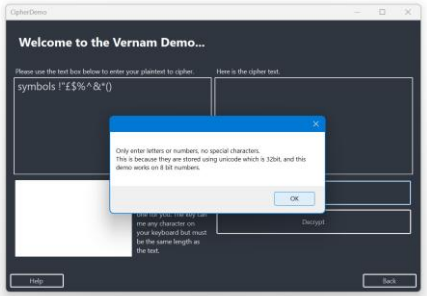
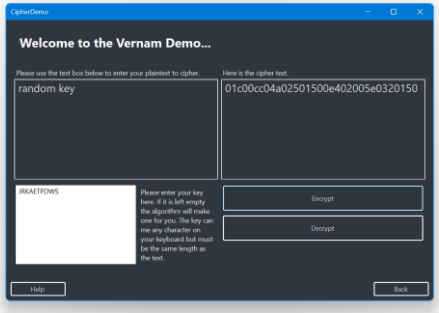
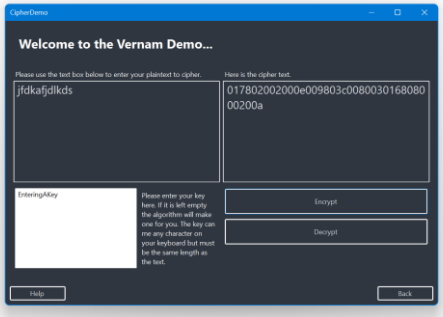
20	**LESSON LIBRARY**	Intuitive selection process	Lesson Page	Users with a range of technical abilities will use this application, so it must be such that the student or teacher can easily and intuitively find the correct content. Thus, I will test for a separation of levels, then exam boards and then lessons.	Any user's login details.	The levels, exam boards and lessons should be visually or physically separated through different windows, buttons and overall page design.	5.1, 5.5	 	**Pass**
21	**LESSON LIBRARY**	Intuitive lesson ordering	Lesson Page	A check that all lessons logically flow so that no knowledge is assumed must occur. In addition, a logical numbering and titling system must be present. Users must be able to see both when	Any user's login details.	Lessons should have numbers that follow in a logical way without assuming knowledge.	5.2, 5.3, 5.4		**Pass**

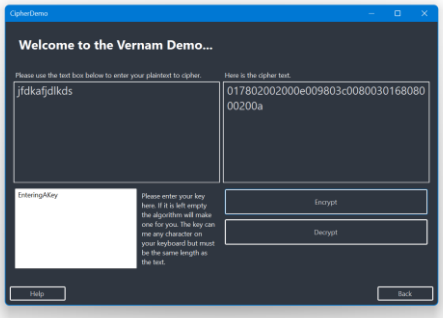
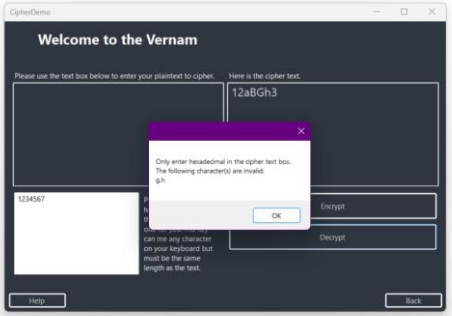
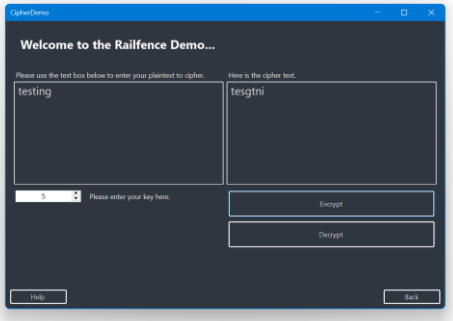
				selecting a lesson.					
22		Database design and Lesson content	Lesson Page	The content must not be duplicated; instead, it must be filtered each time.	Any user's login details.	The content should be stored in a fully normalized database so that individual lessons can be accessed by multiple syllabi.	5.6		**Pass**
23		Database design, Lesson content and Page Function	Lesson Page	All lessons must be generated dynamically, to save time with development and reduce the storage required to install this application *(this is important as at schools' storage and resources are limited by cost)*.	Any user's login details.	The lessons will be loaded from the database.	5.7		**Pass**

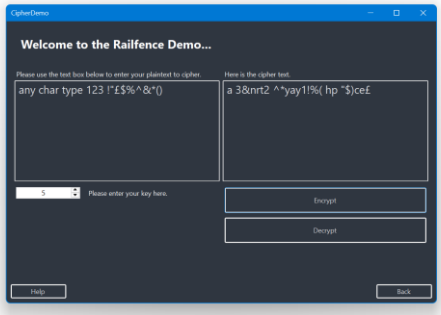
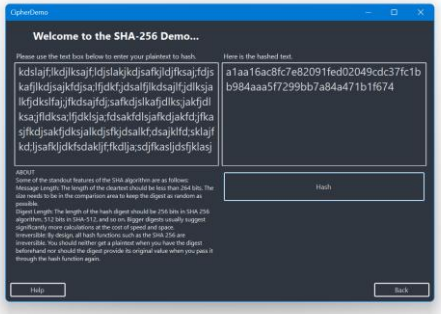
24	**THE IN-LESSON SCREEN**	Lesson content	Lesson Page	It is important to ensure that all information in the lessons is correct and concise (below my 200-word criteria).	Any user's login details.	All lessons should be below the 200-word criterion.	6.1	 The above image contains 99 words and is one of the longer lessons. It is clear from this and other tests that all lessons contain less than 200 words.	**Pass**
25		The display of the lesson information	Lesson Page	The lesson information must be displayed in a manner that is appropriate for visual learners.	Any user's login details.	Lessons must contain images to aid visual learners.	6.2.1		**Pass**
26		The display of the lesson information	Lesson Page	The lesson information must be displayed in a manner that is appropriate for auditory learners.	Any user's login details.	Lessons must contain a button which will read through the text when pressed.	6.2.2	See the buttons above…	**Pass**
27		The display of the	Lesson Page	The lesson information must be	Any user's login details.	Lessons must contain a help button which	6.2.3	See help button on the above…	**Pass**

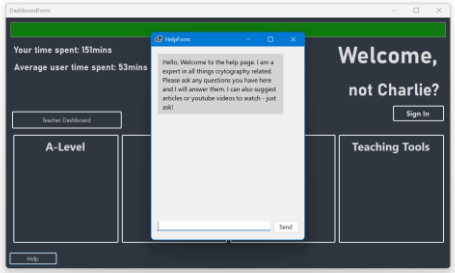
		lesson information		displayed in a manner that is appropriate for practical learners.		opens a window where they can talk to a chatbot.			
28		The display of the lesson information	Lesson Page	The lesson information must be displayed in a manner that is appropriate for social learners.	Any user's login details.	Lessons must contain a work prompt.	6.2.4	See the work prompt in bold above…	**Pass**
29		The help button	Base Forms Page	It must be possible to access the help and other navigational buttons on all pages.	Any user's login details.	The help button should be a stock part of almost all pages.	6.3, 7.7	See all screenshots – all pages have a link to the help window in the bottom left corner.	**Pass**
29		Text size	Base Forms Page and others	The text and content must be appropriately sized to ensure readability.	Any user's login details.	Text size will be appropriate throughout.	6.4	See all screenshots.	**Pass**
30		The teaching tools button	The General Dashboard	This button must be accessible to all users – not just the teacher. It should open a page which contains a list of	Any user's login details.	The tools should be functional, easily accessible, and clear to use.	7.1		**Pass**

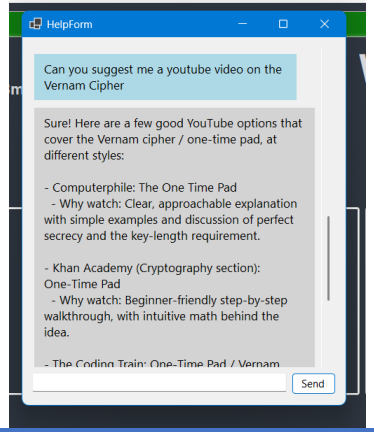
				encryption techniques.					
31	**THE TEACHING DEMONSTRATIONS**	The Caesar Cipher's function	Caesar Demo Page and CaesarCipher	The Caesar Cipher should be able to accept a string that contains non-alphabetic characters. In addition, it must be able to process it appropriately (as specified).	Any string that contains a range of non-alphabetic characters eg) "*($&!)£^#@"	If the user enters a non-alphabetical character, it must not have a shift applied to it; the punctuation can stay in place.	7.2.1, 7.2.2		**Pass**
32		The Caesar Cipher's function	Caesar Demo Page and CaesarCipher	Negative keys must be accounted for.	A negative key text pair with a known encrypted result.	Encrypting with negative keys should be the same as applying a decrypt with the positive version of that key.	7.2.4		**Pass**

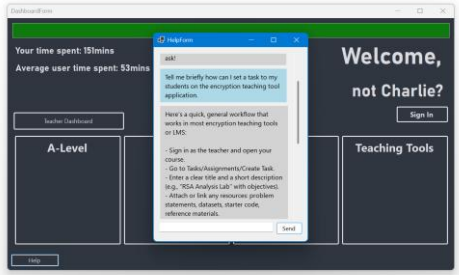
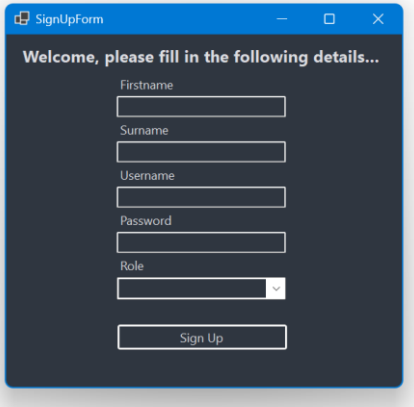
33		The Vernam Cipher's function	Vernam Demo Page and VernamCipher	The cipher must be able to accept textual data with any non-alphanumeric symbols (with an encoding value below 127) and produce the correct result.	Plain text and key pairs with known results from the encryption.	Any textual data that does not have any non-8-bit characters must not be accepted.	7.3.1		**Pass**
34		The Vernam Cipher's function	Vernam Demo Page and VernamCipher	The key is harder to create for this encryption algorithm – therefore the application should be able to create and use randomly generated key.	Some random plain text (the text used is not important).	If left blank by the user, the key will be randomly created each time but must only contain 1s and 0s. The user should be made aware of the key (this key must be displayed).	7.3.2		**Pass**
35		The Vernam Cipher's function	Vernam Demo Page and VernamCipher	The teacher or user must be able to see how the key effects the result, thus should be able to enter a key.	Some plain text and key pairs.	The user must be able to input a key using a text box. Appropriate validation must occur to ensure it is of the correct length.	7.3.3		**Pass**

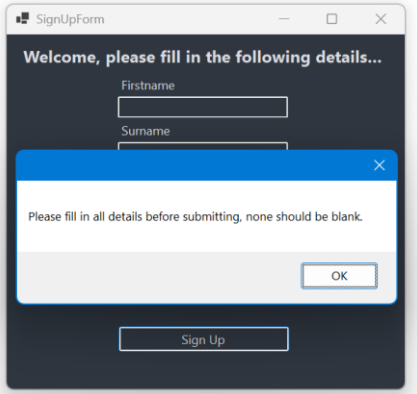
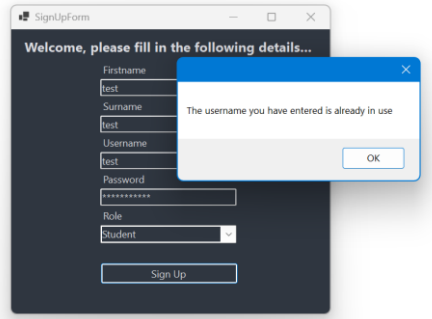
36		The Vernam Cipher's function	Vernam Demo Page and VernamCipher	This cipher returns binary data which can only be displayed using hexadecimal.	Any key and plaintext pair.	The result should be displayed in hexadecimal.	7.3.4		**Pass**
37		The Vernam Cipher's decrypt function	Vernam Demo Page and VernamCipher	Due to a lack of understanding the user may enter a non-hexadecimal value into the cipher text box and press the decrypt button.	A non-hexadecimal cipher text.	The user should be told which characters break the rule and all uppercase chars should be converted to lowercase.	7.3.5		**Pass**
38		The Railfence Cipher's function	Railfence Demo Page and RailfenceCipher	It must not be possible for the users to enter a non-integer key. In addition, the user should not be able to enter a negative key because this is not possible in railfence.	A non-numeric key and a negative key.	The user should be able to enter a numeric key; it should not be possible for them to enter a non-numeric or non-integer value.	7.4.1	 The numeric up down feature allowed me to add a range to the key.	**Pass**

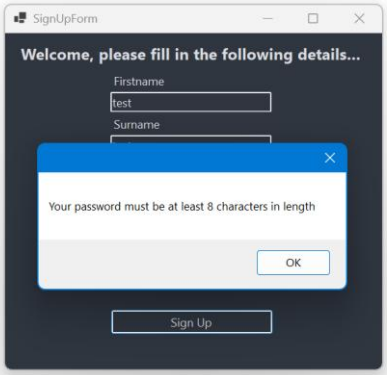
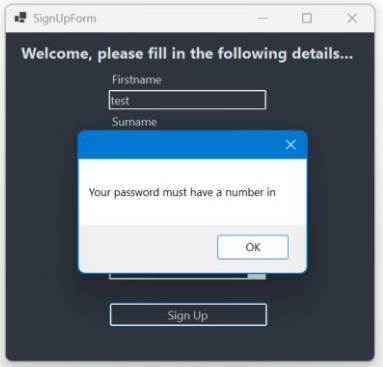
								Therefore, I ensured that it could not be below 0	
39		The Railfence Cipher's function	Railfence Demo Page and RailfenceCipher	The plain text entered should not affect the result.	Plain text with a range of alphanumeric and non-alphanumeric characters should be used, the result of the process must be known.	A string containing all character types can be entered for this cipher.	7.4.2		**Pass**
40		The SHA-256 hash's function	SHADemo and SHA256Hash	The plain text entered should not affect the result. The result must be correct.	A range of different length of plain text (extreme values should be used). A non-numeric key and know results of key text pairs.	No matter the length of the input the hash should produce a 64-digit hexadecimal. The hash result must be correct.	7.5.1		**Pass**
41		The SHA-256 hash's function	SHADemo and SHA256Hash	Hexadecimal is the best way of storing and displaying long binary data.	Some random plain text.	All hash results should be returned in hexadecimal.	7.5.2	See above…	**Pass**
42		Encryption and Decryption buttons	Any Demo Page	Each tool must allow the user to perform both an encryption and a	Any user's login details.	Buttons should be visible for this to happen.	7.6	See all encryption demos above – note there is no decrypt function for SHA therefore there is only a hash button on that page.	**Pass**

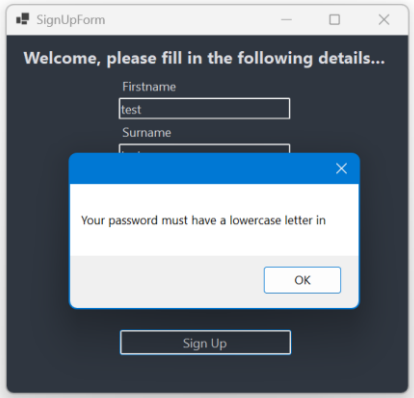
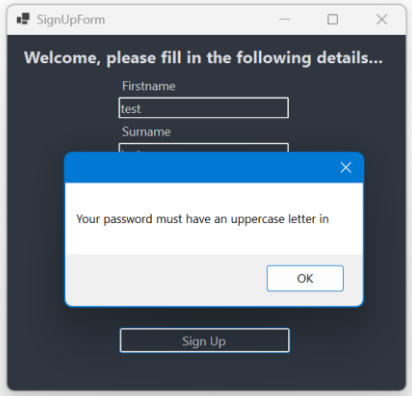
				decryption process for each cipher.					
43		The Help window	Help Page and ChatBot	The help page should pop up on top of the current page. This is because we want the lesson progress/task to be preserved and for the user to be able to see what they were doing.	Any user's login details.	This page should serve as a great resource for students to clarify their ideas and knowledge.	8.1		**Pass**
44		The Aesthetic of the Help window	Help Page	It should be closable at any time and feature a modern/sleek layout that ties in with the rest of the program. The screen must use rounded buttons, contrasting colours and a dark (eye-friendly) colour palette.	Any user's login details.	This should already be the case.	8.2	See all screenshots above…	**Pass**

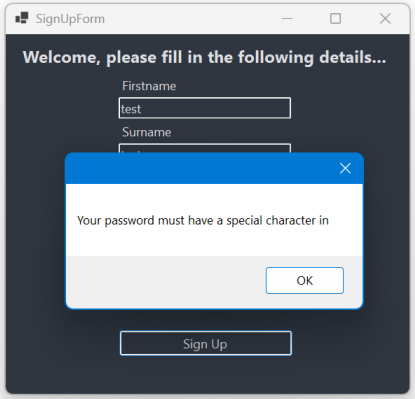
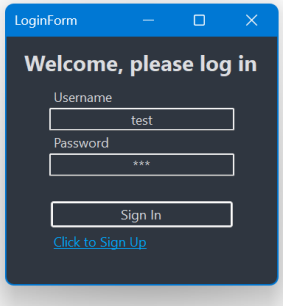
45	**HELP & EXPLAINATION SCREEN**	The functionality of the LLM	Help Page and ChatBot	We need to ensure that the help bot is used and as helpful as possible.	Any user's login details and some basic prompts.	The LLM should provide further resources, like YouTube videos. In addition, the user must be made aware of this through a message when the window opens.	8.3		**Pass**
46		The Help window	Help Page	The implementation of the help button must be simple and functional.	Any user's login details, with the page being opened from many different windows.	No matter where the help button has been pressed, it must open the same window *(without needing to provide page context to the query)*; this is because the user will likely be giving context anyway when asking the question.	8.4	This is the case, as the class for the help page is created using the button. This has been tested fully too.	**Pass**

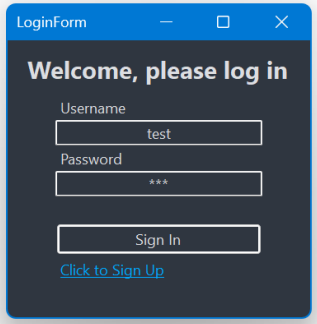
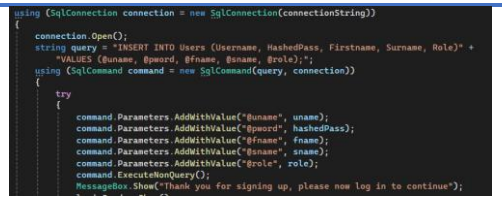
47		The functionality of the application	Help Page and ChatBot	The LLM should be given the workings of the application, so that it can help guide users around the UI.	A range of questions from both the teacher and the users' perspectives about the application.	When asked a question on how to access things like the demos, the application should return a suitable application-specific answer.	8.5		**Pass**
48		The data collected	SignUp Form	Only essential data should be collected by the application. However, sufficient levels must be collected to ensure a highly personalised user experience.	No data needed.	The user should be prompted for a username, password, role (teacher or student) first and last name.	9.1.1, 9.1.2, 9.1.3, 9.1.4, 9.1.5	 This is the bare minimum data need to be collected.	**Pass**
49		How the password is stored in the database	SignUp Form and Database	The password must not be stored in plain text, as this is poor data protection.	Any user's password must be looked up in the database.	The user's password must not be stored in plaintext; instead, a hash should be applied and then stored in the database.	9.2	The hashed password is stored in the database.	**Pass**

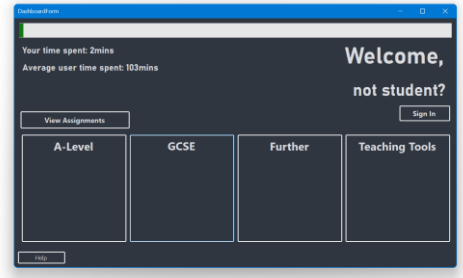
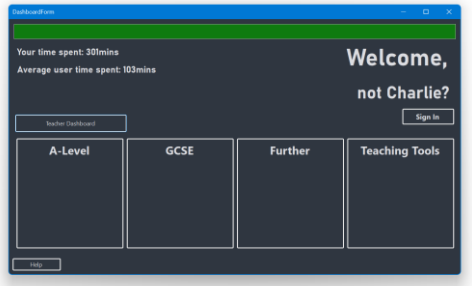
50	**LOG-IN & SIGN-UP**	Validation message box	Message Box	Validation must occur on the data that a user enters, and users must be clearly shown their mistakes. If the user were to have a non-unique username, then they would receive the same tasks as the other individual, even if they aren't meant for them. Similarly, they could end up with the same login details as someone else. This should not be able to happen.	A range of incorrect data and non-unique usernames.	All details should be stored appropriately in a database – if any information is incorrectly entered in the sign-up page, then the user must be told what the issue is. If the user has a non-unique username they should be told.	9.3.1, 9.3.2	 	**Pass**

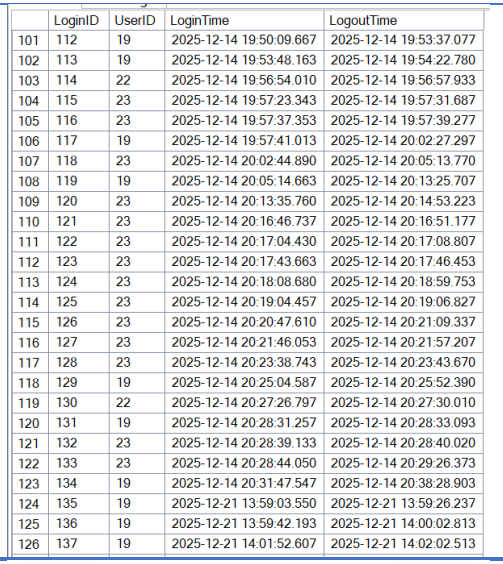
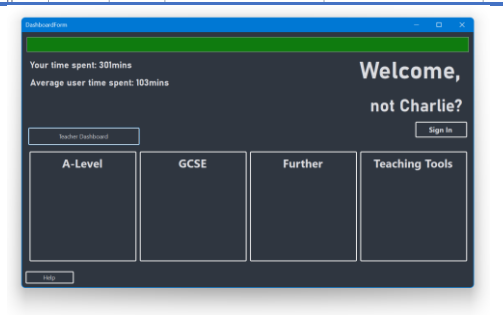
51		Validation message box	Message Box	It is typical that students may not attempt to set the most secure of passwords. However, it is my priority to ensure that passwords used on this application are suitable. Therefore, it must fit with the typical requirements outlined.	A range of passwords that do not fit the requirements.	Typical password requirements must apply when creating an account: 1. At least 8 characters long 2. Contains at least one upper case letter 3. Contains at least one lowercase letter 4. Contains at least one number 5. Contains at least one special character	9.4.1, 9.4.2, 9.4.3, 9.4.4, 9.4.5	 	**Pass**

								 SignUpForm Welcome, please fill in the following details... Firstname test Surname Your password must have a lowercase letter in OK Sign Up  SignUpForm Welcome, please fill in the following details... Firstname test Surname Your password must have an uppercase letter in OK Sign Up	

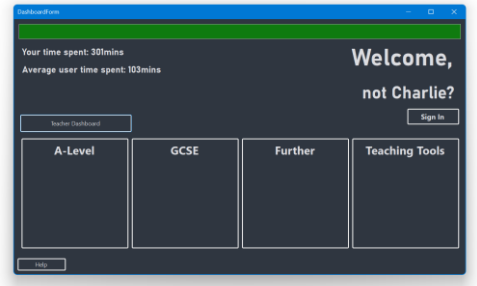
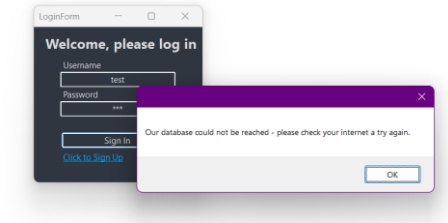
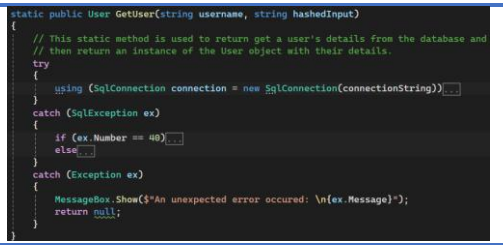
									
52		Sign-Up link	Message Box	In addition to telling a user that they do not already have an account with a certain username. It must be easy for them to find the link to the sign-up window.	A username with no associated account.	A link for the sign-up window should be logically placed on the log-in page.	9.5		**Pass**

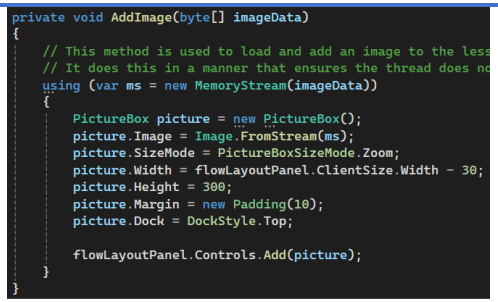
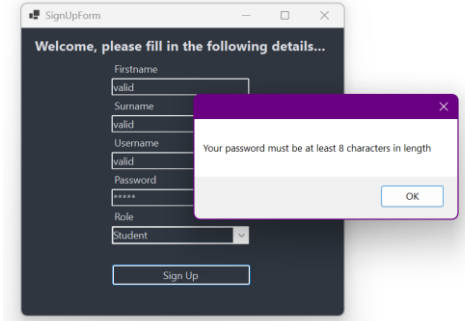
53		The Password box in the Sign-Up page	Text Box	It is important to protect users from shoulder surfing where possible.	Any string to be placed in the password box	When a user enters a password on either the log-in or sign-up page, the password should not be visible – for each character, the '*' symbol must appear.	9.6		**Pass**
54		The Log-In and Sign-Up pages	Text Box, SignUp Form and LogIn Form	SQL prevention is a real risk; therefore, measures should be taken to avoid it.	Attempts to inject, followed by a look into the code behind the queries on both the log-in and sign-up pages.	Appropriate measures should be taken to prevent SQL injection on both pages	9.7	 Using the command.parameters.addwithvalue will prevent SQL injection. The alternative to this would be string interpolation of the values.	**Pass**

55		The General Dashboard's Progress Bar	General Dashboard Form and LogIn Form	Keeping users motivated is important; therefore, there must be a progress bar on the general dashboard based on time spent. This will also be useful for their own acknowledgement of their time spent on the application.	A new user should be created and left with the application open for a given time (which is recorded). This time should then be compared to the time that it says they have been on the application. In addition, the average should be checked manually using the SQL entry viewer on SSMS.	The progress bar will successfully display a user's time spent on the application compared to the average.	10.1	 	**Pass**

56		The General Dashboard's Progress Bar and all Closing Application buttons (including Sign In buttons).	Login Form, Base Page Form	The audit log should be updated appropriately for the progress bar to work and for 10.1 to be successful.	Log in and out at various times (recorded) and compare them to the values stored in the table.	The users' log-in and log-out times must be stored in an audit log.	10.1, 10.2		**Pass**
57		The General Dashboard's Progress Bar	General Dashboard Form	The progress bar will be relative to all users' progress; therefore, it must be able to successfully and dynamically plot different times.	A range of different average log-in and user-specific totals must be plotted using this system.	The progress bar should show the user compared to the average with textual data to confirm this. The max value must be at least 2x the mean; any outliers should be plotted at the 100% mark.	10.3		**Pass**

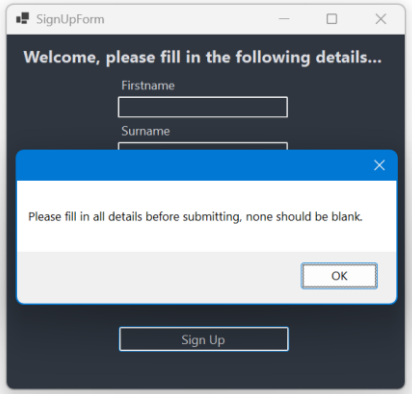
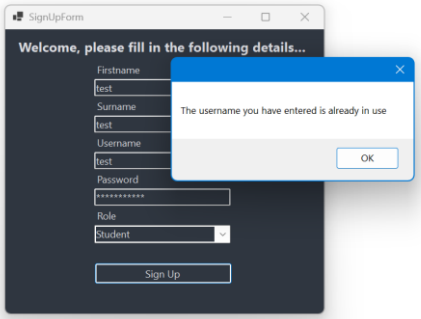
	LoginID	UserID	LoginTime	LogoutTime
101	112	19	2025-12-14 19:50:09.667	2025-12-14 19:53:37.077
102	113	19	2025-12-14 19:53:48.163	2025-12-14 19:54:22.780
103	114	22	2025-12-14 19:56:54.010	2025-12-14 19:56:57.933
104	115	23	2025-12-14 19:57:23.343	2025-12-14 19:57:31.687
105	116	23	2025-12-14 19:57:37.353	2025-12-14 19:57:39.277
106	117	19	2025-12-14 19:57:41.013	2025-12-14 20:02:27.297
107	118	23	2025-12-14 20:02:44.890	2025-12-14 20:05:13.770
108	119	19	2025-12-14 20:05:14.663	2025-12-14 20:13:25.707
109	120	23	2025-12-14 20:13:35.760	2025-12-14 20:14:53.223
110	121	23	2025-12-14 20:16:46.737	2025-12-14 20:16:51.177
111	122	23	2025-12-14 20:17:04.430	2025-12-14 20:17:08.807
112	123	23	2025-12-14 20:17:43.663	2025-12-14 20:17:46.453
113	124	23	2025-12-14 20:18:08.680	2025-12-14 20:18:59.753
114	125	23	2025-12-14 20:19:04.457	2025-12-14 20:19:06.827
115	126	23	2025-12-14 20:20:47.610	2025-12-14 20:21:09.337
116	127	23	2025-12-14 20:21:46.053	2025-12-14 20:21:57.207
117	128	23	2025-12-14 20:23:38.743	2025-12-14 20:23:43.670
118	129	19	2025-12-14 20:25:04.587	2025-12-14 20:25:52.390
119	130	22	2025-12-14 20:27:26.797	2025-12-14 20:27:30.010
120	131	19	2025-12-14 20:28:31.257	2025-12-14 20:28:33.093
121	132	23	2025-12-14 20:28:39.133	2025-12-14 20:28:40.020
122	133	23	2025-12-14 20:28:44.050	2025-12-14 20:29:26.373
123	134	19	2025-12-14 20:31:47.547	2025-12-14 20:38:28.903
124	135	19	2025-12-21 13:59:03.550	2025-12-21 13:59:26.237
125	136	19	2025-12-21 13:59:42.193	2025-12-21 14:00:02.813
126	137	19	2025-12-21 14:01:52.607	2025-12-21 14:02:02.513

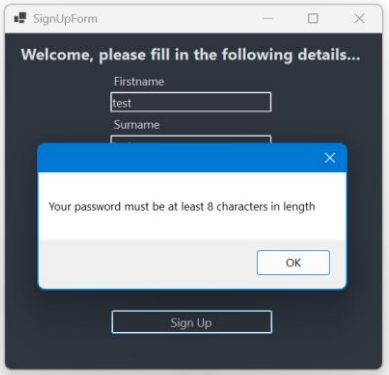
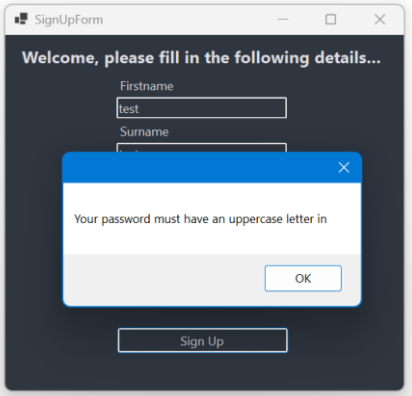
58		The General Dashboard's Progress Bar	General Dashboard Form	The application must be responsive (especially on low-powered computers); therefore, the progress bar must not be too computationally intensive.	A look at the code and a runtime test must occur.	The average total for each user should be computed efficiently *(perhaps using aggregate SQL)*, as this process mustn't slow the application down.	10.4		**Pass**
59		Message Boxes on all pages	All forms classes *(tested on the login form)*	The user should not be shown the complexities of the database; instead, when there is an issue, it should be abstracted through a brief message.	An invalid connection string while logging in.	A brief message should pop up if there is an issue with the connection to the database.	11.1		**Pass**
60		N/A	All forms *(tested on the lesson form)*	It is likely that for a range of other reasons that the queries may lead to a crash.	An invalid query (an incorrect table name in the lesson fetching).	All classes with SQL queries should execute them within try-catch blocks.	11.2		**Pass**

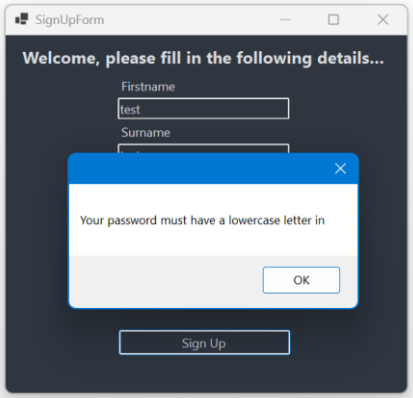
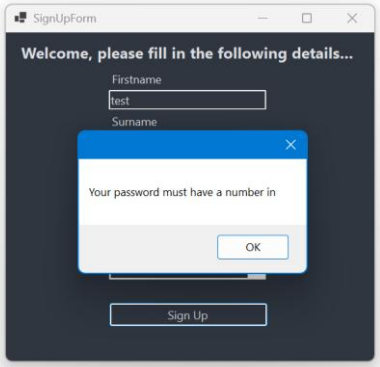
61		Lesson Page (images)	Lesson Page	Media can be slow to load due to high quality and size.	A large image attached to a lesson and a code inspection.	Media, such as images in the lesson pages, should be loaded in an efficient manner.	11.3	 Using memory stream addresses this issue.	**Pass**
62		Message Boxes on all pages	All forms *(tested on the demos and sign-up form)*	Although it is helpful to flag errors, it is also just as helpful to show how these can be corrected in simple language.	Invalid fields in the sign-up window and a teaching tool.	Error pop-ups should show how the user can correct the issue in non-technical language.	11.4		**Pass**

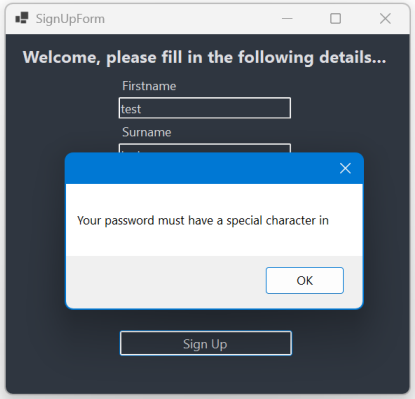
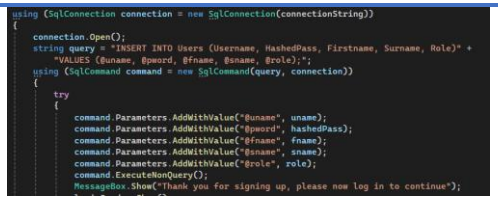
Validation tests

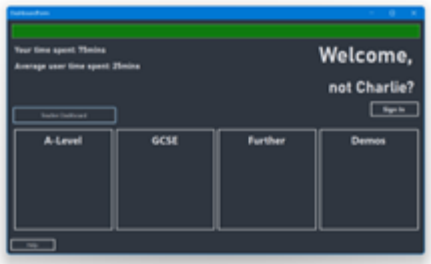
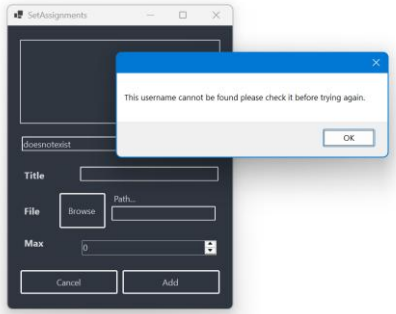
Some of these tests have already been covered as part of my generic tests above; however, the validations that I drafted up in the Design section are key to the robustness of my program. Therefore, I will go over these validations, ensuring that all have been met.

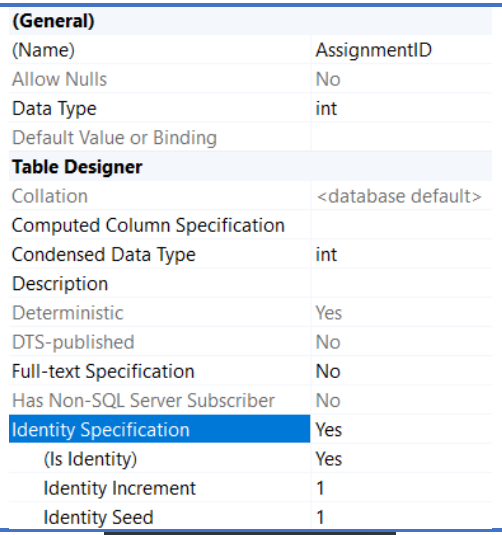
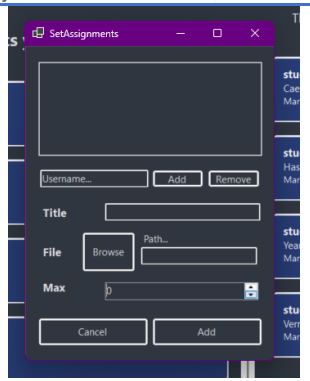
Test No.	Test description:	Data used:	Validation Objective met:	Evidence:	Pass / Fail:
63	Presence check ~ this check should occur to ensure that all fields of username, password first name, etc are not empty or null.	**Erroneous Data** A range of empty and non-empty fields.	1.1.1		**Pass**
64	Uniqueness check ~ the username inputted when creating an account must not already be in use	**Erroneous Data** A non-unique username.	1.1.2		**Pass**
65	Password criteria ~ a check must occur to ensure that the password is of the correct strength	**Erroneous Data**	1.1.3	See screenshots below…	**Pass**

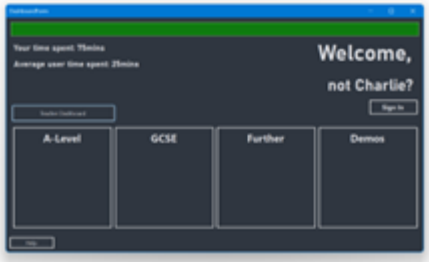
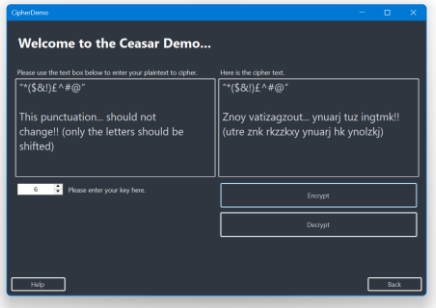
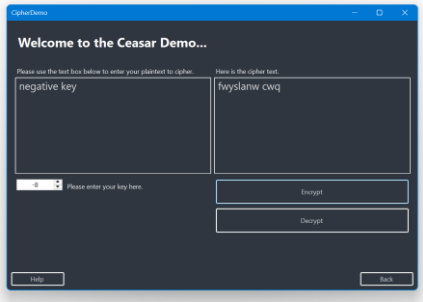
			A range of correct and incorrect fields.			
66		Minimum length of 8 chars	**Boundary Data** A 6-digit password.	1.1.3.1		**Pass**
67		Must contain at least one uppercase letter	**Erroneous Data** A password with no uppercase letters.	1.1.3.2		**Pass**

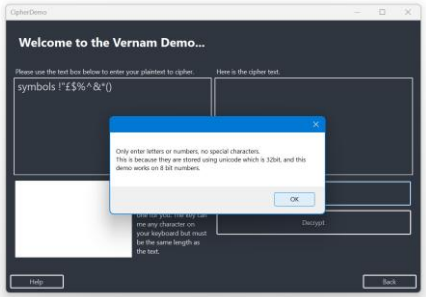
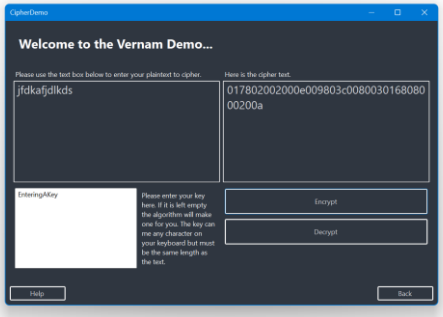
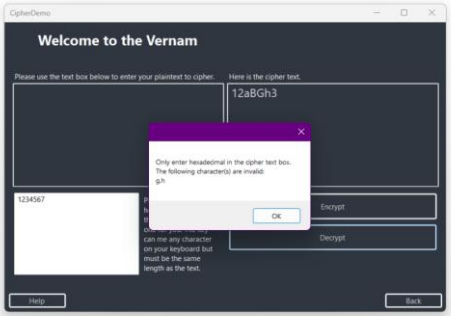
68		Must contain at least one lowercase letter	**Erroneous Data** A password with no lowercase letters.	1.1.3.3		**Pass**
69		Must contain at least one digit (0-9)	**Erroneous Data** A password with no numbers in.	1.1.3.4		**Pass**

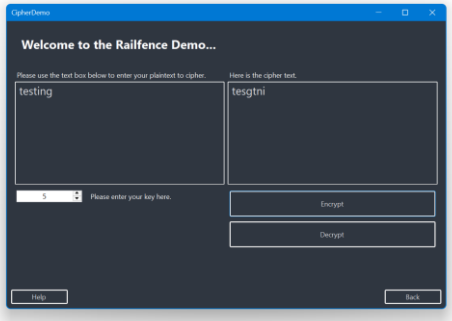
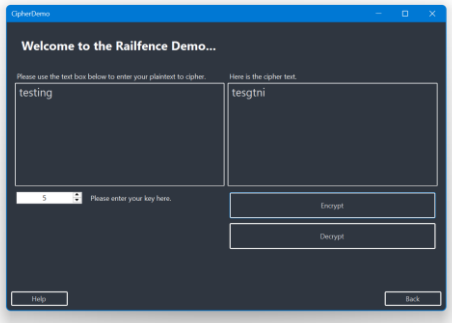
70		Must contain at least one symbol	**Erroneous Data** A password with no symbols.	1.1.3.5		**Pass**
71		SQL injection prevention ~ parameterised queries must be used to validate the input	**Erroneous Data** Attempts to inject, followed by a look into the code behind the queries on both the log-in and sign-up pages.	1.1.4	 Using the command.parameters.addwithvalue will prevent SQL injection. The alternative to this would be string interpolation of the values.	**Pass**

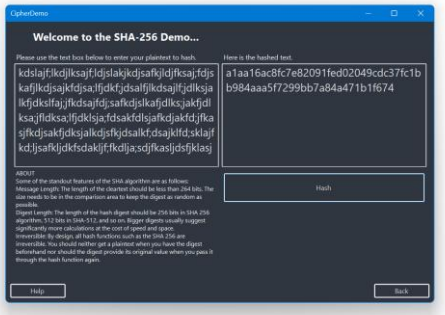
72		The user's role must be validated before allowing access to the teacher dashboard	**Erroneous Data** A teacher and a student's login details (to view the general dashboard).	1.2.1		**Pass**
73		The username must be verified, ensuring that it exists, before adding to the assignment	**Erroneous Data** A teacher's login details, plus valid and invalid students' usernames.	1.2.2		**Pass**

74		Seeds must be used to prevent an assignment being created with a non-unique ID	**Erroneous Data** A check on SSMS.	1.2.3		**Pass**
75		The max mark must be a positive integer	**Boundary Data** A negative max mark was attempted to be input.	1.2.4	 This numeric selector has a property that allows me to ensure that a value less than 0 cannot be entered. I have used this to fulfil this validation.	**Pass**

76		Outlier handling ~ if a user's total time spent is greater than 2x the average, they should be plotted at the 100% mark rather than attempting to plot it at the 100+% mark.	**Boundary Data** I created a user with a time spent of greater than 2x the average, to see if the progress bar is filled to a suitable level.	1.3.1		**Pass**
77		Non-alpha chars ~ if found in the Caesar cipher's input, they must not be shifted	**Erroneous Data** Any string that contains a range of non-alphabetic characters eg) "*($&!)£^#@"	2.1.1		**Pass**
78		Key validation ~ the key should be properly accounted for regardless of its sign (positive or negative)	**Boundary Data** A negative key text pair with a known encrypted result.	2.1.2		**Pass**

79		Key and plain text ASCII char range ~ inform the user if and which char in the text or key has a value greater than 256	**Boundary Data** Plain text and key pairs with known results from the encryption.	2.2.1		**Pass**
80		Ensure the key length is the same as the plain – if this is not the case the user should be told	**Erroneous Data** Some plain text and key pairs of varying lengths.	2.2.2		**Pass**
81		Decrypting non-hexadecimal cipher text ~ if the user enters any values which are not hexadecimal in the cipher text, they should be told so	**Erroneous Data** A non-hexadecimal cipher text.	2.2.3		**Pass**

82		The key must be validated to ensure that only a positive integer key can be entered	**Erroneous Data** A non-numeric key and a negative key.	2.3.1	 The numeric up down feature allowed me to add a range to the key.	**Pass**
83		Range check ~ the key must be greater than or equal to 1	**Boundary Data** Attempted to enter the key -1.	2.3.2	 The numeric up down feature allowed me to add a range to the key ensuring its greater than or equal to 1.	**Pass**

84		Length check ~ ensure that the output is exactly 64 hexadecimal chars – if necessary, padding should be applied	**Boundary Data** A range of different length of plain text (extreme values should be used). A non-numeric key and know results of key text pairs.	2.4.1		**Pass**

Evaluation

Summary

This program has been a success; it will be useful for both teachers and students, saving them time and aiding their learning. The application has fulfilled the suitability for a range of learning types and allowed for independent learning to occur. Each lesson was created to be stand-alone and features text-to-speech for auditory learners (and visually impaired); images for visual learners; work prompts for the social learners and the chatbot for the more practical independent workers. The teachers will benefit from being able to send resources and receive marks from students, while also being able to actively demonstrate the function of cryptographic algorithms. In addition, the chatbot does provide great help for students, allowing them to discuss cryptography and actively pursue knowledge. It is very well suited for independent learners who need to clarify thoughts and understanding quickly – as opposed to using the internet. The fact that the chatbot can guide the users to further resources will enhance the effect and use of this feature for students. Finally, the progress bars do allow for the user to gauge how they are doing on the application and the time that they are denoting to learning. It creates a game-like feel to the application and would motivate me to do work – so I do believe it will help many others gain motivation.

Objectives & Completeness of Solution

Objective	Evaluation	Test No	Condition met?
1.1	All learner types have been very much accounted for – especially the key ones. The tool accompanies the following: - Visual through images and demonstrations - Practical through work/discussion prompts and demonstrations - Auditory through the speech synthesis - Discussion-based through discussion prompts and the chatbot - Textual/reader-based learners through lesson pages	1	**Pass**
1.2	This was met very well through planning the user interface, creating buttons for each section and selection menus for each exam board.	2	**Pass**
1.3	A student is able to go through this application from start to finish with no assumed knowledge and completely understand each lesson/topic. Therefore, I have completely fulfilled this objective.	2	**Pass**
1.4	For the most part teachers and students have few differences relating to the UI. They are both able to access all of the same areas – apart from the teacher dashboard or assignment viewer (the elements relating to assignments). As a result, a teacher can view the lessons and a student can use the teaching tool to aid their understanding.	3	**Pass**

2.1	This objective has been completely met as the student can see their progress on the general dashboard, with the easy-to-understand progress bar. In addition, below the bar is a comparison of the average total time spent on the application and the current users total time spent of the application.	4	**Pass**
2.2	By using a selector in the lesson navigational page, I have met this objective. Before a user can view the lessons for their syllabus, they must select the level on the dashboard and then the exam board in the navigational page.	5	**Pass**
2.3	The help button is a stock part of each page; therefore, the user can always ask the chat bot a question. This meets the objective, especially because the LLM is fully functional as mentioned in later objectives.	6	**Pass**
2.4	This objective is fully fulfilled as the current user is welcomed and confirmed displaying an option to sign in to a different account.	7	**Pass**
2.5	By pressing the teacher dashboard button, the teacher dashboard button from the general dashboard, the teacher dashboard is opened. Note that the button is only visible when a teacher is signed in.	8	**Pass**
2.6	The encryption tool and lessons can be accessed by navigating through the dashboard and lesson pages. As a result, this objective has been well met.	9	**Pass**
2.7	This objective has been met well as all buttons are large and can be easily used by touch screen devices.	10	**Pass**
3.1	The assignment set by a teacher can be viewed easily on the teacher dashboard. Each assignment is represented as bubbles containing the important information needed to distinguish the tasks, so this objective has been well met.	11	**Pass**
3.2	Assignment and teacher features can be accessed by using buttons on the teacher dashboard therefore fulfilling this assignment.	11	**Pass**
3.3	This task is well met as the teacher dashboard button is only visible when the current user is a teacher. Which prevents students for being able to set assignments.	8	**Pass**
3.4	The recently set assignments are show in bubbles with key information such as the title and date set therefore meeting this objective solidly.	11	**Pass**
3.5	Recent results can be seen on the dashboard through per person bubbles which show how different students have recently performed in the assignments, meeting this task.	12	**Pass**

4.1	This was well met due to the dynamic UI generation: a student sees a different assignment viewer to the teacher (who has their teacher dashboard). On these different assignment viewers different actions – relating to their roles – can be completed.	8	**Pass**
4.2	The teacher must upload documents to each assignment before they can submit it, therefore meeting this objective well.	13	**Pass**
4.3	This objective has been firmly met due to the intuitive and robust username selection system.	14	**Pass**
4.4	If the username does not exist the teacher is told therefore completing this task.	14	**Pass**
4.5	Similarly, this task has been met by validating each username and telling the teacher if the username has already been added.	15	**Pass**
4.6	This objective is met as the teacher can easily remove a student by username, through entering their name and pressing the remove button instead of the add button. This is validated so that they are told if the user was not on the list.	16	**Pass**
4.7	This is well met as the teacher is prompted for a maximum mark to be sent with the task. If left blank the teacher is told of this.	17	**Pass**
4.8	This has been fulfilled well through the assignment viewer window which dynamically loads all assignments set for a user. Each assignment is placed in a bubble which contains various actions for each assignment such as downloading the file or uploading marks.	18	**Pass**
4.9	As mentioned above they users can download files and upload marks using buttons that are dynamically created for each bubble. This completes the objective solidly as the database is also updated correctly.	18, 19	**Pass**
4.10	When the marks are uploaded the database is updated correctly, and the result can be correctly viewed on the teacher dashboard, thus meeting this objective.	19	**Pass**
5.1	This intuitive user interface has been created by implementing the lessons page with each lesson sorted in a nested manner as mentioned in this objective. This completes the objective, and the use of the LLM will add to this if users get stuck.	20	**Pass**
5.2	This objective has been met – such that any user can go through the lessons from start to finish with no extra knowledge needed.	21	**Pass**
5.3	When storing lessons in the syllabus table there is a non-nullable attribute relating to the lesson number. This ensures that each lesson has been given an appropriate number, thus accomplishing this aim.	21	**Pass**

5.4	In addition, there is a non-nullable title attribute in the defined lessons table, therefore reaching this target.	21	**Pass**
5.5	This objective has been well fulfilled because before lessons are queried the user has to select a given exam board for their level.	20	**Pass**
5.6	Content is not duplicated, which meets this objective. This is because I have created a table for defined assignments and then a table for the assignments needed for each syllabus. This creates a perfectly normalised database and then any extra details for different levels can then be added in a separate lesson.	22	**Pass**
5.7	All lessons – from images to text and work prompts – are loaded dynamically through adding forms objects to the flow layout panel, which are created using the data queried from appropriate tables. This thus ensures the objective has been completed.	23	**Pass**
6.1	This has been met as the longest lesson only contains 99 words. The lessons have been created using the spec thus are factually correct and contain only the necessary information.	24	**Pass**
6.2	This overall aim has been met by completing all of the following 4 objective.	1, 25, 26, 27, 28	**Pass**
6.2.1	The lesson pages each contain an image to aid learning, the image is stored inside the defined lesson table as binary data. Therefore, meeting this objective.	25, 1	**Pass**
6.2.2	I used speech synthesis to complete this task. I implemented a button that when pressed reads the text aloud to the user.	26,1	**Pass**
6.2.3	A help button was added as a stock part of all pages (using inheritance). On the lesson page is acts the same as elsewhere – it opens the help window for users to query a chat bot. This objective has been fulfilled.	27, 1	**Pass**
6.2.4	I completed this by incorporating a non-nullable work prompt attribute to each column. This work prompt is then displayed to encourage learning for social learners.	28, 1	**Pass**
6.3	Through inheritance, and dynamically loading each lesson, the help, navigational and text-to-speech buttons are visible on all lesson pages, thus meeting this aim.	29	**Pass**
6.4	The text size on all pages – most specifically the lesson page – has been chosen by viewing the window in a variety of different display sizes.	30	**Pass**
7.1	This has been met as the user is able to select appropriate teaching tools and demonstrations of encryption techniques in a navigational page, after they have pressed the teaching tool button on the general dashboard.	31	**Pass**

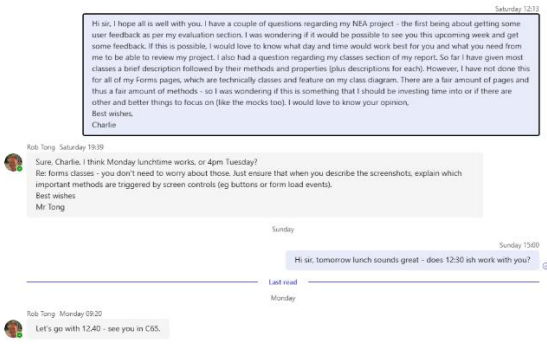
7.2	A Caesar cipher has been incorporated into the teaching tools.	32, 33	**Pass**
7.2.1	I ensured that any type of textual data can be added as input for this cipher, as shown in the appropriate test for this section.	32	**Pass**
7.2.2	Following on from the above, when the user enters a non-alphabetic character, a shift will not be applied to it – simply because it is not in an alphabet. I implemented this into the function of the algorithm.	32	**Pass**
7.2.3	The results produced by this cipher have been validated in my tests and shown to be correct and displayed as a string with all character's show.	32	**Pass**
7.2.4	Negative keys have been accounted for through careful development of the algorithm. Thus, the user can enter a negative key.	33	**Pass**
7.3	A Vernam cipher has been implemented as part of the demonstrations.	34, 35, 36, 37, 38	**Pass**
7.3.1	The cipher does produce a correct result for any key text pair given that none of the characters entered are represented with more than 8-bits. Therefore, the user is warned if they enter any non-ASCII represented characters.	34	**Pass**
7.3.2	When the user leaves the key blank a random key will be used and displayed, thus meeting this objective.	35	**Pass**
7.3.3	The user is able to input a key using the text box and appropriate validation does occur, ensuring it is of the same length as the plain text.	36	**Pass**
7.3.4	The result of this cipher is displayed in hexadecimal, therefore fulfilling all the Vernam cipher objectives.	37	**Pass**
7.3.5	The user should not be able to enter a non-hexadecimal result into the cipher-text box and press decrypt. They must be told of the incorrect chars and the upper case must be automatically replaced with lowercase.	38	**Pass**
7.4	A Railfence cipher has been implemented as part of the further/extension ciphers.	39, 40	**Pass**
7.4.1	It is not possible for the user to enter a non-integer key.	39	**Pass**
7.4.2	A string with any type of characters should be accepted, this is because this cipher reorders the items rather than changes how the items are represented.	40	**Pass**
7.5	A Hashing algorithm has been incorporated as part of the teaching tools and is used to hash the user's passwords, for storage in the database.	41, 42, 43	**Pass**

7.5.1	No matter the length of the input the hash does produce a 64-digit hexadecimal result, thus meeting this objective.	41	**Pass**
7.5.2	All has results are returned as hexadecimal as mentioned above.	42	**Pass**
7.6	This has been met as each tool (except the hash as this is not possible) has an encryption and decryption process/button, therefore allowing the user to perform both types of operations.	43	**Pass**
7.7	This has been met as the help and navigational buttons are visible on all of these pages, done so by inheritance.	43, 29	**Pass**
8.1	This aim has been met as the help screen opens in a separate window, thus not affecting the lesson or task currently being completed.	44	**Pass**
8.2	The page is closable at any time and features a modern layout that aesthetically ties in with the rest of the program, this meets the objective.	45	**Pass**
8.2.1	The above it also met through this sub-task. I used rounded buttons, contrasting colours and a dark (eye-friendly) colour palette, for the pages.	45	**Pass**
8.3	The LLM does provide further resources and lets the user know of it through its welcoming message.	46	**Pass**
8.4	The help page functions exactly the same no-matter where it is called from, therefore accomplishing this aim.	47	**Pass**
8.5	The LLM can direct the user around the application if they get stuck.	48	**Pass**
9.1	Data is collected as successfully stored in the database when a user signs up to the application.	49	**Pass**
9.1.1	The user's first name is stored in the table.	49	**Pass**
9.1.2	The user's surname is also collected and stored.	49	**Pass**
9.1.3	The username entered is checked for uniqueness and then stored.	49	**Pass**
9.1.4	The password is validated for secureness and the hashed before being stored in the database.	49	**Pass**
9.1.5	The role is collected and stored in the user's table. I stored it as a string to allow me to add different types of users in the future. Note that the user does not enter their role, instead they must choose it from a discrete number of strings in a selection box.	49	**Pass**

9.2	As mentioned above this has been met as the user's password is hashed before being stored in the database.	50	**Pass**
9.3	All details should be stored appropriately in a database – if any information is incorrectly entered in the sign-up page, then the user must be told what the issue is and required to fix the issue before re-trying and creating the account. The following objectives categorize what is incorrect…	51	**Pass**
9.3.1	This has been met as when the create user button is pressed the database I queried with the username. If already present on the database, then the individual is prompted to make a new username.	51	**Pass**
9.3.2	If any information is left empty the user is told about this.	51	**Pass**
9.4	Along with the information above the password must also be validated. In the project I met this objective by strictly validating the password, if it was not to standard the user is required to re-enter/fix the data so it is suitable. The validations that must be carried out are as follows…	52	**Pass**
9.4.1	A length check was run on the password ensuring it is longer than 8 characters. The user is prompted if the password is not of correct length.	52	**Pass**
9.4.2	The user is prompted if the password does not contain more than one uppercase character – they are then required to re-enter a better password.	52	**Pass**
9.4.3	Similarly, this objective was met by ensuring that the password contains at least one lowercase letter, if not the user is prompted and required to re-enter their password.	52	**Pass**
9.4.4	The password is validated so that it must contain at least one letter, this is what allowed me to fulfil this objective.	52	**Pass**
9.4.5	The password must contain at least one symbol so this is fulfilled by validating this.	52	**Pass**
9.5	A label link is added to the log-in page that when pressed will open the sign-in page. It is through this label that I met this aim.	53	**Pass**
9.6	The password is not visible but instead replaced with the '*' characters whenever entered.	54	**Pass**
9.7	SQL injection has been prevented by using the '.AddParameter()' function on the query object.	55	**Pass**
10.1	I met this objective by creating a progress bar on the dashboard that displays the time the user has spent on the application – compared to that of the average user.	56, 57	**Pass**

10.2	The users log-in and log-out times have been successfully added to the audit log, I met this objective and it is visible through the progress bar and testing.	57	**Pass**
10.3	The progress bar shows the user compared to the average user and will plot a progress of up to 2x the average. This ensures that outliers are handled correctly. Through this I have met the objective.	58	**Pass**
10.4	The average for each user has been computed using aggregate SQL to ensure that processing occurs at the server and not on the user's computer (client) – thus not slowing it down.	59	**Pass**
11.1	When the database is not available the user is told, therefore, if there is a network error on the school the user can be made aware of it without the application crashing (potentially for an unknown reason).	60	**Pass**
11.2	All queries are embedded inside try catch blocks to ensure that if there is an issue the user is told of the issue. I meet this objective and extend it by informing the user of what the issue is.	61	**Pass**
11.3	Media – such as images – is loaded efficiently using async to ensure that the user can view the rest of the page while the single elements load.	62	**Pass**
11.4	Most errors do contain guidance on how to solve the issue in a non-technical manner, so that even the most non-technical users can fix the issue. Therefore, this aim has been completed.	63	**Pass**

User Feedback



I held an in-person meeting with my client to discuss both the positive areas of this project and some potential next steps to improve it further.

Below, I will outline each of these, explaining what I took from their advice. All changes that I would make from this information are mentioned separately under the Improvements section; this section is only concerned with outlining what was said.

FIGURE 33 – EVIDENCE OF THE MEETING WITH MY CLIENT

Overall thoughts

Ultimately, my client did think that the application would be extremely helpful. He added that he has not yet had access to teaching tools covering these topics, and thus, this solves a key issue for him and potentially will for other teachers. As a result, this will help my client with explaining the difficult encryption concepts, which is the key aim of the application. He also considered the students' perspective, mentioning that the application provides a wealth of resources for them to clarify and develop knowledge, while being accommodating and interactive, which is essential for keeping students engaged.

Key positive aspects

The first point my client made was that the application gives great attention to all types of learners. They noted that it was very evident that different people had been considered for the application, due to all the different features inside the lesson pages. As a result, I will look to keep this aspect during further development – as I add new sections and features, I must continue to consider different types of people.

Secondly, he said that the content is well-tailored so that students aren't overwhelmed with unnecessary content. I agree with this comment as a student myself. I am keen to break down the information into small and digestible chunks where I only retain the key information needed for exams. My takeaway from this was to ensure lessons remain brief as I expand the content that this tool covers (adding more lessons).

Thirdly, my client found great interest in the chatbot. He agrees with my point that there are two types of students: those who ask questions to ensure they fully understand every detail and those who tend to ignore their confusion. Therefore, agrees that this feature helps to bridge the gap, making it easier for the second type of students to find answers to their questions and encouraging independent learning. This has shown me the great use of this section of the application; thus, in the future would look to see how I can push the use of this further *(see later sections for concepts)*.

The next comment they added was on the flexible design of the application. He mentioned that the design supports change, as it was built in a modular way with inheritance used appropriately. This extends further with the database design, which allows for new exam boards and lessons to be easily added. He added that 'part of this flexible design comes from the dynamic nature of all content'.

My client pointed out that the application allows a user to practically 'play' with the key cryptography algorithms. This allows them to see how various inputs are reflected in the output – perhaps in a systematic way (or not with hashes). From this, I saw that this section was useful to the teacher for explanations, but mainly for the user, with 'playing' with the algorithms. This conversation moved on to further talks of how I can ensure that it is more useful for teachers, as originally planned *(see later sections for concepts).*

The final area that my client complimented was the ease of gauging progress, not just for teachers being able to take in marks, but also for the students' understanding of where they sit. However, he did add that a focus would be to take this a step further – it is a useful feature but does not entirely help for tracking progress *(more on this in later sections).* From this, I can see that the ability to follow progress is much more useful than I would have thought.

Areas for further development

The first area was on improving the most essential part of this application, which was the demonstrations or teaching tools. My client saw great use in the tools for conveying the relationship between inputs and outputs to a cryptographic algorithm. However, they added that it does not show too much of what is happening behind the algorithm, which is the key area that they need to explain and perhaps is arguably the hardest thing to explain. He did add that the lessons do this sufficiently, even with the images, but lack a visual step-by-step walkthrough.

Secondly, he added that not all schools will have hardware suitable for running the application, which is mostly not due to performance, but also to operating systems or software. He said that it would be useful to also provide a web environment to go along with the application. Thus, users can access the tool from anywhere.

Thirdly, my client added that the selection process of the assignment system could be improved by also organising students into groups or classes. I agree with this, and we discussed an extension of it where lessons could be set for groups of students. This means the teacher does not have to find their own resources for homework or self-study.

We continued further on this idea by talking about quizzes. My client showed lots of enthusiasm over the idea of students being able to complete pre-existing assignments or quizzes. He mentioned the platform SmartRevise, which has a collection of pre-written questions for users to complete; each question has an option to self-mark or mark using AI. The flashcards can then be reviewed at regular intervals.

Another point raised was to do with the colours of the application. My client finds them appealing and suitable for most individuals – however, not all individuals. Thus, he thought it would be appropriate to add a settings page for the visual elements of the application. This could include text size, background colours, foreground colours and more. I agree with this idea, and this can be easily added given the inheritance system of my forms classes.

The final key idea was on expanding the application's scope. Rather than just covering the encryption section of both computer science A-level and GCSE, it could be extended (again alike SmartRevise) to cover all the content that can be found in the specification. This would not be too hard to add as the basic interface of the application is there and can be built upon. Another difficult topic mentioned was Boolean algebra (perhaps a demo or quiz could be created for that).

Many other ideas discussed above are the key ones, but some honourable mentions are as follows:

- Improving the detail in the Asymmetric encryption lesson

- Training separate chatbots on lesson details specific to exam boards, which ensures that only the necessary and correct information is mentioned by the chatbot corresponding to the user's exam board.
- Managing overwork – identifying when a user has spent a prolonged period on the application, and thus at a computer.

Improvements

In this section, I am going to cover each area for further development and discuss how I am going to improve this area within my application.

Firstly, is showing the processes behind each encryption process. I can easily split each algorithm into 2-4 key processes (eg for Vernam, the XOR operation would be one). For each of these processes, I can then create a visual feature displaying how it works/occurs. These visual elements can be layered one after another in a scrollable page, where each element updates dynamically based on the input entered. This can be done by altering the demo's specific forms page.

Secondly, my application can be launched as a web application with minimal changes by using ASP.NET. The advantage of this is that the function will not differ hugely between being run on the user's browser or as a desktop application. In addition, my code can be reused as ASP.NET is built upon C# and integrates well. Finally, ASP.NET would also allow for a mobile-responsive version, which would be useful for students who want to study small amounts of content often (like regularly going through flashcards on the go).

Thirdly, two new tables could be added to allow students to be grouped in classes. A LinkedClass table for storing UserIDs and corresponding ClassIDs, and a DefinedClass table for storing information such as the associated teacher, and other useful information. Then, a new page can be linked to the teacher's dashboard, where they can alter classes. I would also have to create a ClassOfStudents class to store the appropriate information for groups of students.

The quizzes could be added as a different section of the application. Within the level-specific (eg GCSE) lesson selector page, a link to relevant quizzes could be added. Each quiz can be stored in a table in the database, which is linked to another table containing questions and their corresponding answers. There could be written questions with associated mark schemes, each of which can be marked by the student or by an AI – perhaps the LLM used.

In addition, a settings button can be added to the general dashboard, which opens a page containing useful configurations; one of these will be the colour theme. As I have used inheritance for most of my forms pages, it would be easy to implement two variables which control the background and foreground colours. Then, during the InitalizeComponents subroutines, the hardcoded colours can be replaced with the variables containing the hexadecimal colours. Other settings can be added to this, such as a float for scaling the text across the application – this could be implemented similarly as above.

Finally, the scope of the application can easily be developed. I would look to split the lessons into further categories (such as Boolean algebra), which would need another table in the database for storing the topics. As I have already built the basic UI and functionality of the application is there. If I covered all topics for each exam board, it would be useful to create a Topics table which lines lessons to topics, and then in the Syllabus table, topics can be linked to lessons.